



Análisis y Diseño de Sistemas I

Curso	Análisis y Diseño de Sistemas I (2392)
Formato	Manual de curso
Autor Institucional	Cibertec
Páginas	126 p.
Elaborador	Guzmán Mariluz, Gisella
Revisor de Contenidos	Morales Flores, Gustavo

Índice

Presentación	5
Red de contenidos	6

UNIDAD DE APRENDIZAJE 1: INTRODUCCIÓN A LA INGENIERÍA DE SOFTWARE

1.1 Tema 1 : Ingeniería de software, AUP y UML	8
1.1.1 : Elementos claves de la ingeniería de software	8
1.1.2 : Procesos de software	10
1.1.3 : Modelos de proceso de software	11
1.1.4 : Metodologías de desarrollo	12
1.1.5 : Metodología RUP (Rational Unified Process)	15
1.1.6 : Metodología AUP (Agile Unified Process)	18
1.1.7 : UML (Lenguaje de Modelamiento Unificado)	26
1.1.8 : Diagramas de UML 2.5	28
1.2 Tema 2 : Herramientas CASE	31
1.2.1 : Objetivos de las herramientas CASE	31
1.2.2 : Tipos de herramientas CASE	31
1.2.3 : Ejemplos de herramientas CASE	32

UNIDAD DE APRENDIZAJE 2: DISCIPLINA DE MODELADO – PARTE I

2.1 Tema 3 : Modelado de negocio	39
2.1.1 : ¿Cuándo es necesario realizar el modelado de negocio?	40
2.1.2 : ¿Cuándo no es necesario realizar el modelado de negocio?	40
2.2 Tema 4 : Visión general del entorno CASE IBM Engineering Systems Design Rhapsody	42
2.2.1 : Proyectos, vistas y toolbars	42
2.2.2 : Toolkit para ingeniería de sistemas	44
2.2.3 : Tips de navegación	45
2.2.4 : Creación de un proyecto con la estructura del modelado	46
2.3 Tema 5 : Modelo de casos de uso del negocio – MCUN	51
2.3.1 : Artefactos del modelo de casos de uso de negocio	51
2.3.2 : Reglas básicas de nomenclaturas	52
2.3.3 : Actividades para identificar los artefactos de la vista externa del negocio	53
2.3.4 : Identificación de artefactos del modelo de casos de uso de negocio para el caso fashion importaciones	53

2.4 Tema 6	: Modelo de análisis del negocio – MAN	66
2.4.1	: Artefactos del modelo de análisis de negocio	66
2.4.2	: Reglas básicas de nomenclaturas	66
2.4.3	: Actividades para identificar los artefactos de la vista interna del negocio	67
2.4.4	: Identificación de artefactos del modelo de análisis de negocio para el caso fashion importaciones	67
2.5 Tema 7	: Caso propuesto de MCUN y MAN	78
2.5.1	: Caso: Play and Learning Zone S.A.C.	78
2.5.2	: Caso: Impulse Center	79
2.5.3	: Otros casos propuestos	80

UNIDAD DE APRENDIZAJE 3: DISCIPLINA DE MODELADO – PARTE II

3.1 Tema 8	: Captura de requisitos	83
3.1.1	: Importancia de la captura de requisitos	83
3.1.2	: Dificultades de la captura de requisitos	83
3.1.3	: Artefactos de la captura de requisitos	84
3.1.4	: Reglas básicas de nomenclaturas	85
3.1.5	: Actividades para realizar la captura de requisitos	85
3.1.6	: Requisitos FURPS+	86
3.1.7	: Técnicas para capturar requisitos	89
3.1.8	: Experiencia de usuario (UX Design)	89
3.1.9	: Captura de requisitos a solicitud del cliente	91
3.1.10	: Identificación de actividades para sistematizar a partir del diagrama de actividades de un proceso - Caso: Venta de productos de fashion importaciones	91
3.2 Tema 9	: Modelo de casos de uso	96
3.2.1	: Artefactos del modelo de casos de uso	96
3.2.2	: Reglas básicas de nomenclaturas	96
3.2.3	: Identificación de artefactos del modelo de casos de uso para el caso fashion importaciones	97
3.3 Tema 10	: Estructurando el modelo de casos de uso	113
3.3.1	: Objetivos	113
3.3.2	: Tipos de relaciones	113
3.3.3	: Casos de uso abstractos y concretos	116
3.3.4	: Especificación de casos de uso -ECU	117
3.3.5	: Priorización de casos de uso	118

Bibliografía	125
---------------------	------------

Presentación

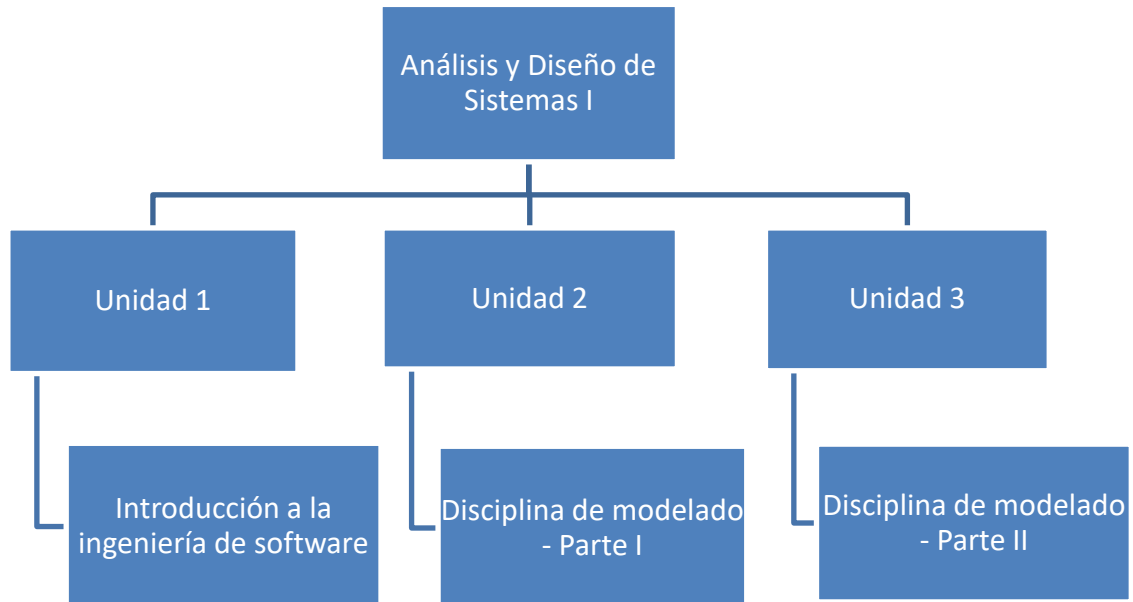
En el mundo de Ingeniería de Software, el análisis y diseño de sistemas son actividades claves antes de pensar en implementar o programar un producto software confiable y de calidad. El avance de nuevas tecnologías en herramientas que asisten al ciclo de vida de desarrollo del software hace que los profesionales de sistemas puedan tomar mejores decisiones, regidos por un enfoque o marco de trabajo, a la hora de brindar una solución tecnológica acorde a las necesidades de una compañía o empresa.

En definitiva, la producción o creación de software utiliza métodos, técnicas y herramientas para desarrollar un producto innovador aplicando las buenas prácticas de las metodologías de desarrollo elegidas. Dicho producto brinda soporte a los procesos de una empresa permitiendo que el trabajador, quien es usuario del sistema, realice sus labores de forma más eficaz y eficiente. Hoy en día las empresas requieren funcionar con un software, el cual determina la calidad de su infraestructura tecnológica y de los productos desarrollados o adquiridos de acuerdo con sus necesidades.

El manual para el curso ha sido diseñado bajo la modalidad de unidades de aprendizaje, las que se desarrollan durante un número determinado de semanas. En cada una de ellas, se indica: (1) los logros que se busca alcanzar al término de cada unidad, (2) el temario a tratar y (3) las actividades que permitirán afianzar los conocimientos adquiridos en cada sesión de clase.

El curso es teórico-práctico, el cual consiste en un taller de desarrollo de proyectos de software. En primer lugar, se inicia con la comprensión de la Ingeniería de Software. Continúa con el proceso de desarrollo de software utilizando el Proceso Unificado Ágil: AUP. Luego, con la presentación del Lenguaje del Modelamiento Unificado: UML. Por último, se desarrolla la primera disciplina de AUP conocida como Modelado, la cual incluye el Modelado del Negocio y la Captura de Requisitos con un enfoque ágil.

Red de contenidos





INTRODUCCIÓN A LA INGENIERÍA DE SOFTWARE

LOGRO DE LA UNIDAD DE APRENDIZAJE

Al término de la unidad, el alumno es capaz de identificar las ventajas y desventajas de los modelos de procesos de software, y la importancia de emplear una metodología de desarrollo de Software como el AUP. Finalmente, el alumno es capaz de elaborar los diagramas UML y se familiariza en el uso de una Herramienta CASE.

TEMARIO

1.1 Tema 1 : Ingeniería de software, AUP y UML

- 1.1.1 : Elementos claves de la ingeniería de software
- 1.1.2 : Proceso de software
- 1.1.3 : Modelos de proceso de software
- 1.1.4 : Metodologías de desarrollo
- 1.1.5 : Metodologías RUP (Rational Unified Process)
- 1.1.6 : Metodología AUP (Agile Unified Process)
- 1.1.7 : UML (Lenguaje de Modelamiento Unificado)
- 1.1.8 : Diagramas de UML 2.5

1.2 Tema 2 : Herramientas CASE

- 1.2.1 : Objetivos de las herramientas CASE
- 1.2.2 : Tipos de herramientas CASE
- 1.2.3 : Ejemplos de herramientas CASE

ACTIVIDADES PROPUESTAS

- Los alumnos identifican las características de cada una de las tres metodologías: la metodología del ciclo de vida del desarrollo de sistemas (SDLC), metodologías ágiles y metodologías orientada a objetos. Para ello, elabora un cuadro comparativo con los tres tipos de metodologías.
- Los alumnos identifican los tipos de diagramas que podemos realizar con UML.
- Los alumnos mencionan los objetivos principales de cada disciplina de RUP.
- Los alumnos mencionan los objetivos principales de cada disciplina de AUP.

1.1. INGENIERÍA DE SOFTWARE, AUP Y UML

1.1.1. Elementos claves de la ingeniería de software

Hoy en día, gracias al avance tecnológico, existen muchos elementos claves a considerar en el campo de la Ingeniería de Software, pero ¿qué es Ingeniería de Software? A continuación, se presentará algunas definiciones de esta área a fin de comprender sus características y elementos claves.

El concepto “ingeniería de software” se propuso originalmente en 1968, en una conferencia organizada por la OTAN (Organización del Tratado del Atlántico Norte) para discutir la “crisis del software”. La crisis del software fue el nombre que se le dio a las dificultades encontradas en el desarrollo de sistemas grandes y complejos en la década de 1960. Se propuso que la adopción de un enfoque de ingeniería para el desarrollo de software reduciría los costos del desarrollo de software y conduciría a un software más confiable. (Sommerville, 2008)

Luego, surgieron otras definiciones de “ingeniería de software” formuladas por prestigiosos autores, tales como:

- “La ingeniería de software trata del establecimiento de los principios y métodos de la ingeniería a fin de obtener software de modo rentable, que sea fiable y trabaje en máquinas reales”. Definida por Friedrich L. Bauer, en 1972.
- “Ingeniería de software es el estudio de los principios y metodologías para el desarrollo y mantenimiento de sistemas software”. Formulada por Zelkowitz, en 1978.
- “Ingeniería de software es la aplicación práctica del conocimiento científico al diseño y construcción de programas de computadora y a la documentación asociada requerida para desarrollar, operar y mantenerlos. Se conoce también como desarrollo de software o producción de software”. Expresada por Bohem, en 1976.
- “Es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software; es decir, la aplicación de la ingeniería al software”, definida por IEEE, en 1993.

En esencia, la producción o creación del software es un proceso intrínsecamente creativo y la ingeniería del software trata de sistematizar este proceso con el fin de acotar el riesgo del fracaso en la consecución del objetivo creativo por medio de diversas técnicas que se han demostrado adecuadas en base a la experiencia previa.

La ingeniería de software se puede considerar como la ingeniería aplicada al software, esto es, por medios sistematizados y con herramientas preestablecidas, la aplicación de ellos de la forma más eficiente para la obtención de resultados óptimos; objetivos que siempre busca la ingeniería. No es sólo de la resolución de problemas, sino más bien teniendo en cuenta las diferentes soluciones, elegir la más apropiada.

Con el objetivo de comprender y tener una mayor visión sobre lo que trata la ingeniería de software, en la figura 1 se resume algunas preguntas planteadas con frecuencia con respecto de esta área. (Sommerville, 2011)

Pregunta	Respuesta
¿Qué es software?	Programas de cómputo y documentación asociada. Los productos de software se desarrollan para un cliente en particular o para un mercado en general.
¿Cuáles son los atributos del buen software?	El buen software debe entregar al usuario la funcionalidad y el desempeño requeridos, y debe ser sustentable, confiable y utilizable.
¿Qué es ingeniería de software?	La ingeniería de software es una disciplina de la ingeniería que se interesa por todos los aspectos de la producción de software.
¿Cuáles son las actividades fundamentales de la ingeniería de software?	Especificación, desarrollo, validación y evolución del software.
¿Cuál es la diferencia entre ingeniería de software y ciencias de la computación?	Las ciencias de la computación se enfocan en teoría y fundamentos; mientras la ingeniería de software se enfoca en el sentido práctico del desarrollo y en la distribución de software.
¿Cuál es la diferencia entre ingeniería de software e ingeniería de sistemas?	La ingeniería de sistemas se interesa por todos los aspectos del desarrollo de sistemas basados en computadoras, incluidos hardware, software e ingeniería de procesos. La ingeniería de software es parte de este proceso más general.
¿Cuáles son los principales retos que enfrenta la ingeniería de software?	Se enfrentan con una diversidad creciente, demandas por tiempos de distribución limitados y desarrollo de software confiable.
¿Cuáles son los costos de la ingeniería de software?	Aproximadamente 60% de los costos del software son de desarrollo, y 40% de prueba. Para el software elaborado específicamente, los costos de evolución superan con frecuencia los costos de desarrollo.
¿Cuáles son los mejores métodos y técnicas de la ingeniería de software?	Aun cuando todos los proyectos de software deben gestionarse y desarrollarse de manera profesional, existen diferentes técnicas que son adecuadas para distintos tipos de sistema. Por ejemplo, los juegos siempre deben diseñarse usando una serie de prototipos, mientras que los sistemas críticos de control de seguridad requieren de una especificación completa y analizable para su desarrollo. Por lo tanto, no puede decirse que un método sea mejor que otro.
¿Qué diferencias ha marcado la Web a la ingeniería de software?	La Web ha llevado a la disponibilidad de servicios de software y a la posibilidad de desarrollar sistemas basados en servicios distribuidos ampliamente. El desarrollo de sistemas basados en Web ha conducido a importantes avances en lenguajes de programación y reutilización de software.

Figura 1: Preguntas planteadas con frecuencia sobre el software

Fuente. - Tomado de <https://www.biblionline.pearson.com/Pages/BookRead.aspx>

Según (Pressman, 2010) los elementos claves de la ingeniería de software lo definen cuatro capas, y son las que se detallan a continuación:

- **Compromiso de calidad:** Cualquier enfoque de ingeniería (incluso la de software) debe tener como base un compromiso organizacional con la calidad. La administración total de la calidad, Six Sigma y otras filosofías similares alimentan la cultura de mejora continua, y es esta cultura la que lleva en última instancia al desarrollo de enfoques cada vez más eficaces de la ingeniería de software.
- **Procesos:** El fundamento de la ingeniería de software es la capa proceso, el cual define un marco de trabajo para un conjunto de áreas clave que forman la base del control de gestión de proyectos de software y establecen el contexto en el cual se aplican los métodos técnicos, se producen resultados de trabajo, se establecen hitos, se asegura la calidad y el cambio se gestiona adecuadamente.

- **Métodos:** Indican cómo construir técnicamente el software. Los métodos abarcan una gran gama de tareas que incluyen análisis de requisitos, diseño, construcción de programas, pruebas y mantenimiento. Estos métodos dependen de un conjunto de principios básicos que gobiernan cada área de la tecnología e incluyen actividades de modelado y otras técnicas descriptivas.
- **Herramientas:** Las herramientas de la ingeniería del software proporcionan un soporte automático o semi-automático para el proceso y los métodos, a estas herramientas se les llama herramientas CASE (Computer-Aided Software Engineering en inglés).



Figura 2: La ingeniería de software como una tecnología multicapa

Fuente. - Tomado <http://ingenieraupoliana.blogspot.com/2010/10/capas-del-proceso-de-software.html>

1.1.2. Procesos de software

La ingeniería de software es una disciplina de ingeniería que se interesa por todos los aspectos de la producción de software. El software no es sólo un programa o conjunto de programas, sino que también incluye documentación: modelos, diagramas, y documentos que siguen una plantilla o formato. De modo que, todo proceso de software debe incluir actividades relacionadas que conllevan a la creación de un producto software.

A lo largo de las décadas de 1970 y 1980 se desarrolló una variedad de nuevas técnicas y métodos de ingeniería de software, tales como la programación estructurada, el encubrimiento de información y el desarrollo orientado a objetos. Se perfeccionaron herramientas y notaciones estándar y ahora se usan de manera extensa. (Sommerville, 2008).

Según Sommerville (2011) existen muchos diferentes procesos de software, pero todos deben incluir cuatro actividades que son fundamentales para la ingeniería de software, tales como especificación, diseño e implementación, validación, y evolución:

1. Especificación del software. En la que se definen tanto la funcionalidad del software como las restricciones de su operación.
2. Diseño e implementación del software. debe desarrollarse el software para cumplir con las especificaciones.
3. Validación del software. Hay que validar el software para asegurarse de que cumple lo que el cliente quiere.
4. Evolución del software. El software tiene que evolucionar para satisfacer las necesidades cambiantes del cliente.

Además de tener en cuenta qué actividades se llevan a cabo en un proceso de software, es necesario conocer cuál es la secuencia u orden de ejecución de éstas y de sus subactividades, qué productos se deben realizar (resultados de una actividad del proceso), quiénes son los responsables (roles) de crearlos y según (Sommerville, 2011) toda actividad debe describir precondiciones y postcondiciones (declaraciones válidas antes y después de que se realice una actividad del proceso o se cree un producto).

1.1.3. Modelos de proceso de software

Según la RAE (2020), el término “modelo” es un arquetipo o punto de referencia para imitarlo o producirlo. Representación en pequeño de alguna cosa. Asimismo, se define como un esquema teórico, generalmente en forma matemática, de un sistema o de una realidad compleja.

De esta definición podríamos considerar las siguientes ideas claves “arquetipo”, “representación”, y “esquema de una realidad compleja”. De modo que, un modelo de proceso de software podría definirse como “la representación o esquema de un proceso que conduce la construcción de un producto software”.

La definición de Sommerville (2011) es: “Un modelo de proceso de software es una representación simplificada de este proceso. Cada modelo del proceso representa a otro desde una particular perspectiva y, por lo tanto, ofrece sólo información parcial acerca de dicho proceso.”

En esta sección se explicará tres modelos de proceso muy generales, que algunos profesionales los llaman “paradigmas de proceso” y otros los conocen como “frameworks del proceso” puesto que no describen los detalles de las actividades específicas solo un marco del proceso. De modo que, estos modelos genéricos no son descripciones definitivas de los procesos de software, sino abstracciones del proceso que se utilizan para explicar los diferentes enfoques del desarrollo de software. Estos modelos son los siguientes según Sommerville (2011):

1. El modelo en cascada (waterfall), toma las actividades fundamentales del proceso de especificación, desarrollo, validación y evolución y, luego, los representa como fases separadas del proceso, tal como especificación de requerimientos, diseño de software, implementación, pruebas, etc.

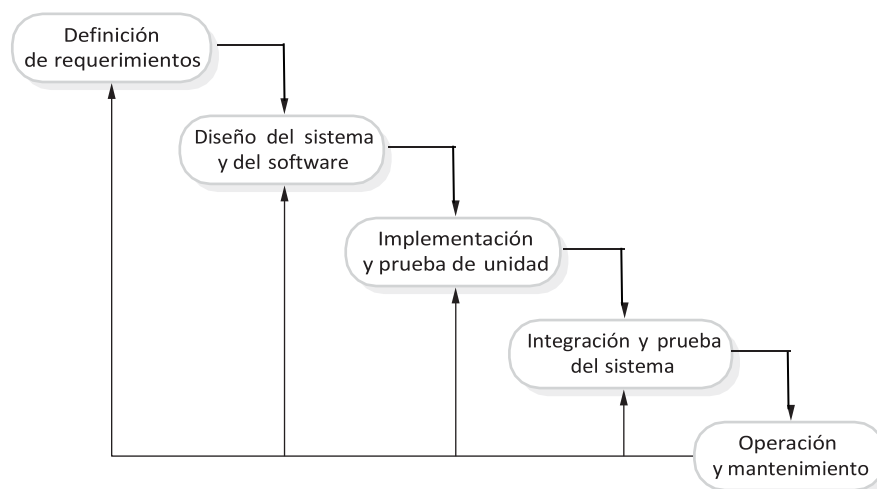


Figura 3: El modelo de cascada

Fuente. - Tomado de <https://www.biblionline.pearson.com/Pages/BookRead.aspx>

2. **Desarrollo incremental**, este enfoque vincula las actividades de especificación, desarrollo y validación. El sistema se desarrolla como una serie de versiones (incrementos), y cada versión añade funcionalidad a la versión anterior.

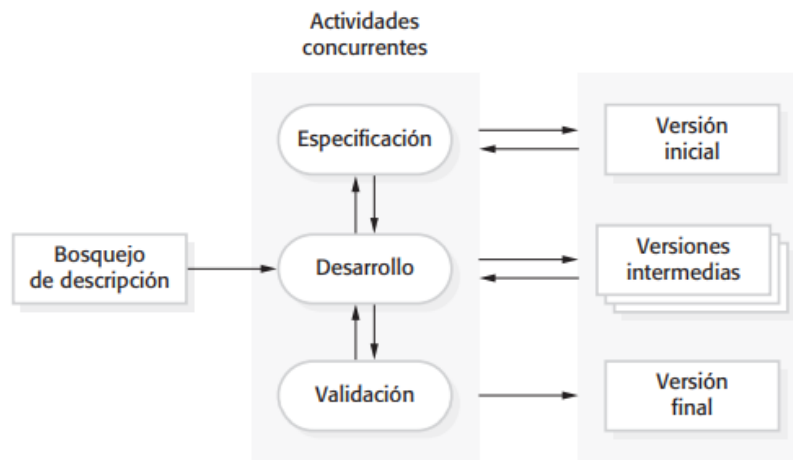


Figura 4: Desarrollo incremental

Fuente. - Tomado de <https://www.biblionline.pearson.com/Pages/BookRead.aspx>

3. **Ingeniería de software orientada a la reutilización**, este enfoque se basa en la existencia de un número significativo de componentes reutilizables. El proceso de desarrollo del sistema se enfoca en la integración de estos componentes en un sistema, en vez de desarrollarlo desde cero.

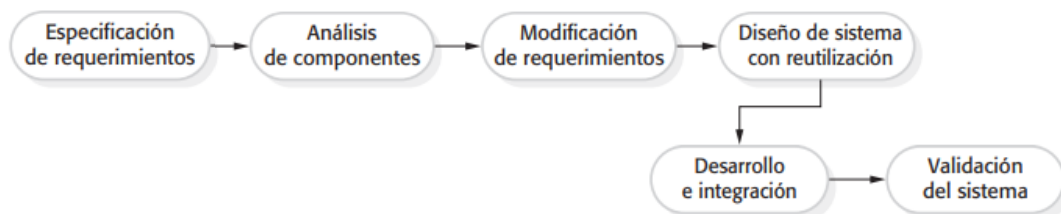


Figura 5: Ingeniería de software orientado a la reutilización

Fuente. - Tomado de <https://www.biblionline.pearson.com/Pages/BookRead.aspx>

1.1.4. Metodologías de desarrollo

Una metodología de desarrollo de software, a diferencia de un modelo de proceso, define de forma específica cómo llevar a cabo un proyecto de desarrollo en términos de técnicas, herramientas y modelos¹. A continuación, se describe las tres principales metodologías de desarrollo (Kendall, 2011): ciclo de vida del desarrollo de sistemas (SDLC), las metodologías ágiles y las orientadas a objetos con UML, junto con los motivos y las situaciones que indican cuándo utilizarlos.

¹ Modelo en este apartado hace referencia a un conjunto de artefactos (diagramas, componentes, y/o documentación) que se generan en una actividad.

1.1.4.1. Ciclo de vida del desarrollo de sistemas (SDLC)

El Ciclo de vida de desarrollo de sistemas (SDLC)² es una metodología en fases para el análisis y diseño, de acuerdo con la cual los sistemas se desarrollan mejor al utilizar un ciclo específico de actividades del analista y los usuarios.

El ciclo se puede dividir en siete fases, como se muestra en la figura 6. Varias actividades pueden ocurrir al mismo tiempo, e incluso pueden repetirse.

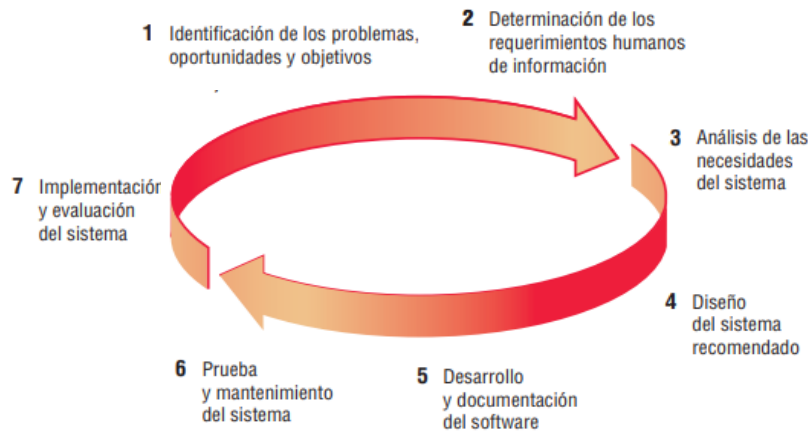


Figura 6: Fases del SDLC

Fuente. - Tomado de <https://www.biblionline.pearson.com/Pages/BookRead.aspx>

1.1.4.2. Metodologías ágiles

Estos enfoques tienen como base los valores, principios y prácticas básicas. Los cuatro valores son comunicación, simpleza, retroalimentación y valentía. Sin embargo, es recomendable que los analistas de sistemas adopten estos valores en todos los proyectos que emprendan y no sólo cuando adopten la metodología ágil.

Para poder terminar con éxito un proyecto con esta metodología, a menudo se debe realizar ciertos ajustes en la administración de este, específicamente, en los importantes recursos de tiempo, costo, calidad y alcance. Cuando se incluyen estas cuatro variables de control en forma apropiada en la planificación, hay un estado de equilibrio entre los recursos y las actividades necesarias para completar el proyecto.

Es más notable llevar las prácticas de desarrollo al extremo cuando se persiguen prácticas únicas para el desarrollo ágil. Las cuatro prácticas ágiles básicas son liberaciones de versiones cortas, la semana de trabajo de 40 horas, hospedar un cliente en el sitio y utilizar programación en pareja.

Hay actividades y comportamientos que determinan la manera en que actúan los miembros del equipo y los clientes durante el desarrollo de un proyecto ágil. Dos palabras que caracterizan a un proyecto realizado mediante una metodología ágil son *interactivo* e *incremental*. En la siguiente figura podrá notar que hay cinco etapas: exploración, planeación, iteraciones para la liberación de la primera versión, puesta en producción y mantenimiento.

² SDLC: Es la sigla en inglés de Systems Development Life Cycle.

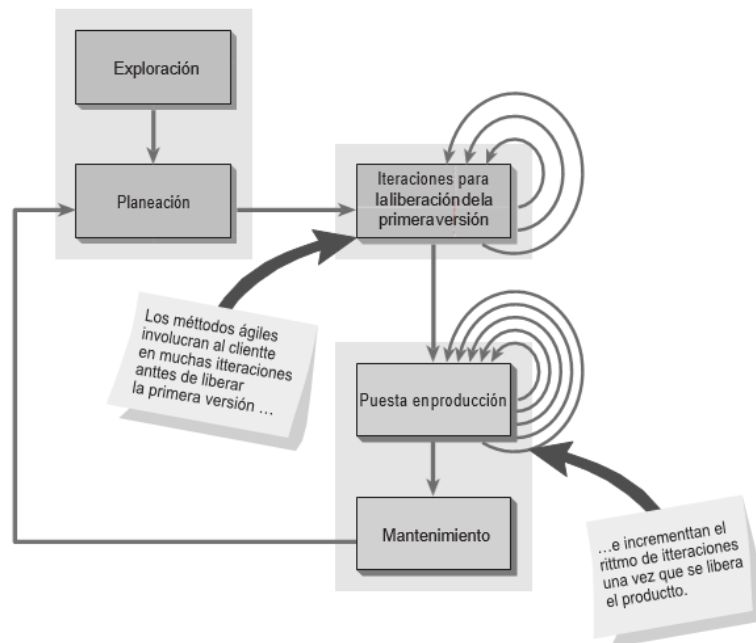


Figura 7: Etapas de la metodología ágil

Fuente. - Tomado de <https://www.biblionline.pearson.com/Pages/BookRead.aspx>

1.1.4.3. Metodología orientada a objetos con UML

Este enfoque funciona bien en situaciones en las que los sistemas de información complicados pasan a través de un proceso continuo de mantenimiento, adaptación y rediseño. Facilita el desarrollo de sistemas que deben cambiar con rapidez en respuesta a los entornos empresariales dinámicos. Utiliza el estándar de la industria para modelar sistemas orientados a objetos, conocido como lenguaje de modelado unificado (UML), para descomponer un sistema en un modelo de caso de uso. En la siguiente figura se muestran los pasos que describen de forma breve las 3 fases de la metodología.

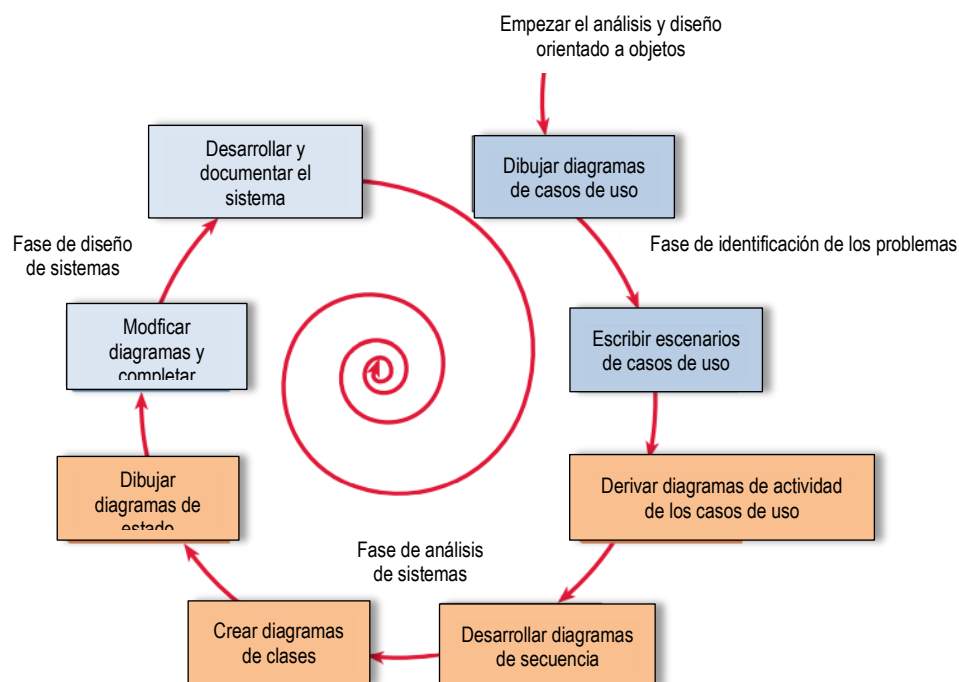


Figura 8: Pasos de la metodología orientada a objetos

Fuente. - Tomado de <https://www.biblionline.pearson.com/Pages/BookRead.aspx>

1.1.4.4. Cómo elegir la metodología de desarrollo

Las diferencias entre las tres metodologías antes descritas no son tan grandes como parecen en un principio. En las tres metodologías, el analista necesita comprender primero a la organización. Después el analista o el equipo del proyecto necesitan elaborar un presupuesto del tiempo y los recursos necesarios para desarrollar la propuesta del proyecto. A continuación, deben entrevistar a los miembros de la organización y recopilar información detallada mediante el uso de cuestionarios, obtener muestras de los datos de los informes existentes y observar cómo se lleva a cabo la actividad empresarial actual. Las tres metodologías tienen todas estas actividades en común.

La metodología SDLC y la metodología orientada a objetos requieren de un proceso exhaustivo de planeación y elaboración de diagramas. La metodología ágil y la metodología orientada a objetos permiten crear subsistemas uno a la vez hasta que se complete todo el sistema. La metodología ágil y la metodología SDLC se interesan por la forma lógica en que los datos se desplazan a través del sistema.

Seleccione	Cuando
La metodología del ciclo de vida del desarrollo de sistemas (SDLC)	- los sistemas se hayan desarrollado y documentado mediante el uso de SDLC - sea importante documentar cada paso del proceso - la administración de nivel superior se sienta más cómoda o segura si utiliza SDLC - haya los recursos y el tiempo adecuados para completar el SDLC completo - sea importante la comunicación en relación con la forma en que funcionan los nuevos sistemas
Metodologías ágiles	- haya un defensor de proyectos de métodos ágiles en la organización - haya que desarrollar aplicaciones rápidamente en respuesta a un entorno dinámico - haya que realizar un rescate (el sistema falló y no hay tiempo de averiguar qué salió mal) - el cliente está satisfecho con las mejoras incrementales - los ejecutivos y analistas están de acuerdo con los principios de las metodologías ágiles
Metodologías orientadas a objetos	- los problemas modelados se prestan a sí mismos para convertirlos en clases - una organización ofrece apoyo para aprender UML - es posible agregar sistemas en forma gradual, un subsistema a la vez - la reutilización de software escrito con anterioridad es una posibilidad - es aceptable hacer frente a los problemas difíciles primero

Figura 9: Metodologías de Desarrollo de Software

Fuente. - Tomado de <https://www.biblionline.pearson.com/Pages/BookRead.aspx>

Entonces, dada la opción de desarrollar un sistema mediante el uso de una metodología SDLC, una metodología ágil o una metodología orientada a objetos, la pregunta que nos haríamos es ¿cuál metodología escoger? La figura 9 muestra un conjunto de lineamientos que nos ayudará a elegir la metodología a utilizar para desarrollar un sistema.

1.1.5. Metodología RUP (Rational Unified Process)

RUP es una metodología orientada a objetos que fue creado por tres autores: Ivar Jacobson, Grady Booch y James Rumbaugh, quienes trabajaron juntos motivados por contar con un “método unificado” para el desarrollo de software, que combinaría lo mejor de cada uno de sus métodos individuales de análisis y diseño orientado a objetos.

Los tres amigos, como así se les conoce, analizan la necesidad de un proceso del software impulsado por el caso de uso, centrado en la arquitectura, e iterativo e incremental. Estos tres aspectos son las características esenciales de esta metodología:

- **Dirigido por Casos de Uso.** Los casos de uso son un medio para determinar los requisitos correctos y utilizarlos para conducir el proceso de desarrollo.
- **Centrado en la arquitectura.** Se describe mediante diferentes vistas del sistema en construcción.
- **Iterativo e incremental.** Es práctico dividir el trabajo en partes más pequeñas o mini proyectos, cada mini proyecto es una iteración que resulta en un incremento.

Esta metodología utiliza el estándar UML como lenguaje de modelado para representar unificado se usan mucho en proyectos de toda clase orientados a objetos. El modelo iterativo e incremental propuesto por el PU puede y debe adaptarse para que satisfaga necesidades específicas del proyecto.

1.1.5.1. Estructura RUP

RUP es un proceso que puede describirse en dos dimensiones, tal como se muestra en la siguiente figura, a lo largo de dos ejes:

- El eje horizontal representa tiempo y muestra el aspecto dinámico del proceso, expresado en términos de ciclos, fases, iteraciones, y metas.
- El eje vertical representa el aspecto estático del proceso; como está descrito en términos de actividades, artefactos, trabajadores o roles y flujos de trabajo.

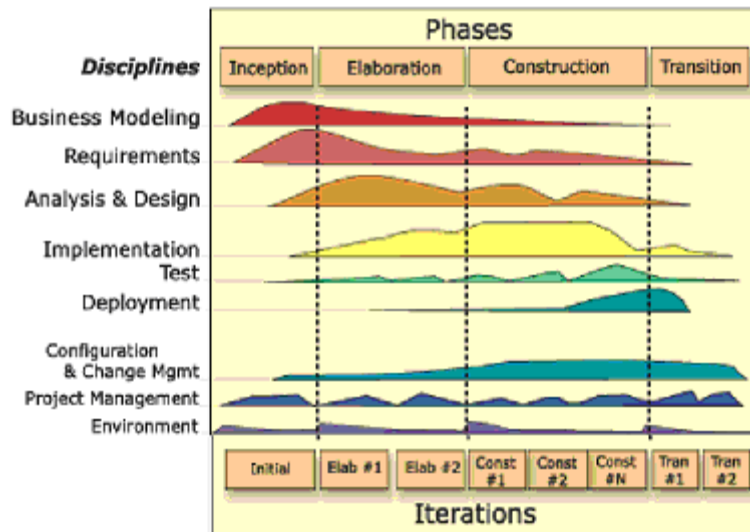


Figura 10: Estructura de RUP: Fases y disciplinas

Fuente. - Tomado de <https://theagileblueprint.files.wordpress.com/2011/03/rup-diagram.png>

1.1.5.2. Fases RUP

Fase de Concepción o de Inicio: Agrupa actividades tanto de comunicación con el cliente como de planeación. Al colaborar con los participantes, se identifican los requerimientos del negocio, se propone una arquitectura aproximada para el sistema y se desarrolla un plan para la naturaleza iterativa e incremental del proyecto en cuestión.

Fase de Elaboración: Mejora y amplía los casos de uso preliminares desarrollados como parte de la fase de concepción y aumenta la representación de la arquitectura para incluir cinco puntos de vista distintos del software: los modelos del caso de uso, de requerimientos, del diseño, de la implementación y del despliegue. En ciertos casos, la elaboración crea una “línea de base de la arquitectura ejecutable” que representa un sistema ejecutable. Además, al terminar la fase de elaboración se revisa con cuidado el plan a fin de asegurar que el alcance, riesgos y fechas de entrega siguen siendo razonables. Es frecuente que en este momento se hagan modificaciones al plan.

Fase de Construcción: Con el uso del modelo de arquitectura como entrada, se desarrolla o adquiere los componentes del software que harán que cada caso de uso sea operativo para los usuarios finales. A medida que se implementan los componentes, se diseñan y efectúan pruebas unitarias para cada uno. Además, se realizan actividades de integración (ensamble de componentes y pruebas de integración). Se emplean casos de uso para obtener un grupo de pruebas de aceptación que se ejecutan antes de comenzar la siguiente fase.

Fase de Transición: En esta fase se da el software a los usuarios finales para las pruebas beta, quienes reportan tanto los defectos como los cambios necesarios. Además, el equipo de software genera la información de apoyo necesaria (por ejemplo, manuales de usuario, guías de solución de problemas, procedimientos de instalación, etc.) que se requiere para el lanzamiento. Al finalizar la fase de transición, se tiene un producto utilizable que se lanza.

1.1.5.3. Flujos de trabajo o disciplinas RUP

Conocidos también como disciplinas, consisten en una secuencia de actividades que producen un resultado observable (artefacto) a cargo de algún miembro del proyecto (rol). Existen dos grupos de flujos de trabajo: de proceso y de apoyo; las que se describirán a continuación.

Los flujos del proceso son aquellos orientados al desarrollo del software. Comprende:

- **Modelado del negocio:** Describe la estructura y la dinámica de la organización donde se va a implantar el sistema que construyamos.
- **Requisitos:** Establece exactamente lo que tiene que hacer el sistema, para ello se extrae los requisitos utilizando diferentes métodos.
- **Análisis y Diseño:** Traduce los requisitos a una especificación que describe cómo implementar el sistema, creando para ello, diferentes vistas arquitectónicas.
- **Implementación:** Cubre el desarrollo de software, las pruebas unitarias y la integración.
- **Pruebas:** Describe la ejecución de pruebas y las métricas para rastreo de defectos.
- **Despliegue o Implantación:** Incluye actividades relacionadas con la entrega de la aplicación.

Los flujos de trabajo de apoyo o soporte están orientados a la gestión del proyecto. Éstos son:

- **Configuración y Control de cambios:** Mantiene la integridad de todos los artefactos que se crean. También mantiene información del proceso evolutivo que se ha seguido.
- **Gestión del proyecto:** Es el arte de lograr un balance al gestionar objetivos, riesgos y restricciones para desarrollar un producto acorde a los requisitos especificados.
- **Entorno:** Cubre la infraestructura necesaria para desarrollar un sistema.

1.1.5.4. Roles RUP

Un rol define el comportamiento y responsabilidades de un individuo, o de un grupo de individuos trabajando juntos como un equipo. Las responsabilidades de un rol son tanto llevar a cabo un conjunto de actividades como ser el dueño de un conjunto de artefactos.

Rol Genérico	Roles Específicos
Analistas	Analista de Procesos de Negocio, Diseñador de Negocio, Analistas de Sistema, Especificador de requisitos.
Desarrolladores	Arquitecto de Software, Diseñador, Diseñador de Interfaz de Usuario, Diseñador de Cápsulas, Diseñador de Base de Datos, Implementador, Integrador.
Gestores	Jefe de Proyecto, Jefe de Control de Cambios, Jefe de Configuración, Jefe de Pruebas, Jefe de Despliegue, Ingeniero de Procesos, Gestor de Pruebas.
Apoyo	Documentador Técnico, Administrador de Sistema, Especialista en herramientas, Artista gráfico.
Especialista en Pruebas	Especialista en Pruebas (Tester), Analista de Pruebas, Diseñador de Pruebas.
Otros roles	Stakeholders, Revisor, Coordinador de revisiones, Revisor Técnico.

Figura 11: Roles RUP
Fuente. - Elaboración propia

1.1.6. Metodología AUP (Agile Unified Process)

El proceso unificado ágil (PUA) o Metodología AUP, por sus siglas en inglés, es una versión simplificada de RUP. Adopta una filosofía **“en serie para lo grande”** e **“iterativa para lo pequeño”** a fin de construir sistemas basados en computadora, liberando entregables incrementales (Ambler, 2006). Al adoptar las actividades en fases clásicas del RUP (concepción, elaboración, construcción y transición), el AUP brinda un revestimiento en serie (por ejemplo, una secuencia lineal de actividades de ingeniería de software) que permite que el equipo visualice el flujo general del proceso de un proyecto de software. Sin embargo, dentro de cada actividad, el equipo repite con objeto de alcanzar la agilidad y entregar tan rápido como sea posible incrementos de software significativos a los usuarios finales.

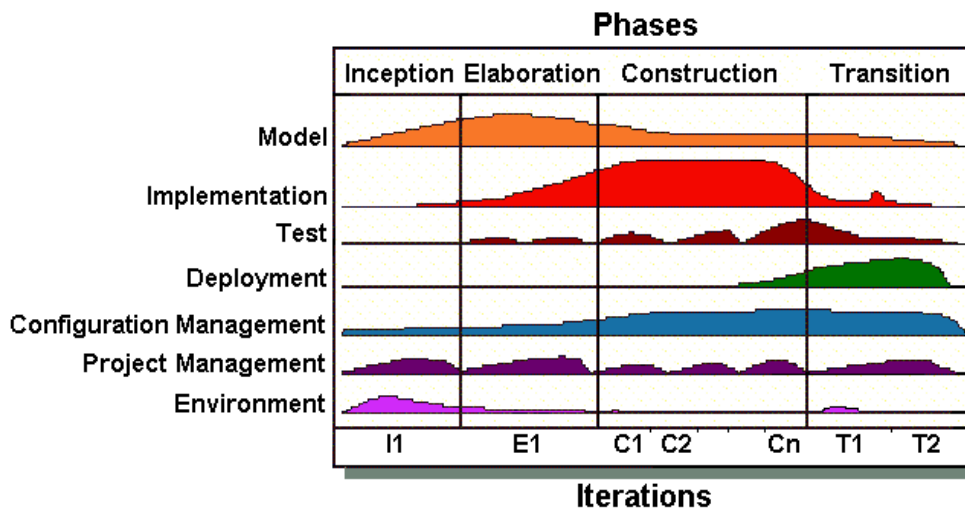


Figura 12: Estructura de AUP: fases y disciplinas
Fuente. - Tomado de <http://www.ambysoft.com/unifiedprocess/agileUP.html#Serial>

1.1.6.1. Fases y Disciplinas AUP

La naturaleza serial de AUP se captura en sus cuatro fases:

- **Inicio.** El objetivo es identificar el alcance inicial del proyecto, una arquitectura potencial para su sistema, y obtener la financiación inicial del proyecto y la aceptación de las partes interesadas.
- **Elaboración.** El objetivo es probar la arquitectura del sistema.
- **Construcción.** El objetivo es crear software de trabajo de forma regular e incremental que satisfaga las necesidades de mayor prioridad de las partes interesadas de su proyecto.
- **Transición.** El objetivo es validar e implementar el sistema en el entorno de producción.

Cada fase presenta hitos que se deben lograr, tal como se muestra en la siguiente figura.

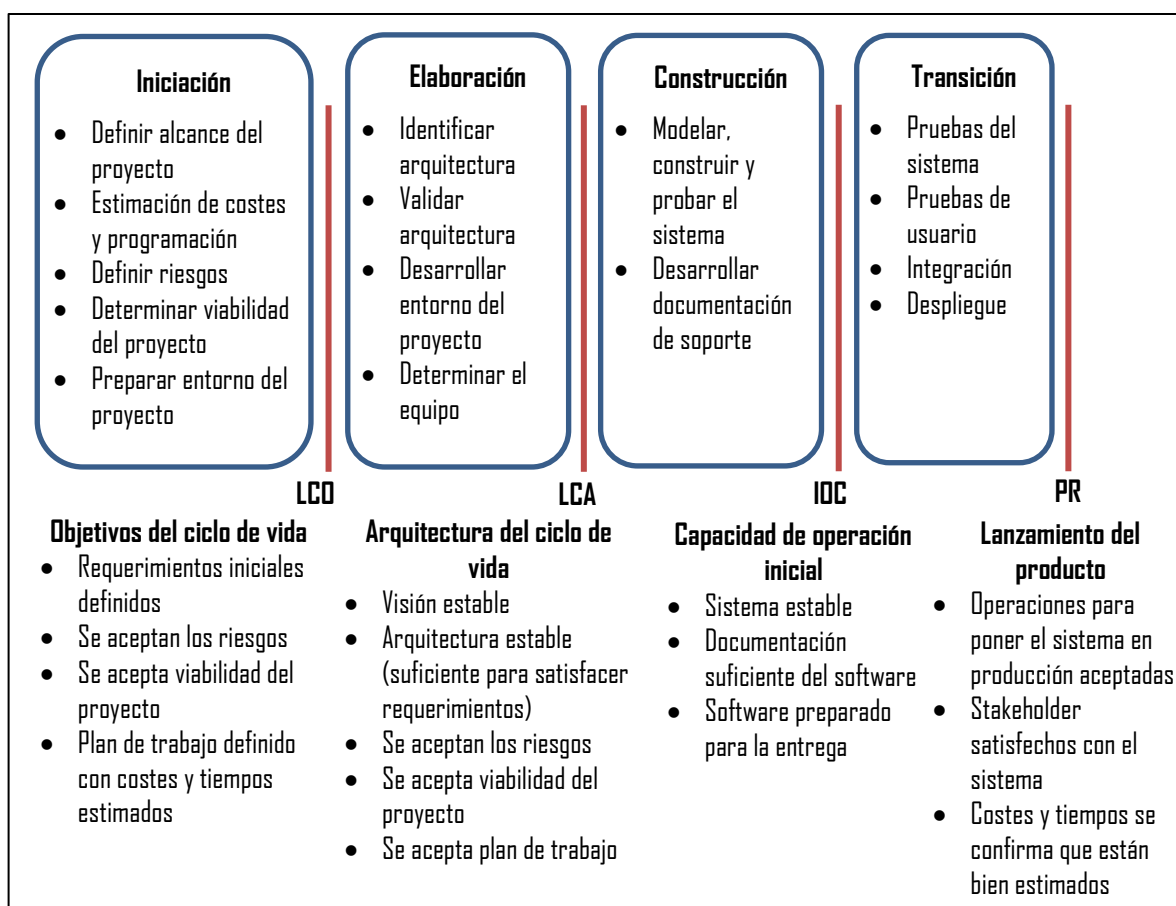


Figura 13: Hitos por cada fase del AUP

Fuente. - Elaboración propia

Las disciplinas se realizan de manera iterativa, definiendo las actividades que los miembros del equipo de desarrollo realizan para construir, validar y entregar software de trabajo que satisfaga las necesidades de sus partes interesadas. Las disciplinas o actividades de cada iteración del AUP son las siguientes:

- **Modelado.** Se crean representaciones de UML de los dominios del negocio y el problema. No obstante, para conservar la agilidad, estos modelos deben ser “solo lo suficientemente buenos” (JBGU)³ para permitir que el equipo avance.
- **Implementación.** Los modelos se traducen a código fuente.
- **Pruebas.** Igual que con la XP⁴, el equipo diseña y ejecuta una serie de pruebas para detectar errores y garantizar que el código fuente cumple sus requerimientos.
- **Despliegue.** El despliegue se centra en la entrega de un incremento de software y en la obtención de retroalimentación de los usuarios finales.
- **Administración de la configuración.** Incluye la administración del cambio y el riesgo, y el control de cualesquiera productos del trabajo persistentes que produzca el equipo.
- **Administración del proyecto.** La administración del proyecto da seguimiento y controla el avance del equipo y coordina sus actividades.
- **Administración del ambiente.** La administración del ambiente coordina una infraestructura del proceso que incluye estándares, herramientas y otra tecnología de apoyo de la que dispone el equipo.

En la primera disciplina se menciona una expresión clave que se debe considerar en el AUP: “solo lo suficientemente bueno” o JBGE. JBGE refleja las necesidades de la situación. Entonces, la cuestión es ¿cómo conseguir un artefacto JBGE?

El primer paso es identificar un propósito válido para crear un modelo y la audiencia para ese modelo, y luego basado en ese propósito y la audiencia, se debe desarrollar hasta el punto en que es suficientemente preciso y suficientemente detallado. Una vez que un modelo ha cumplido sus objetivos, se ha terminado con él por ahora y se debe pasar a otra actividad, como escribir código para mostrar que el modelo funciona. Este principio también se aplica a un cambio en un modelo existente: si se está realizando un cambio, tal vez aplicando un patrón conocido, entonces se debería tener una razón válida para realizar ese cambio (tal vez para admitir un nuevo requisito o para refactorizar su trabajo a algo más limpio). Una implicación importante de este principio es que se necesita conocer a la audiencia, incluso cuando esa audiencia es usted mismo. Por ejemplo, si está creando un modelo para desarrolladores de mantenimiento, ¿qué es lo que realmente necesitan? ¿Necesitan un documento completo de 500 páginas o sería suficiente una visión general de 10 páginas de cómo funciona todo? ¿No sé? Lo que tiene que hacer es ir a hablar con ellos y averiguarlo.

A continuación, se mostrará gráficamente cada flujo de trabajo o disciplina del AUP en el que se incluye objetivos, roles, actividades y resultados.

Disciplina de Modelado. El objetivo de esta disciplina es comprender el negocio de la organización, el dominio problemático que está abordando el proyecto e identificar una solución viable para abordar el dominio del problema.

³ JBGU es la sigla en inglés de *Just Barely Good Enough*.

⁴ XP es el acrónimo de Extreme Programming. Es una metodología ágil de desarrollo de software con bases en la comunicación constante y la retroalimentación.

En el curso enfocaremos nuestros esfuerzos para desarrollar los siguientes artefactos de esta disciplina, los mismos que cubren lo que se resalta con cuadros rojos en la siguiente figura.

- Situación del Negocio (**Documento** que incluye: Situación de la Organización, Especificaciones de Casos de Uso de Negocio, Especificación de Reglas del Negocio y Glosario de términos del proyecto.)
- Modelo de Casos de Uso de Negocio (**Diagramas** que incluye actores de negocio, casos de uso de negocio y Diagrama General de Casos de Uso de Negocio)
- Modelo de Análisis de Negocio (**Diagramas** que incluye entidades de negocio, trabajadores de negocio y realizaciones de negocio)
- Especificación del Software (**Documento** que incluye: Requisitos funcionales, Requisitos No funcionales o técnicos y Especificación de Casos de Uso con prototipos)
- Modelo de Casos de Uso (**Diagramas** que incluye actores, casos de uso y Diagrama de Casos de Uso Estructurado)

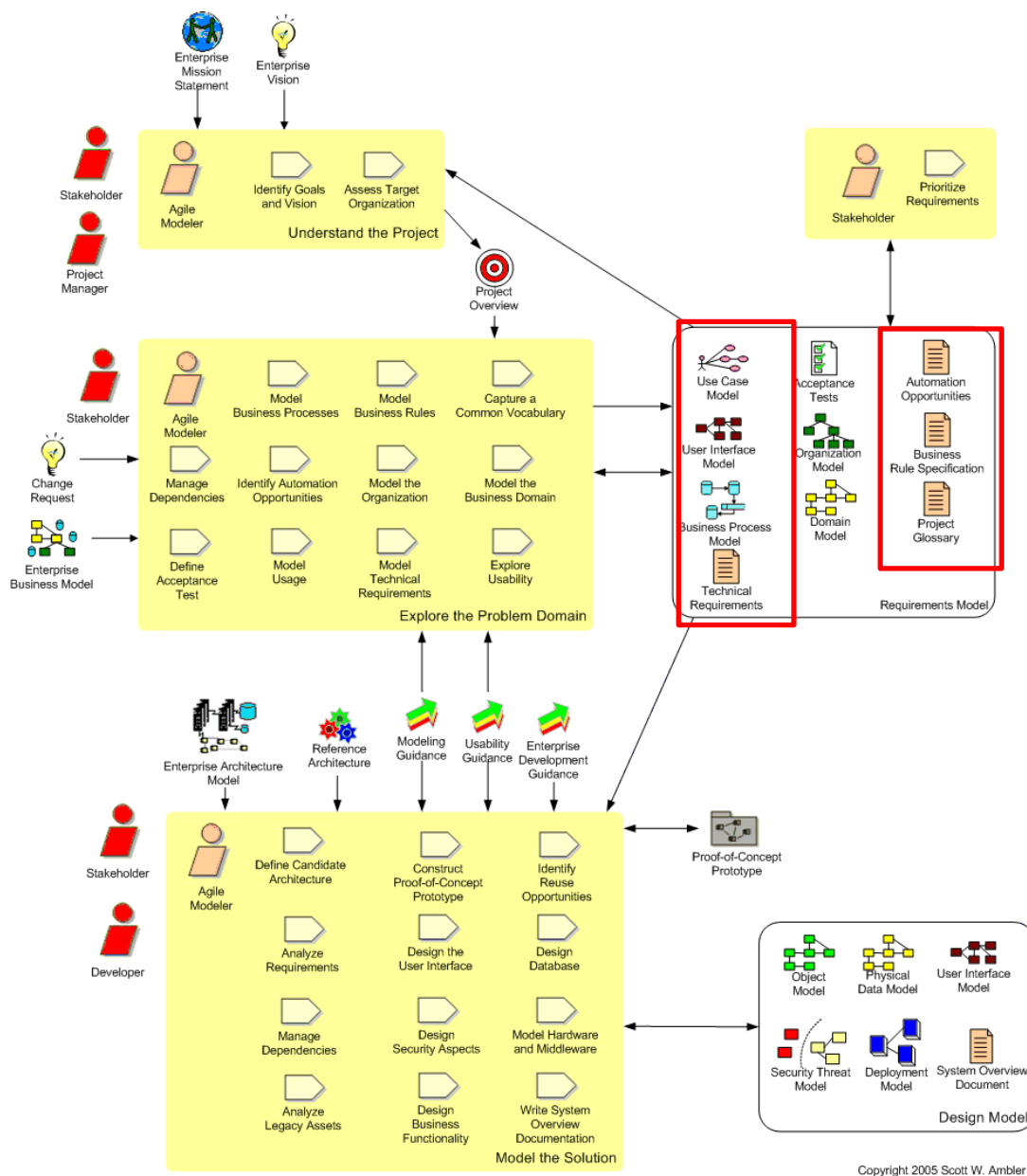


Figura 14: Disciplina de Modelado del AUP

Fuente. - Tomado de <http://www.ambysoft.com/unifiedprocess/aup11/html/model.html>

Disciplina de Implementación. El objetivo de esta disciplina es transformar los modelos en código ejecutable y realizar un nivel básico de pruebas, en particular las pruebas unitarias.

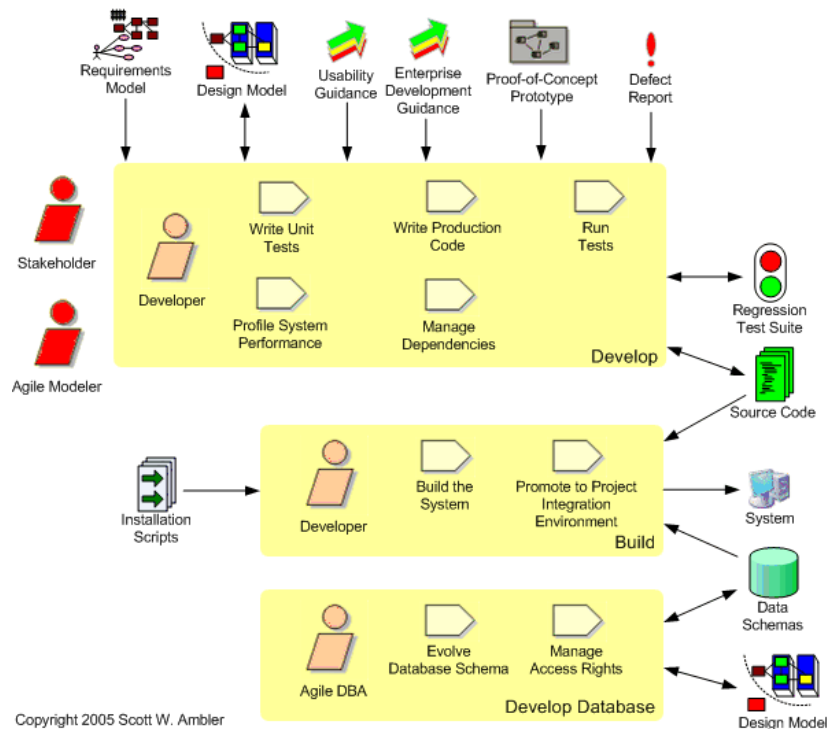


Figura 15: Disciplina de Implementación del AUP

Fuente. - Tomado de <http://www.ambysoft.com/unifiedprocess/aup11/html/implementation.html>

Disciplina de Pruebas. El objetivo es realizar una evaluación objetiva para garantizar la calidad. Incluye la búsqueda de defectos, la validación de que el sistema funciona según lo diseñado y la verificación de que cumplen los requisitos.

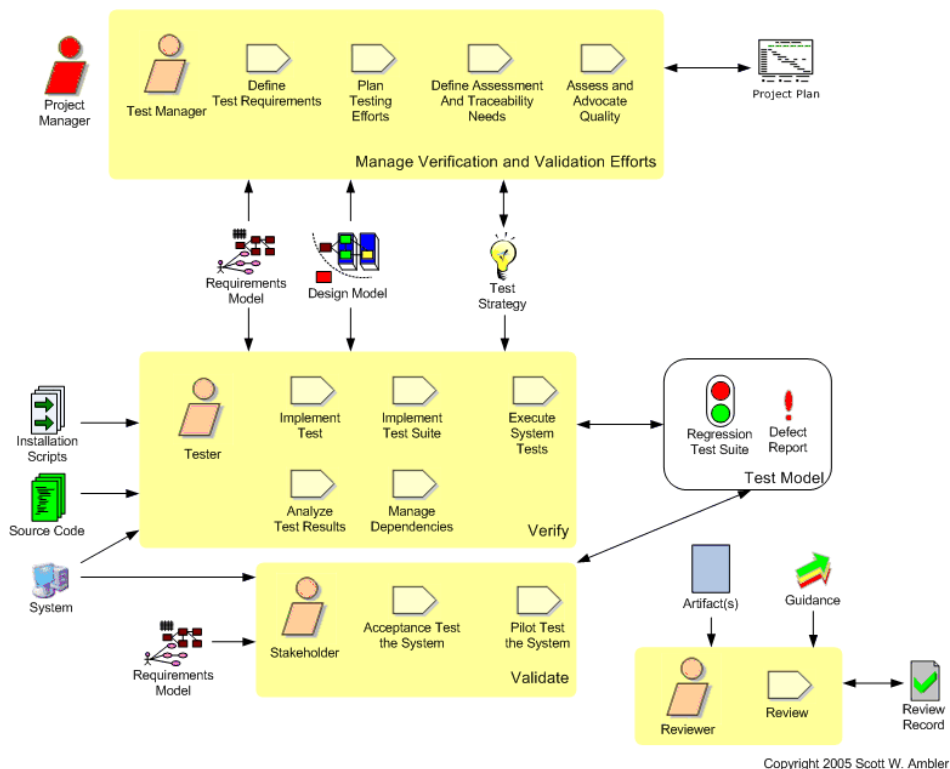


Figura 16: Disciplina de Pruebas del AUP

Fuente. - Tomado de <http://www.ambysoft.com/unifiedprocess/aup11/html/test.html>

Disciplina de Despliegue. El objetivo de esta disciplina es planificar la entrega del sistema y ejecutar el plan para poner el sistema a disposición de los usuarios finales.

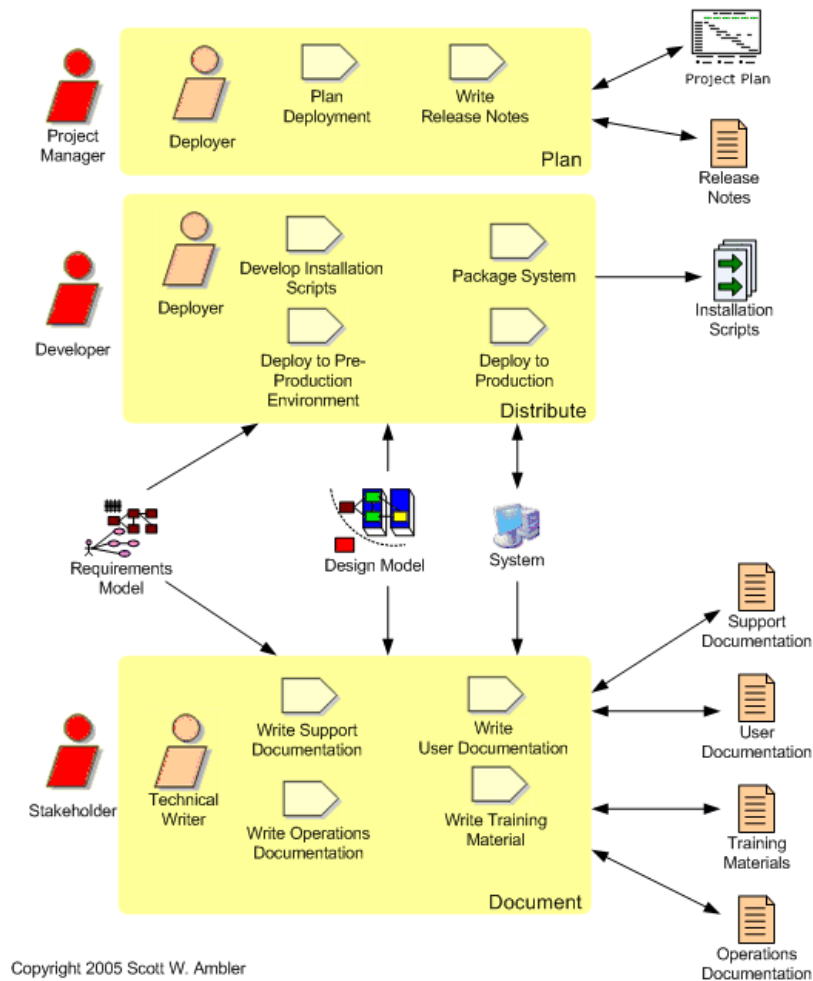


Figura 17: Disciplina de Despliegue del AUP

Fuente. - Tomado de <http://www.ambysoft.com/unifiedprocess/aup11/html/deployment.html>

Disciplina de Administración de la Configuración. El objetivo es gestionar el acceso a los productos de trabajo de su proyecto. Esto incluye no sólo el seguimiento de las versiones del producto de trabajo a lo largo del tiempo, sino también el control y la administración de los cambios en ellas.

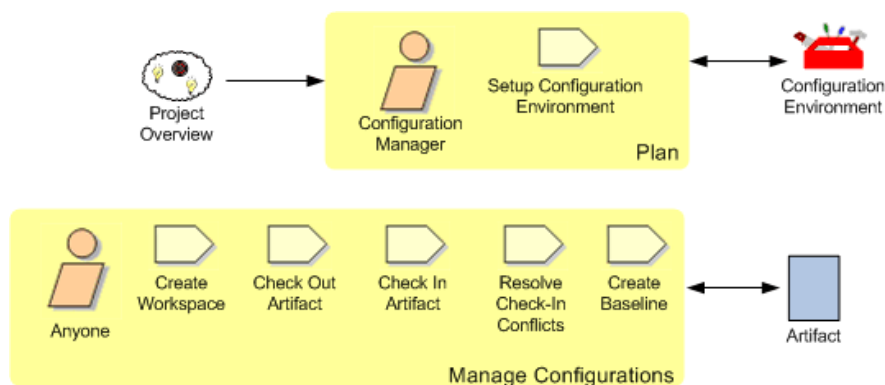


Figura 18: Disciplina de Administración de la Configuración del AUP

Fuente. - Tomado de <http://www.ambysoft.com/unifiedprocess/aup11/html/configurationManagement.html>

Disciplina de Administración del proyecto. El objetivo es dirigir las actividades que se llevan a cabo en el proyecto. Esto incluye la gestión de riesgos, la dirección de las personas (asignación de tareas, seguimiento del progreso, etc.) y la coordinación con personas y sistemas fuera del ámbito del proyecto para asegurarse de que se entrega a tiempo y dentro del presupuesto.

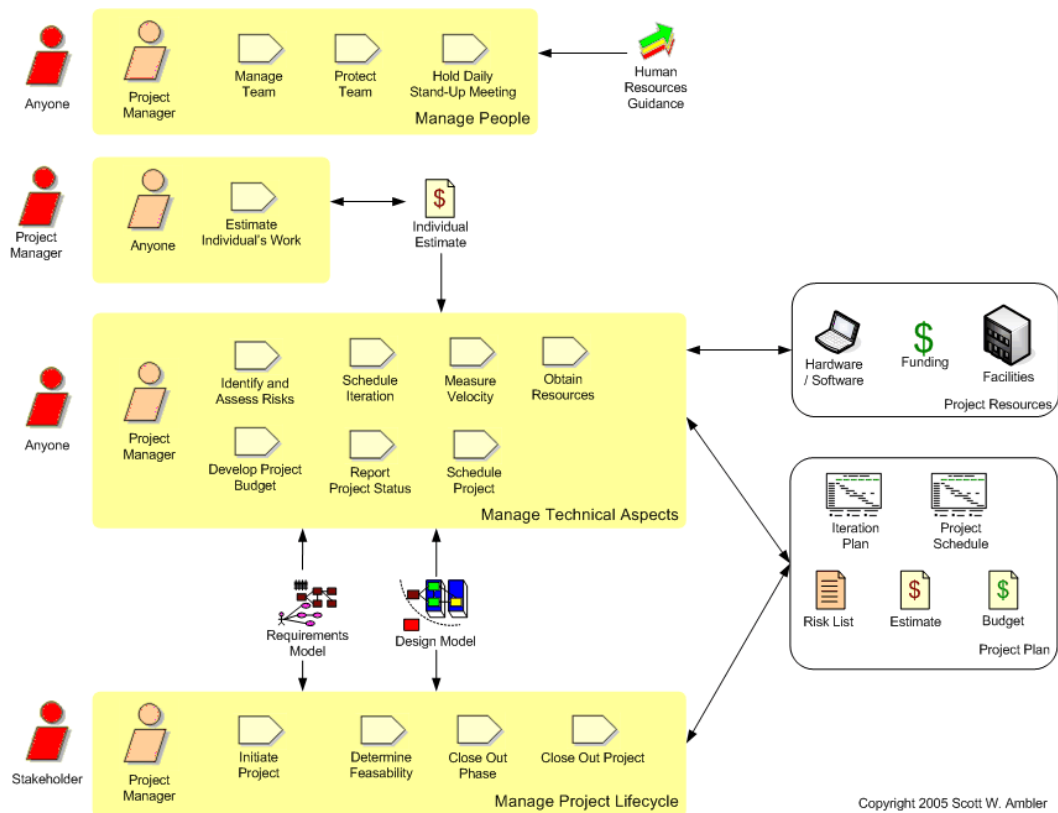


Figura 19: Disciplina de Administración del Proyecto del AUP

Fuente. - Tomado de <http://www.ambysoft.com/unifiedprocess/aup11/html/projectManagement.html>

Disciplina de Administración del Ambiente. El objetivo de esta disciplina es apoyar el resto del esfuerzo asegurando que el proceso adecuado, orientación (estándares y directrices) y herramientas (hardware, software, etc.) estén disponibles para el equipo según sea necesario.

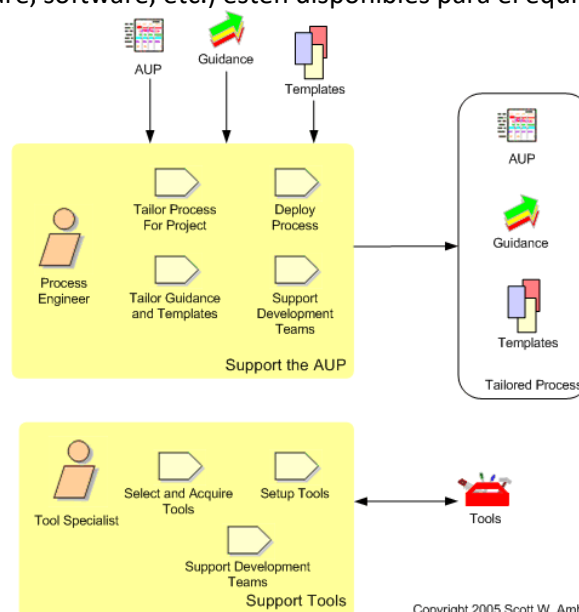


Figura 20: Disciplina de Administración del Ambiente del AUP

Fuente. - Tomado de <http://www.ambysoft.com/unifiedprocess/aup11/html/environment.html>

1.1.6.2. Roles AUP

A continuación, se muestran los roles en el orden según como van participando en las disciplinas: del Modelado a la Administración del ambiente. Algunos nombres de roles se mantienen en el idioma nativo (inglés), los cuales son conocidos en el ámbito de la Ingeniería de software.

Papel	Descripción	Disciplina(s)
Administrador del proyecto	Administra y protege a los miembros del equipo, construye relaciones con los stakeholders ⁵ , coordina las interacciones con los stakeholders, planifica, administra y asigna recursos, da forma a las prioridades y mantiene al equipo enfocado.	Modelado Prueba Despliegue Gestión de proyectos
Modelador ágil	Alguien que crea y desarrolla modelos, ya sean bocetos, tarjetas de índice o archivos complejos de herramientas CASE, de una manera evolutiva y colaborativa.	Modelado Implementación
Stakeholders	Es cualquier persona que es un usuario directo, usuario indirecto, gerente de usuarios, gerente sénior, miembro del personal de operaciones, miembro del personal de soporte (help desk), desarrolladores que trabajan en otros sistemas que integran o interactúan con el que está en desarrollo, o profesionales de mantenimiento potencialmente afectados por el desarrollo y/o implementación de un proyecto de software.	Modelado Implementación Prueba Despliegue Gestión de proyectos
Desarrollador	Un desarrollador escribe código fuente, prueba y construye software.	Modelado Implementación Despliegue
Administrador de la Configuración	Es responsable de proporcionar la infraestructura y crear el medio ambiente para el equipo de desarrollo.	Gestión de la configuración
DBA⁶ ágil	Es quien trabaja en colaboración con los miembros del equipo del proyecto para diseñar, probar, y brindar soporte a los esquemas de datos de la aplicación.	Implementación
Revisor	Evalúa los productos de trabajo del proyecto, a menudo "el trabajo en progreso", proporcionando retroalimentación al equipo de trabajo.	Prueba
Administrador de pruebas	Son responsables del éxito del esfuerzo de pruebas, incluyendo la planificación, la gestión y de promover las pruebas y las actividades de calidad.	Prueba
Tester	Son responsables de escribir, llevar a cabo y registrar los resultados de los esfuerzos de prueba.	Prueba
Implementador	Un implementador es responsable de poner el sistema en los ambientes de preproducción y producción.	Despliegue
Documentador técnico	Son responsables de producir documentación para los stakeholders, como materiales de capacitación, documentación de operaciones, documentación de soporte o mantenimiento y documentación del usuario.	Despliegue
Anyone	Cualquier persona en otro rol distinto.	Gestión de la configuración Gestión de proyectos
Especialista de Procesos	Desarrolla, adapta y da soporte a los materiales de procesos de software de sus organizaciones (descripciones de procesos, plantillas, orientación, ejemplos, ...).	Administración del ambiente
Especialista en herramientas	Los especialistas en herramientas son responsables de seleccionar, adquirir, configurar y dar soporte en herramientas.	Administración del ambiente

Figura 21: Tabla con Roles AUP

Fuente. - Elaboración propia

⁵ Stakeholders es un término en inglés. Se refiere a las partes interesadas o involucrados en el éxito de un proyecto.

⁶ DBA es el acrónimo en inglés de *Database Administrator*. En español es Administrador de Base de Datos

1.1.6.3. Un poco de historia

Scott Ambler es uno de los protagonistas en la historia de las metodologías ágiles. Es ingeniero de software, que nació en Canadá, y es reconocido por sus investigaciones y libros enfocados en el desarrollo ágil de proyectos de software. Escribió por primera vez sobre cómo mejorar el RUP en las páginas de Desarrollo de Software a mediados de 1999. La mayor parte de su trabajo de RUP se centró en ampliarlo a través de Enterprise Unified Process™ (EUP) para describir cómo convertir el RUP en un ciclo de vida de TI completo. Paralelamente, a partir de principios de 2001, comenzó a escribir sobre cómo "agilizar" el RUP a través de sus escritos en la web⁷ y en su libro de modelado ágil publicado en la primavera de 2002. Desde entonces otros, en particular Craig Larman, Gary Evans, Sinan Si Alhry el antiguo equipo de RUP de IBM, también han escrito sobre sus experiencias haciéndolo. Object Mentor incluso desarrolló un "XP plug-in" para el RUP para que se vea tanto como XP como sea posible. En septiembre de 2005, Scott liberó la primera versión del producto AUP en su sitio web, en la que se puede encontrar descripciones de todos los flujos de trabajo o disciplinas, y fases de la metodología.

1.1.6.4. ¿El equipo de desarrollo debería adoptar el AUP?

Si el equipo quiere algo entre XP o Programación Extrema (metodología ágil) y RUP tradicional, un proceso que es ágil pero que incluye explícitamente actividades y artefactos a los que está acostumbrado, entonces el AUP es una buena opción.

Muchas organizaciones son recelosas con XP porque parece ser demasiado ligera: XP no muestra explícitamente cómo crear algunos de los artefactos que la administración quiere ver. Esta es una actitud desafortunada porque XP es un gran proceso. En el otro extremo del espectro está RUP, que la administración parece amar, pero los desarrolladores parecen recelosos debido a la gran cantidad de artefactos. Esto también es desafortunado porque el RUP tiene mucho que ofrecer, y se puede reducir a algo bastante útil (que es exactamente lo que IBM Rational recomienda que haga).

Por consiguiente, el AUP aterriza entre los dos, adoptando muchas de las técnicas ágiles de XP y otros procesos ágiles conservando parte de la formalidad del RUP.

1.1.7. UML (Unified Modeling Language)

El Lenguaje Unificado de Modelado (UML) es un lenguaje de modelado visual que se usa para visualizar, especificar, construir y documentar artefactos de un sistema de software.

UML capta la información sobre la estructura estática y el comportamiento dinámico de un sistema. Un sistema se modela como una colección de objetos discretos que interactúan para realizar un trabajo que finalmente beneficia a un usuario externo. La estructura estática define los tipos de objetos. El comportamiento dinámico define la historia de los objetos en el tiempo y la comunicación entre objetos para cumplir sus objetivos. El modelar un sistema desde varios puntos de vista, separados pero relacionados, permite entenderlo para diferentes propósitos.

UML también contiene construcciones organizativas para agrupar los modelos en paquetes, lo que permite a los equipos de software dividir grandes sistemas en piezas de trabajo, para

⁷ Sitio web de las publicaciones de Scott Ambler: <http://www.ambysoft.com/unifiedprocess/agileUP.html>

entender y controlar las dependencias entre paquetes, y para gestionar las versiones de las unidades de modelo, en un entorno de desarrollo complejo. Contiene construcciones para representar decisiones de implantación y para elementos de tiempo de ejecución.

UML no es un lenguaje de programación. Las herramientas pueden ofrecer generadores de código de UML para una gran variedad de lenguajes de programación, así como construir modelos por ingeniería inversa a partir de programas existentes.

1.1.7.1. Un poco de historia

Los lenguajes de modelado de objetos aparecieron en la década de 1980, llegando a ser unos 50 a mediados de la década de 1990. Unos pocos métodos empezaron a ganar importancia, entre ellos: el Método OMT (Object-Modeling Technique) de James Rumbaugh, que era mejor para el análisis orientado a objetos, el Método Booch de Grady Booch, el cual era mejor para el diseño orientado a objetos, y el Método OOSE (Object-Oriented Software Engineering) de Ivar Jacobson, que era un método de ingeniería de software orientado a objetos.

El esfuerzo para la definición de UML comenzó en octubre de 1994, cuando Rumbaugh se unió a Booch en Rational Software Corporation (empresa donde trabajaba Booch). Unificaron sus métodos de modelado y elaboraron el borrador de la versión 0.8 de UML. En 1995 Jacobson también se incorporó a Rational, incorporando así OOSE a UML. En 1996 se publicó la versión 0.9 de UML. Las tres personalidades que dieron el origen a UML, y también al Proceso Unificado de Rational, son conocidas como “los Tres Amigos”.

Luego de varios años y varias modificaciones, OMG⁸ adoptó la versión oficial de UML⁹ 2.0 a principios del año 2005, versión que integra varios esfuerzos para la definición de una semántica de la especificación más sólida.

UML es evolutivo, actualmente va por la versión 2.5.1, la cual fue liberada en diciembre del 2017; esta versión presenta una estructura de lenguaje más modular y una gran capacidad mejorada para modelar sistemas a gran escala.

1.1.7.2. Objetivos de UML

Los objetivos de UML son muchos, y son utilizados no solo en metodologías orientada a objetos sino también en metodologías ágiles como el AUP. Sus objetivos se pueden sintetizar de la siguiente manera:

- **Visualizar:** UML permite expresar de una forma gráfica un sistema de forma que otro lo puede entender.
- **Especificar:** UML permite especificar cuáles son las características de un sistema antes de su construcción.
- **Construir:** A partir de los modelos especificados se pueden construir los sistemas diseñados.
- **Documentar:** Los propios elementos gráficos sirven como documentación del sistema desarrollado que pueden servir para su futura revisión.

⁸ OMG es la sigla en inglés de Object Management Group. Es una asociación, sin fines de lucro, que se encarga de la definición y mantenimiento de estándares para aplicaciones de la industria de la computación.

⁹ Sitio web de OMG que muestra las especificaciones de UML: <https://www.omg.org/spec/UML/>

1.1.8. Diagramas de UML 2.5

Actualmente, en UML existen diferentes tipos de diagramas. Para comprenderlos de manera concreta, es útil conocerlos jerárquicamente, como se muestra en la siguiente figura:

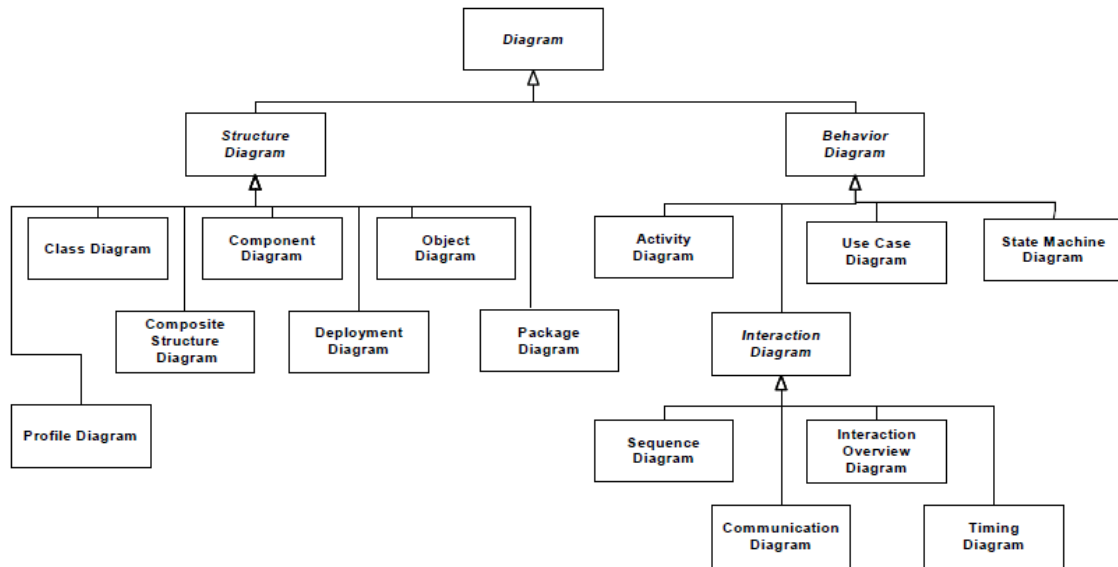


Figura 22: Taxonomía de los Diagramas de estructura y comportamiento de UML

Fuente. - Tomado de https://www.researchgate.net/figure/Overview-of-diagram-types-defined-by-UML-version-251-taken-from-the-official_fig2_344403670

Los Diagramas de Estructura enfatizan en los elementos que deben existir en el sistema modelado:

- Diagrama de clases.
- Diagrama de componentes.
- Diagrama de objetos.
- Diagrama de estructura compuesta (A partir de UML 2.0).
- Diagrama de despliegue.
- Diagrama de paquetes.

Los Diagramas de Comportamiento enfatizan en lo que debe suceder en el sistema modelado:

- Diagrama de actividades.
- Diagrama de casos de uso.
- Diagrama de máquina de estados.

Los Diagramas de Interacción son un subtipo de diagramas de comportamiento, que enfatiza sobre el flujo de control y de datos entre los elementos del sistema modelado:

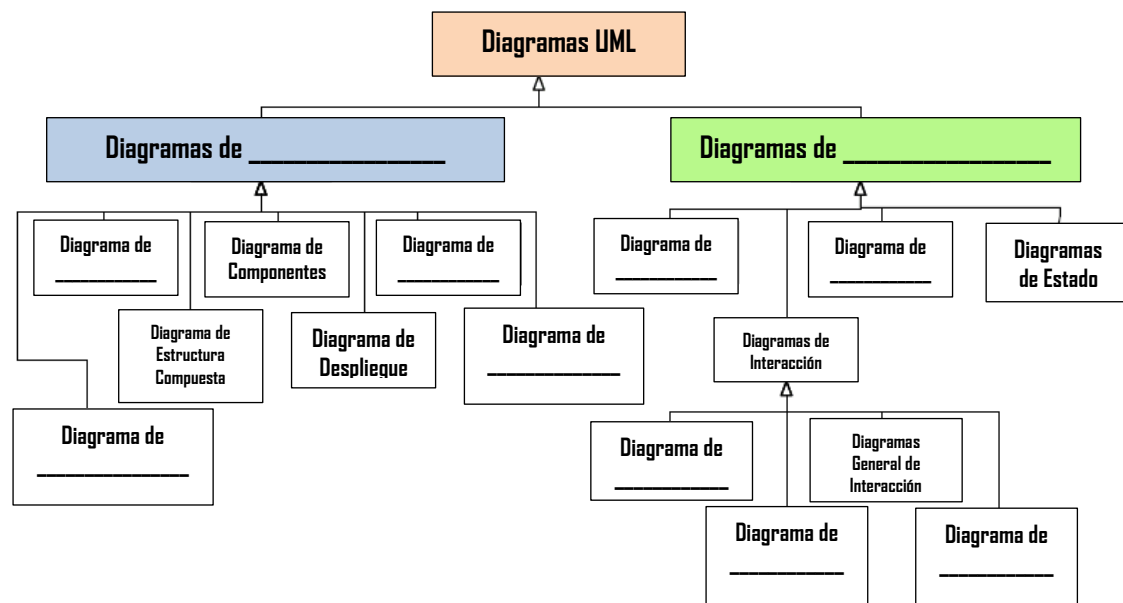
- Diagrama de secuencia.
- Diagrama de comunicación.
- Diagrama de tiempos (A partir de UML 2.0).
- Diagrama de descripción de la interacción (A partir de UML 2.0).

ACTIVIDADES PROPUESTAS

- Identifique las características de cada una de las tres metodologías: la metodología del ciclo de vida del desarrollo de sistemas (SDLC), metodologías ágiles y metodologías orientada a objetos. Para ello, elabore un cuadro comparativo con los tres tipos de metodologías, tal como se muestra a continuación. Proponga como mínimo 3 aspectos a comparar:

Cuadro comparativo de los tres tipos de Metodologías de desarrollo de sistemas			
ASPECTO	SDLC	Ágiles	Orientada a Objetos
Aspecto 1 para comparar			
Aspecto 2 para comparar			
...			
Aspecto "n" para comparar			

- Mencione los objetivos principales de cada disciplina de RUP.
- Mencione los objetivos principales de cada disciplina de AUP.
- Identifique los tipos de diagramas que podemos realizar con UML. Para ello, complete la taxonomía de los diagramas UML que se muestra a continuación.



Resumen

1. La ingeniería de software es una disciplina de la ingeniería que se interesa por todos los aspectos de la producción de software.
2. Un modelo de proceso de software es una representación simplificada de este proceso. Los modelos no describen los detalles de las actividades específicas, solo un marco del proceso. De modo que, los modelos genéricos no son descripciones definitivas de los procesos de software, sino abstracciones del proceso que se utilizan para explicar los diferentes enfoques del desarrollo de software. Se conocen los siguientes modelos: el modelo de cascada, desarrollo incremental y el orientado a la reutilización.
3. Una metodología de desarrollo de software, a diferencia de un modelo de proceso, define de forma específica cómo llevar a cabo un proyecto de desarrollo en términos de técnicas, herramientas y resultados a generar en cada actividad. Se conocen tres categorías: el ciclo de vida del desarrollo de sistemas (SDLC), las metodologías ágiles y las metodologías orientadas a objetos.
4. Las diferencias entre las tres metodologías de desarrollo de software (SDLC, ágiles y orientadas a objetos) no son tan grandes como algunos lo perciben. En las tres metodologías, se presentan actividades en común: el equipo necesita comprender a la organización, elaborar un presupuesto del tiempo y los recursos necesarios, entrevistar a los miembros de la organización y recopilar información detallada, obtener muestras de los datos de los informes existentes y observar cómo se lleva a cabo la actividad empresarial actual. Sin embargo, dependiendo del entorno de la empresa y el equipo de desarrollo, es importante considerar lineamientos que ayuden a elegir la metodología a utilizar.
5. La metodología RUP es orientada a objetos que tiene tres características esenciales: impulsado por el caso de uso, centrado en la arquitectura, e iterativo e incremental.
6. AUP es una metodología de desarrollo ágil que adopta una filosofía “en serie para lo grande” e “iterativa para lo pequeño” liberando entregables incrementales. Describe de una manera simple y fácil de entender la forma de desarrollar software usando técnicas ágiles y conceptos que aún se mantienen válidos en RUP.

Recursos

Pueden revisar los siguientes enlaces para ampliar los conceptos vistos en esta unidad:

- <http://www.enterpriseunifiedprocess.com/essays/history.html>
- <https://www.ibm.com/developerworks/rational/library/content/RationalEdge/jan01/WhatIsTheRationalUnifiedProcessJan01.pdf>
- <http://www.ambysoft.com/unifiedprocess/agileUP.html>
- <https://www.omg.org/spec/UML/2.5.1/PDF>

1.2. HERRAMIENTAS CASE

Las herramientas CASE (Computer Aided Software Engineering en inglés) o herramientas de Ingeniería de Software Asistida por Computadora, conocidas también como herramientas de productividad son diversas aplicaciones informáticas destinadas a aumentar la productividad en el desarrollo de software y reduce el costo de estas en términos de tiempo y de dinero.

Además, los analistas emplean herramientas CASE no solo para aumentar la productividad, sino también para comunicarse con los usuarios de una manera más efectiva e integrar el trabajo que realizan en el sistema, desde el inicio hasta el fin del ciclo de vida de desarrollo de software.

1.2.1. Objetivos de las herramientas CASE

A continuación, se indican los objetivos que se logran con la utilización de herramientas CASE:

- Mejorar la productividad en el desarrollo y mantenimiento del software.
- Aumentar la calidad del software.
- Mejorar el tiempo y coste de desarrollo y mantenimiento de los sistemas informáticos.
- Mejorar la planificación de un proyecto.
- Automatizar desarrollo del software, documentación, generación de código, pruebas de errores y gestión del proyecto.
- Ayudar a la reutilización del software, portabilidad y estandarización de la documentación.

1.2.2. Tipos de herramientas CASE

Según Kendall (2011) algunos analistas marcan la diferencia entre las herramientas CASE superiores e inferiores.

Una herramienta CASE superior permite al analista crear y modificar el diseño del sistema. Toda la información sobre el proyecto se almacena en una enciclopedia conocida como repositorio CASE, una extensa colección de registros, elementos, diagramas, pantallas, informes y demás información relacionada. Es posible producir informes del análisis mediante el uso de la información del repositorio para mostrar en qué partes está incompleto el diseño o dónde hay errores. Las herramientas CASE superiores también ayudan a sustentar el modelado de los requerimientos funcionales de una organización, auxiliar a los analistas y usuarios para dibujar los límites de un proyecto dado y ayudarlos a visualizar la forma en que el proyecto encaja con otras partes de la organización.

Las herramientas CASE inferiores se utilizan para generar código fuente de computadora, con lo cual se elimina la necesidad de programar el sistema; ello ofrece varias ventajas:

- El sistema se puede producir con más rapidez que si se escribieran programas computacionales.
- La cantidad de tiempo invertido en el mantenimiento se reduce con la generación de código.
- Se puede generar código en más de un lenguaje computacional, por lo que es más sencillo migrar los sistemas de una plataforma a otra.
- La generación de código provee una manera efectiva en costo de personalizar los sistemas que se compran a terceros distribuidores para ajustarlos a las necesidades de la organización.
- El código generado está libre de los errores típicos de los programas computacionales.

Existe otra clasificación, el cual se enfoca según las fases del ciclo de desarrollo que cubren:

- **Upper CASE (U-CASE)**, herramientas que ayudan en las fases de planificación, análisis de requisitos y estrategia del desarrollo, usando, entre otros, diagramas UML.
- **Middle CASE (M-CASE)**, automatizan tareas en el análisis y diseño de la aplicación.
- **Lower CASE (L-CASE)**, herramientas que semiautomatizan la generación de código, crean programas de detección de errores, soportan la depuración de programas y pruebas. Además, automatizan la documentación completa de la aplicación. Aquí pueden incluirse las herramientas de Desarrollo rápido de aplicaciones.
- **Integrated CASE (I-CASE)**, engloban todo el proceso de desarrollo de software.

1.2.3. Ejemplos de herramientas CASE

A continuación, se muestran algunos entornos que soportan UML:

- IBM Engineering Systems Design Rhapsody
- IBM Rational Software Architect
- Enterprise Architect
- Start UML

1.2.3.1. IBM Engineering Systems Design Rhapsody

IBM Engineering Systems Design Rhapsody (Rational Rhapsody) y su familia de productos ofrece una solución comprobada para actividades de modelación y diseño de sistemas que le permite gestionar la complejidad que muchas organizaciones enfrentan con el desarrollo de productos y sistemas. Rhapsody forma parte del portafolio de ingeniería de IBM, que proporciona un desarrollo de diseño colaborativo y un entorno de prueba para ingenieros de sistemas que da soporte a UML, SysML, UAF y AUTOSAR. **Rhapsody** presenta 4 ediciones, tal como se muestra en la siguiente figura. La edición Designer for Systems Engineer es la más completa.



Figura 23: Ediciones del Rational Rhapsody
Fuente. - Tomado durante la instalación del software

Este entorno permite que el profesional de sistemas, dependiendo del rol que cumpla en el equipo de desarrollo de software:

- Analice y elabore los requisitos de proyecto
- Pase rápidamente del diseño a la implementación
- Automatice revisiones de diseño y genere documentación
- Cree prototipos, simule y ejecute diseños para una validación temprana
- Trabaje en un entorno de ingeniería ágil e incorporado en tiempo real

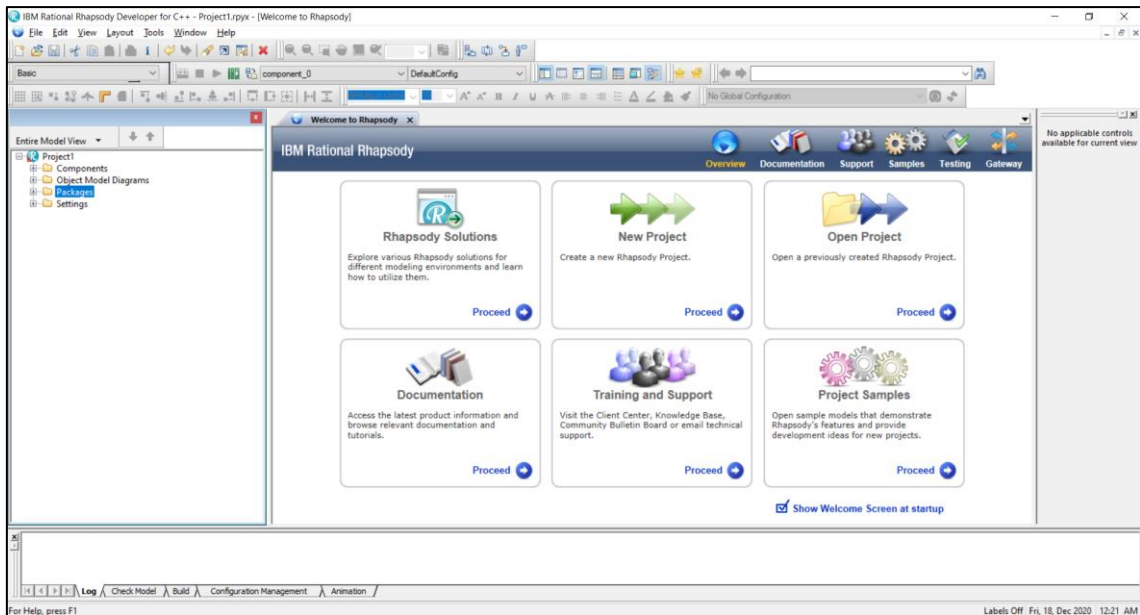


Figura 24: Secciones de la vista principal del Rational Rhapsody
Fuente. - Tomado durante la instalación del software

1.2.3.2. IBM Rational Software Architect (IBM RSA)

IBM RSA es una herramienta de diseño y construcción para arquitectos de software y desarrolladores sénior para crear aplicaciones en la plataforma Java o en C++. Permite un desarrollo basado en modelos con el lenguaje UML (Unified Modeling Language) y unifica todos los aspectos de la arquitectura de la aplicación de software.

La versión actual del Rational Software Architect es 9.7.x la cual trae una mejora en cuanto a creación de modelos y diagramas se refiere.

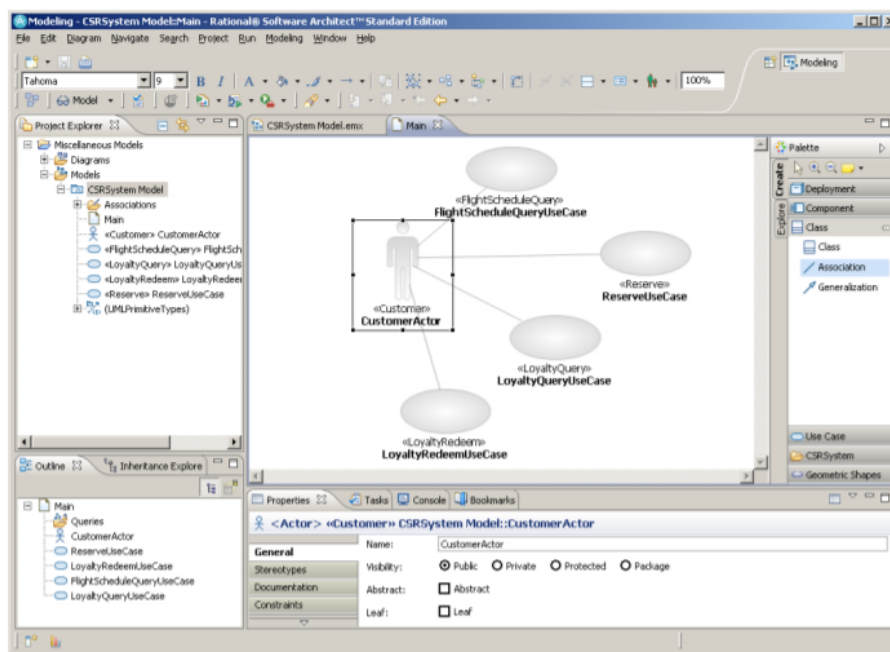
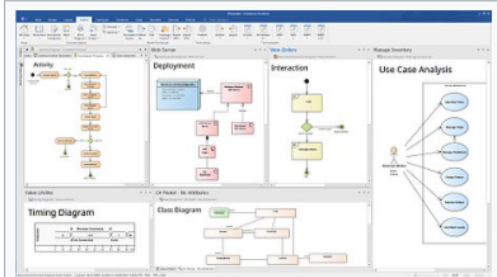


Figura 25: IBM Rational Software Architect
Fuente. - Tomado durante la instalación del software

1.2.3.3. Enterprise Architect

Enterprise Architect es una herramienta de análisis y diseño intuitiva, flexible y poderosa para construir software robusto y mantenible. Desde la recolección de requerimientos, pasando por el análisis, modelado, implementación y pruebas hasta despliegue y mantenimiento; Enterprise Architect es una herramienta de modelado UML rápida, rica en funcionalidad y multiusuario que conduce el éxito de su proyecto de software.

Sparx Systems Enterprise Architect



UML diagrams in Enterprise Architect

Developer(s)	Sparx Systems
Stable release	15.2 Build 1554 (27 August 2020)
Operating system	Windows, Linux via Wine, macOS via CrossOver
Available in	English, German, Japanese, Spanish, Chinese, French
Type	Software modeling, software development, business process modelling
License	proprietary
Website	www.sparxsystems.com

Figura 26: Información actual de Enterprise Architect

Fuente. - Tomado de [https://en.wikipedia.org/wiki/Enterprise_Architect_\(software\)](https://en.wikipedia.org/wiki/Enterprise_Architect_(software))

Enterprise Architect es una herramienta gráfica multiusuario diseñada para al equipo de desarrollo a construir sistemas robustos y mantenibles. Además, posee facilidades de incorporadas de reportes y documentación, de alta calidad. También se considera que es muy ligera y rápida en su uso. Permite el trabajo distribuido por lo que múltiples usuarios pueden crear modelos en paralelo. Posee además la capacidad de integrarse a controladores de versiones.

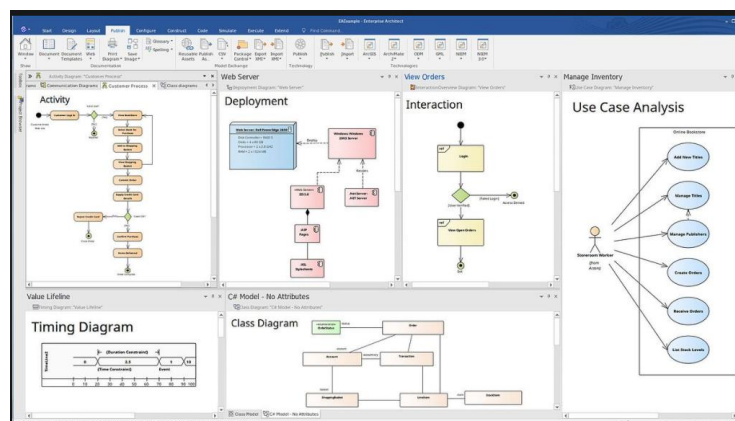


Figura 27: Enterprise Architect

Fuente. - Tomado de [https://en.wikipedia.org/wiki/Enterprise_Architect_\(software\)](https://en.wikipedia.org/wiki/Enterprise_Architect_(software))

1.2.3.4. StarUML

StarUML es una herramienta desarrollada por MKLab. Este software está licenciado bajo una versión modificada de GNU GPL hasta el 2014, fecha en que una versión reescrita 2.0.0 fue liberado para pruebas beta bajo una licencia propietaria.

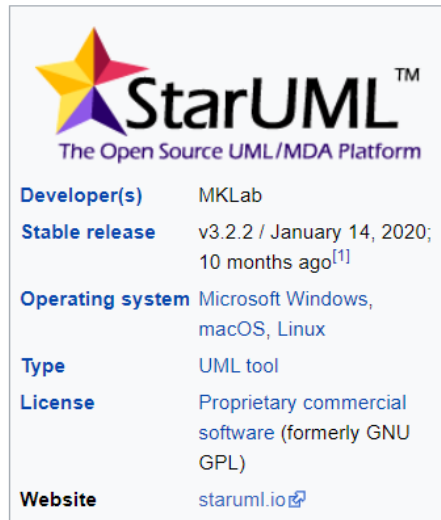


Figura 28: Información actual de StarUML
Fuente. - Tomado <https://en.wikipedia.org/wiki/StarUML>

Después de ser abandonado por algún tiempo, el proyecto tuvo un renacimiento para pasar de Delphi para Java/Eclipse y luego se detuvo de nuevo. En el 2014, la versión reescrita fue lanzada como software propietario. Sin embargo, la comunidad de la versión de código abierto aún continúa activa.

StarUML soporta la mayoría de los tipos de diagramas UML especificados en su versión 2.x. Su versión más reciente de sus autores originales está disponible para su descarga bajo el nombre de StarUML2, aunque no bajo la licencia GPL. Esta nueva versión incluye además soporte para extensiones, compatibilidad con el sistema operativo OSX y una nueva interfaz de usuario.

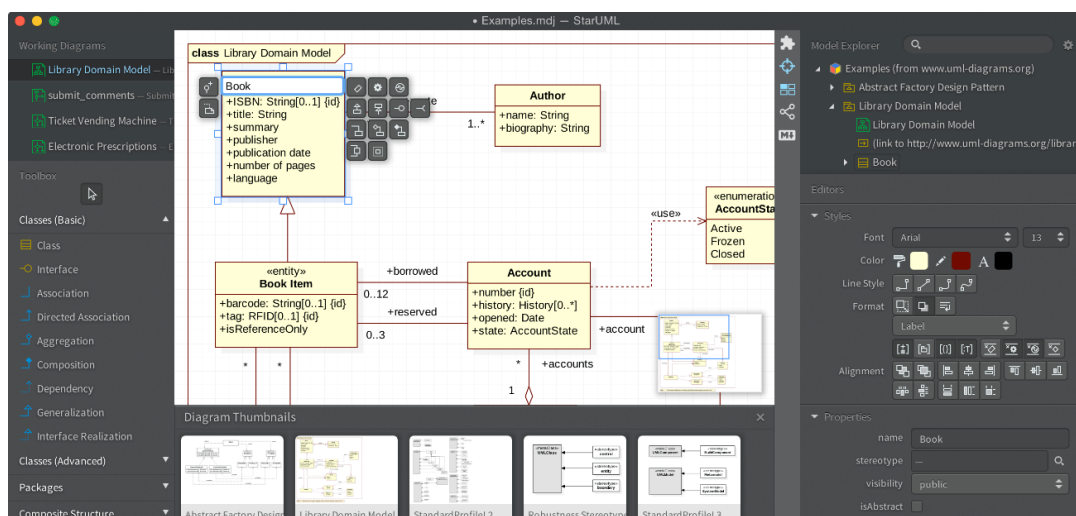


Figura 29: Información actual de StarUML
Fuente. - Tomado de <https://staruml.io/>

Resumen

1. Las herramientas CASE o herramientas de Ingeniería de Software Asistida por Computadora son aplicaciones informáticas destinadas a aumentar la productividad en el desarrollo de software. Asimismo, reducen los costos en términos de tiempo y dinero, permiten la comunicación entre los miembros del equipo de desarrollo y los usuarios durante el ciclo de vida de desarrollo de software.
2. Existe una clasificación de las herramientas CASE enfocada a cada fase del ciclo de desarrollo: Upper CASE, Middle CASE, Lower CASE e Integrated CASE.
3. Los entornos CASE representativos que soportan UML son IBM Engineering Systems Design Rhapsody, IBM Rational Software Architect, Enterprise Architect y Start UML.

Recursos

Pueden revisar los siguientes enlaces para ampliar los conceptos vistos en esta unidad:

- https://www.tutorialspoint.com/es/software_engineering/case_tools_overview.htm
- <https://www.ibm.com/pe-es/products/systems-design-rhapsody>
- <https://sparxsystems.com/>
- <https://staruml.io/>



DISCIPLINA DE MODELADO – PARTE I

LOGRO DE LA UNIDAD DE APRENDIZAJE

Al término de la unidad, el alumno sustenta el primer avance de su proyecto relacionado a la elaboración de la documentación del estudio de Negocio de un caso de estudio utilizando una Herramienta CASE. Este avance incluye el desarrollo del Modelo de Casos de Uso de Negocio (MCUN) y el Modelo de Análisis de Negocio (MAN).

TEMARIO

2.1 Tema 3 : Modelado de negocio

- 2.1.1 : ¿Cuándo es necesario realizar el modelado de negocio?
- 2.1.2 : ¿Cuándo no es necesario realizar el modelado de negocio?

2.2 Tema 4 : Visión general del entorno CASE IBM Engineering Systems Design Rhapsody

- 2.2.1 : Proyectos, vistas y toolbars
- 2.2.2 : Toolkit para ingeniería de sistemas
- 2.2.3 : Tips de navegación
- 2.2.4 : Creación de un proyecto con la estructura del modelado

2.3 Tema 5 : Modelo de casos de uso del negocio – MCUN

- 2.3.1 : Artefactos del modelo de casos de uso de negocio
- 2.3.2 : Reglas básicas de nomenclaturas
- 2.3.3 : Actividades para identificar los artefactos de la vista externa del negocio
- 2.3.4 : Identificación de artefactos del modelo de casos de uso de negocio para el caso fashion importaciones

2.4 Tema 6 : Modelo de análisis del negocio – MAN

- 2.4.1 : Artefactos del modelo de análisis de negocio
- 2.4.2 : Reglas básicas de nomenclaturas
- 2.4.3 : Actividades para identificar los artefactos de la vista interna del negocio
- 2.4.4 : Identificación de artefactos del modelo de análisis de negocio para el caso fashion importaciones

2.5 Tema 7 : Caso propuesto de MCUN y MAN

- 2.5.1 : Caso: Play and Learning Zone S.A.C
- 2.5.2 : Caso: Impulse Center
- 2.5.3 : Otros casos propuestos

ACTIVIDADES PROPUESTAS

- Los alumnos identifican los artefactos del Modelo de Casos de Uso de Negocio (MCUN) de un caso:
 - Objetivos de Negocio
 - Casos de Uso de Negocio
 - Actores de Negocio
- Los alumnos identifican los artefactos del Modelo de Análisis de Negocio (MAN) de un caso:
 - Trabajadores de Negocio
 - Entidades de Negocio
 - Diagrama de Clases y Diagrama de Actividades para un Caso de Uso de Negocio

2.1. MODELADO DE NEGOCIO

Tanto para RUP como para AUP es importante entender el negocio antes de empezar con la implementación del software. El estudio del negocio en AUP, se encuentra en el flujo de trabajo del Modelado, el cual se enfoca en artefactos que sean “solo lo suficientemente buenos”, es decir, generar el artefacto que sea lo suficientemente preciso y suficientemente detallado (just barely good enough) para entender la situación actual del negocio y el dominio del problema. En este sentido, (Ambler, 2006) nos sugiere que no es necesario crear todos los modelos que se muestran en el diagrama de flujo del trabajo de Modelado, sino solo los que son adecuados para la situación del proyecto.

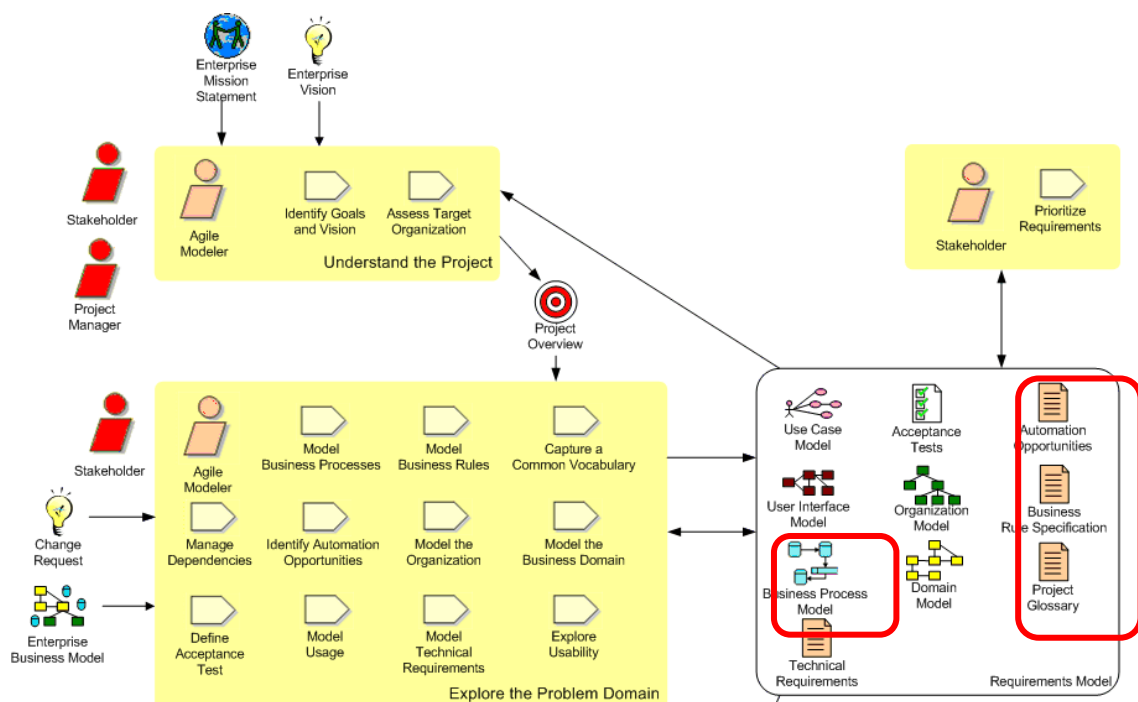


Figura 30: Artefactos relevantes del flujo de trabajo de Modelado

Fuente. - Tomado de <http://www.ambysoft.com/unifiedprocess/aup11/html/model.html>

En el curso, desarrollaremos los siguientes artefactos, los cuales incluyen a los artefactos de la figura 30 resaltados en rojo:

- Situación del Negocio (Documento que contiene las siguientes secciones: Objetivos del negocio, Alcance del proyecto, Situación actual del Negocio, Situación propuesta del Negocio que presenta los Casos de Uso de Negocio con Oportunidades de automatización, Especificación de Reglas del Negocio y Glosario de términos del proyecto).
- Modelo de Casos de Uso de Negocio (Diagramas que incluye Actores de Negocio, Casos de Uso de Negocio y Diagrama General de Casos de Uso de Negocio) como parte del Modelo de Procesos de Negocio de AUP.
- Modelo de Análisis de Negocio (Diagramas que incluye Entidades de Negocio, Trabajadores de Negocio, y por cada caso de uso de negocio se crea un Diagrama de Clases y un Diagrama de Actividades) como parte del Modelo de Procesos de Negocio de AUP.

2.1.1. ¿Cuándo es necesario realizar el modelado de negocio?

La necesidad del estudio del negocio surge ante el hecho de que muchos de los productos software que se desarrollan automatizan algunos o todos los procesos existentes en un negocio, y es necesario estudiar las implicaciones de los cambios producidos al adoptar estos productos. Hay que entender cómo funciona el negocio que se desea automatizar para tener garantías de que el software desarrollado va a cumplir su propósito, y por esto, se hace un estudio en el dominio del negocio además de en el dominio del software.

Los creadores de RUP señalan que el modelo de negocio está soportado por dos artefactos principales, los mismos que se pueden desarrollar bajo un enfoque ágil en AUP, según Ambler, (2006), considerando elementos relevantes del modelo y utilizando una herramienta CASE que permita visualizar la trazabilidad de artefactos que se van generando de un modelo a otro:

- Modelo de casos de uso del negocio (MCUN)
- Modelo de análisis del negocio (MAN)

El modelo de casos de uso de negocio describe los procesos de negocio de una empresa en términos de casos de uso del negocio y actores del negocio que se corresponden con los procesos del negocio y los clientes, respectivamente. Por otro lado, el modelo de análisis del negocio es un modelo interno a un negocio, que describe cómo cada caso de uso de negocio es llevado a cabo por un grupo de trabajadores que utilizan entidades del negocio.

Por consiguiente, se podría precisar que es necesario realizar el modelado de negocio cuando:

- El grupo de trabajo es nuevo en la organización.
- La organización ha enfrentado un reciente proceso de reingeniería de negocios.
- La organización está planificando un proceso de reingeniería de negocios.
- El software por construir será utilizado por una parte importante de la organización.
- Existen flujos de trabajo complejos, dentro de la organización, que no están documentados.
- Cuando se es un consultor en una organización en la cual no se ha trabajado antes.

2.1.2. ¿Cuándo no es necesario realizar el modelado de negocio?

Definitivamente, tanto para RUP como para AUP no consideran realizar un estudio del negocio cuando:

- Se tiene un conocimiento de la estructura de la organización, de las metas, de la visión y de los clientes/usuarios.
- El software por construir será usado por una pequeña parte de la organización, y no tiene un impacto en el resto del negocio.
- Los flujos de trabajo de la organización están bien documentados.
- El tiempo no lo permita, no todos los procesos tienen el tiempo necesario para completar un análisis de negocio.

Resumen

1. Para cualquier metodología de desarrollo de software es importante entender el negocio antes de empezar con la construcción del software. En RUP el estudio de negocio se desarrolla en la disciplina “Business Modeling”, mientras que en AUP, este estudio es parte de la disciplina de Modelado.
2. AUP se enfoca en artefactos que sean “solo lo suficientemente buenos”, es decir, define que se debe generar el artefacto que sea lo suficientemente preciso y suficientemente detallado (just barely good enough) para entender la situación actual del negocio y el dominio del problema. En este sentido, (Ambler, 2006) nos sugiere que no es necesario crear todos los modelos que se muestran en el diagrama de flujo del trabajo de Modelado, sino solo los que son adecuados para la situación del proyecto.
3. La necesidad del estudio del negocio surge ante el hecho de que muchos de los productos software que se desarrollan automatizan algunos o todos los procesos existentes en un negocio, y es necesario estudiar las implicaciones de los cambios producidos por la adopción de estos productos.
4. Los creadores de RUP señalan que el modelo de negocio está soportado por dos artefactos principales, los mismos que se pueden desarrollar bajo un enfoque ágil en AUP, según (Ambler, 2006), considerando elementos relevantes del modelo y utilizando una herramienta CASE que permita visualizar la trazabilidad de artefactos que se van generando de un modelo a otro: Modelo de Casos de Uso de Negocio (MCUN) y Modelo de Análisis de Negocio (MAN).

Recursos

Pueden revisar los siguientes enlaces para ampliar los conceptos vistos en esta unidad:

- http://www.michael-richardson.com/processes/rup_classic/#extend.bus_model/disciplines/rup_business_modeling_discipline_553E20F6.html

2.2. VISIÓN GENERAL DEL ENTORNO CASE IBM ENGINEERING SYSTEMS DESIGN RHAPSODY

La herramienta CASE de IBM conocida como IBM Engineering Systems Design Rhapsody presenta tres características importantes en comparación con IBM Rational Software Architect:

- Permite crear matrices de trazabilidad entre artefactos de un modelo a otro.
- Permite importar de una forma fácil elementos de un archivo CSV (actividades a sistematizar, requisitos y casos de uso) para generar matrices de trazabilidad en un proyecto.
- Permite diseñar prototipos (GUI de casos de uso).

2.2.1. Proyectos, vistas y toolbars

Rational Rhapsody es un entorno CASE que ofrece diversos idiomas para su instalación. Sin embargo, se sugiere que se mantenga la instalación en inglés a fin de estar familiarizado con la terminología que comúnmente se utilizan en inglés en el área de la Ingeniería de Software.

Inicio de Rational Rhapsody

Al iniciar Rational Rhapsody, se mostrará una ventana de Bienvenida, la cual presenta enlaces para ver tutoriales, para crear proyectos, para abrir proyectos, y ejemplos de proyectos:

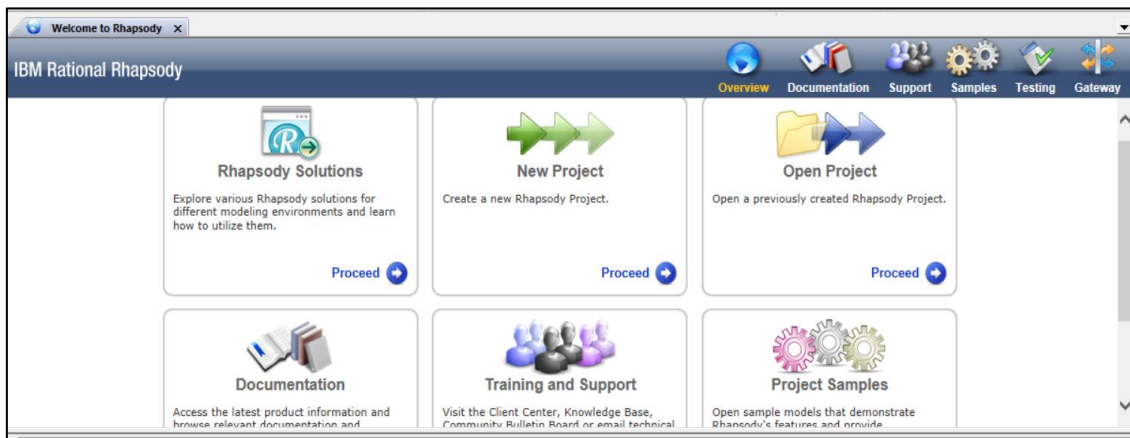


Figura 31: Página de Bienvenida del Rational Rhapsody

Fuente. - Elaboración propia

Creación de un proyecto

Para crear un proyecto con modelado UML, se sigue 4 pasos:

- 1) Se selecciona opción “New Project” desde la vista de Bienvenida. El tipo de proyecto por defecto es UML. De modo que, no es necesario cambiar esta opción.
- 2) Se edita un nombre de proyecto.
- 3) Se selecciona una ruta para la carpeta donde se creará el proyecto. Esta carpeta generalmente es conocida como “WorkSpace”. Para este ejemplo, se creará la carpeta “WS_Rhapsody”.
- 4) Se selecciona botón “OK”.

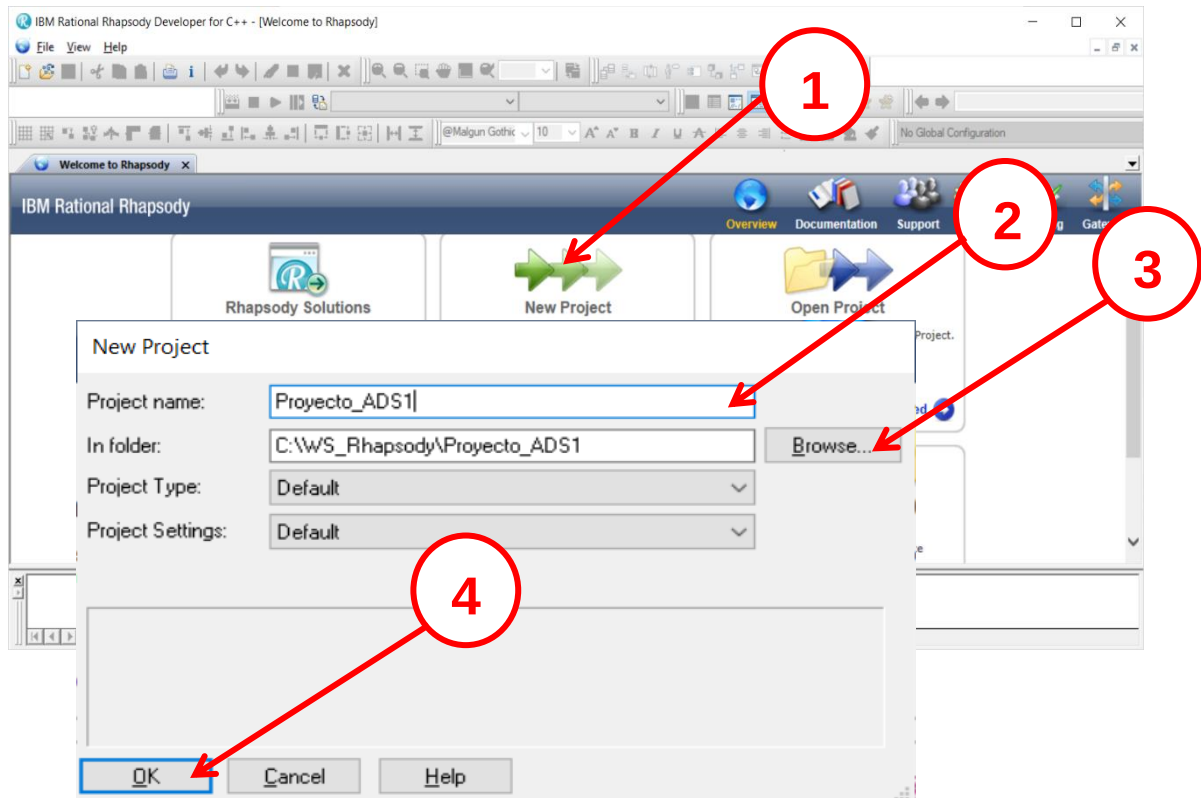


Figura 32: Creación de proyecto desde página de Bienvenida
Fuente. - Elaboración propia

A continuación, se mostrará las vistas y secciones del menú principal para la perspectiva UML básica. Es conveniente trabajar con la perspectiva UML Avanzada, la cual mostrará las opciones de creación de todos los tipos de Diagramas UML. Para ello seleccione “Advanced” en lugar de “Basic” desde la barra de herramientas:

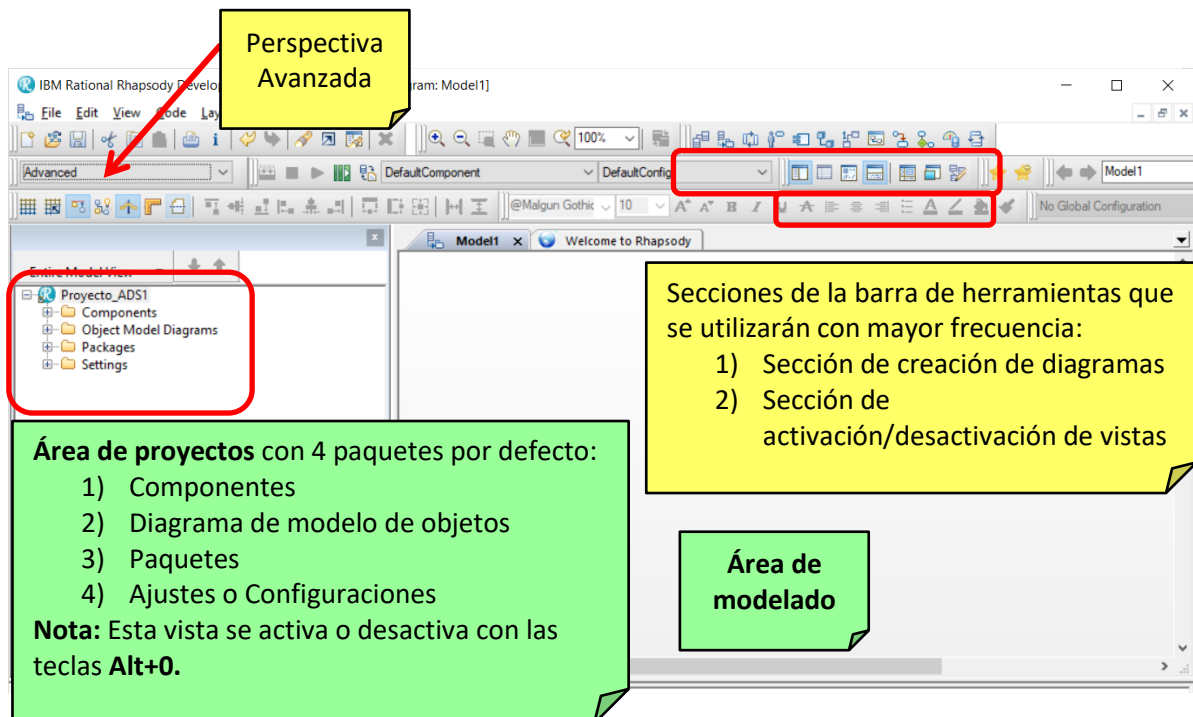


Figura 33: Vistas del Rational Rhapsody con perspectiva UML avanzada
Fuente. - Elaboración propia

2.2.2. Toolkit para ingeniería de sistemas

En Rational Rhapsody existe una opción para crear modelos sobre la base de un perfil que ofrece un toolkit para Ingeniería de Sistemas, el cual consiste en un set de modelos para un proyecto. Este toolkit se encuentra en el profile o perfil “Harmony SE” y puede consultarse para familiarizarse con una estructura completa del ciclo de vida de desarrollo de software.

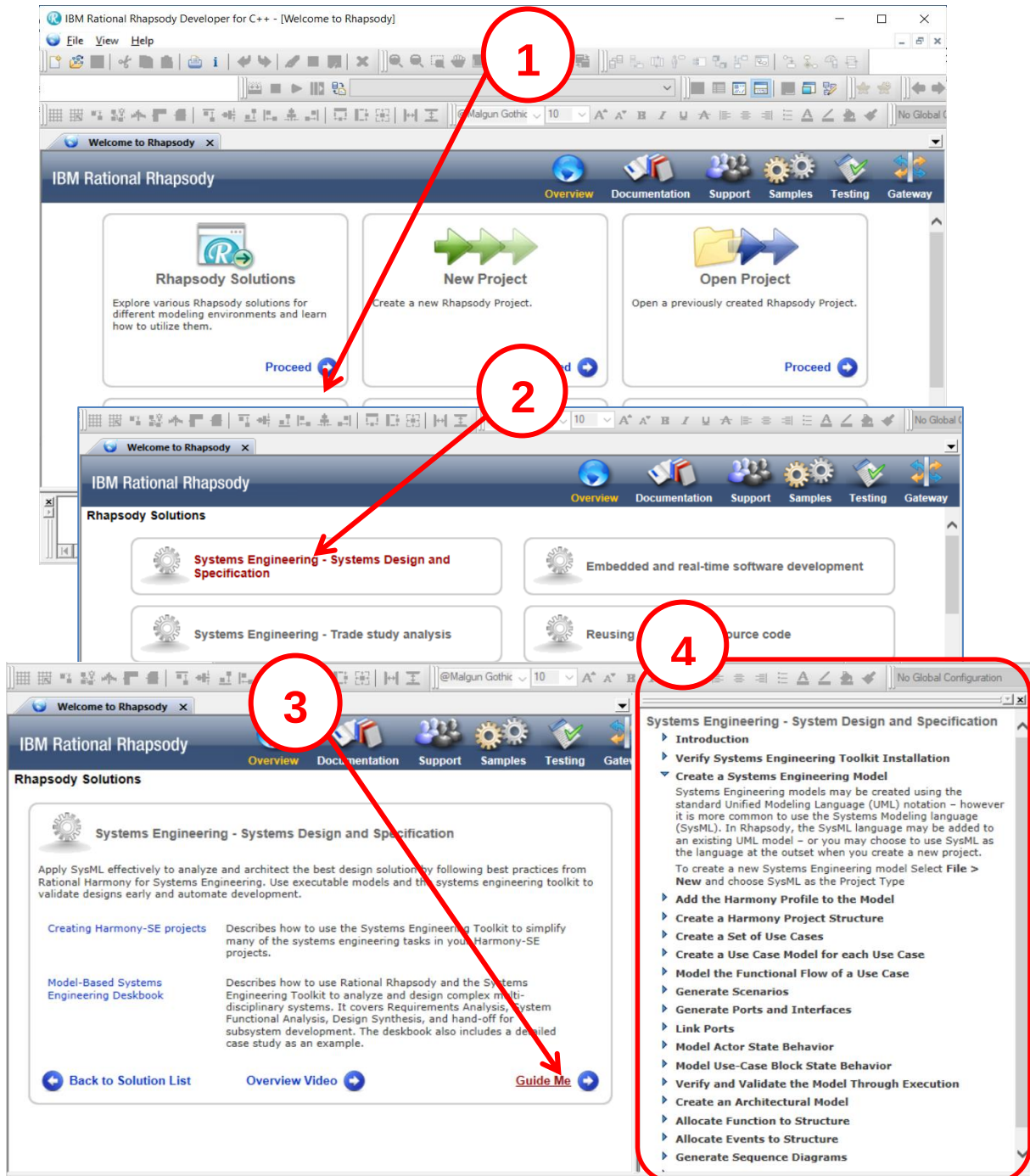


Figura 34: Pasos para ingresar a la guía del Toolkit de Ingeniería de Sistemas.

Fuente. - Elaboración propia

En el paso 4 de la figura anterior se muestra la explicación del proceso de cómo crear y ejecutar modelos SysML utilizando Rational Rhapsody, el kit de herramientas de ingeniería de sistemas y las mejores prácticas de Rational para la ingeniería de sistemas basada en modelos.

SysML es una notación UML pero que ya no solo sirve para aplicar a software, sino también a otro tipo de sistemas (Físicos, procesos, procedimientos, personas y otros incluyendo también software). (SysML, 2003-2021)

2.2.3. Tips de navegación

Una vez creado el proyecto, podrá navegar por las opciones presentadas en la barra de menú. Las opciones que se utilizará con mayor frecuencia son las siguientes:

- 1) Para crear diagramas:
 - a. Diagrama de clases
 - b. Diagrama de modelo de objetos, el cual puede utilizarse como un diagrama de formato libre debido a que puede agregarse cualquier componente UML.
 - c. Diagrama de secuencia
 - d. Diagrama de casos de uso
 - e. Diagrama de componenetes
 - f. Diagrama de despliegue
 - g. Diagrama de comunicación
 - h. Diagrama de estructura
 - i. Diagrama de estado
 - j. Diagrama de actividades
 - k. Diagrama de panel que contiene componentes para diseñar un prototipo
 - l. Diagrama de tiempo
- 2) La opción de Mostrar y Ocultar panel de herramientas para crear los elementos sobre un diagrama. Este panel presenta los elementos que se agregarán en el diagrama creado.

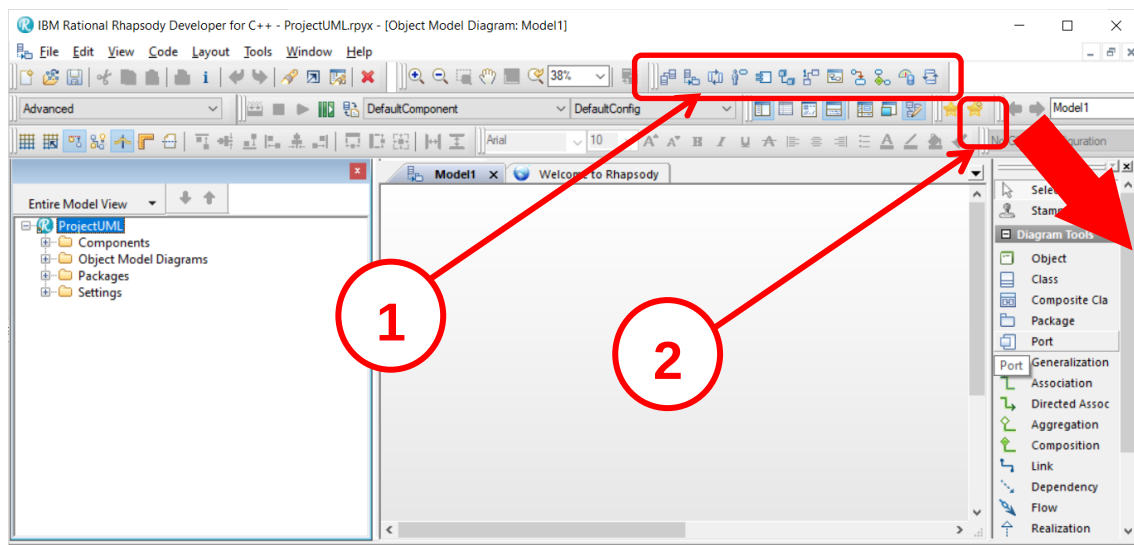


Figura 35: Tips de navegación para crear diagramas y sus elementos

Fuente. - Elaboración propia

2.2.4. Creación de un proyecto con la estructura de modelado

Los pasos para crear la estructura de un proyecto son los siguientes:

Paso 1: Empezamos con la copia de la carpeta profile, el cual contiene el perfil del BusinessModeling, al directorio del workspace de proyectos Rhapsody.

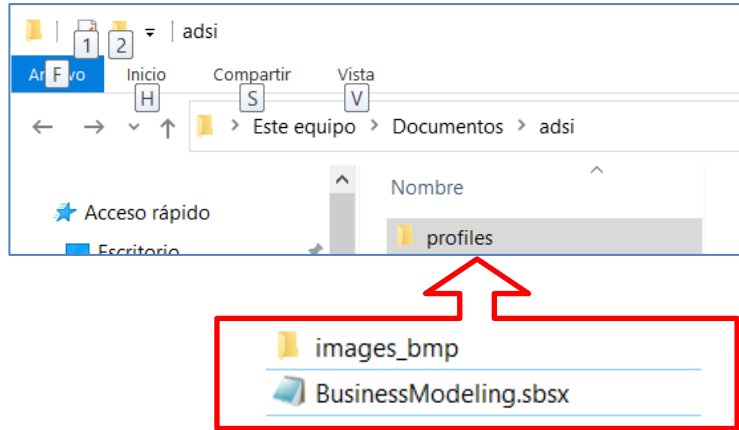


Figura 36: Carpeta que contiene perfil del Business Modeling con extensión *.sbsx
Fuente. - Elaboración propia

Paso 2: A continuación, se crea un proyecto.

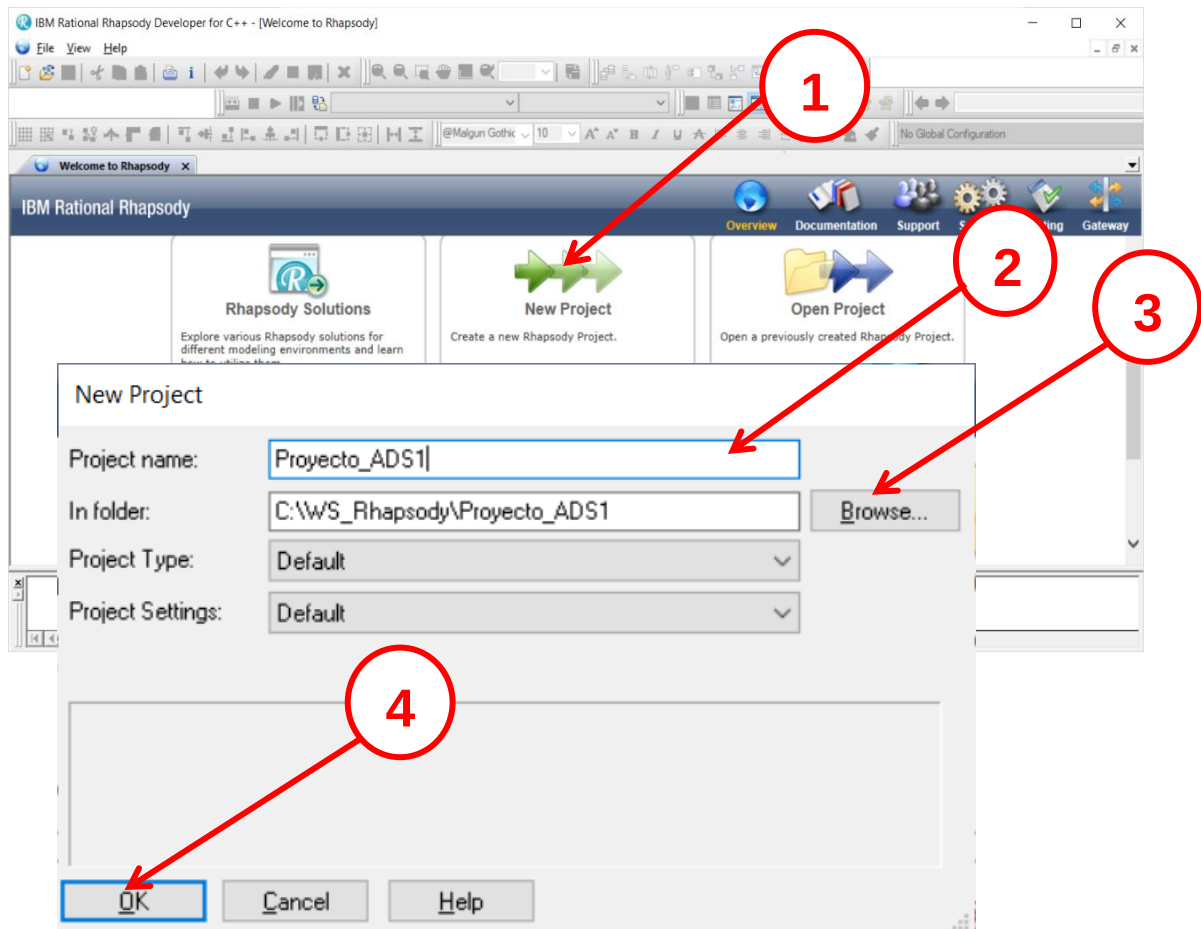


Figura 37: Creación de proyecto desde página de Bienvenida
Fuente. - Elaboración propia

Paso 3: Una vez creado el proyecto, se activa la perspectiva UML avanzada

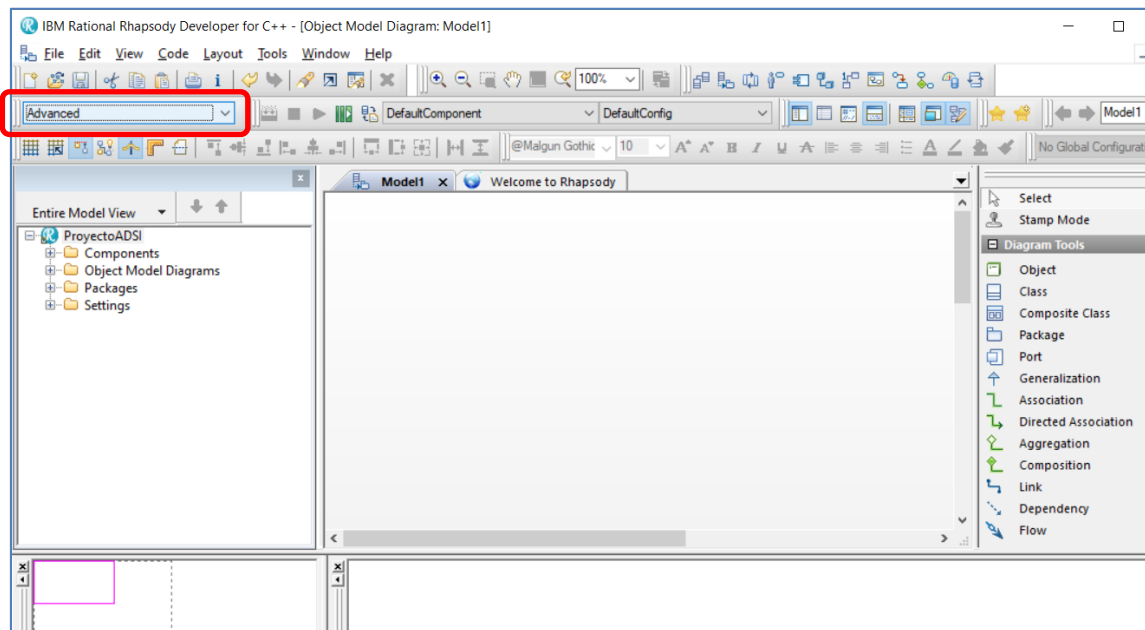


Figura 38: Vista de proyecto con perspectiva UML avanzada

Fuente. - Elaboración propia

Paso 4: Ahora, agregue el perfil del Business Modeling al proyecto creado. Para ello, seleccione el proyecto del browser de proyectos; luego, seleccione menú “File” y opción “Add Profile to Model...”. Finalmente, se ubica el archivo del perfil con extensión *.sbsx.

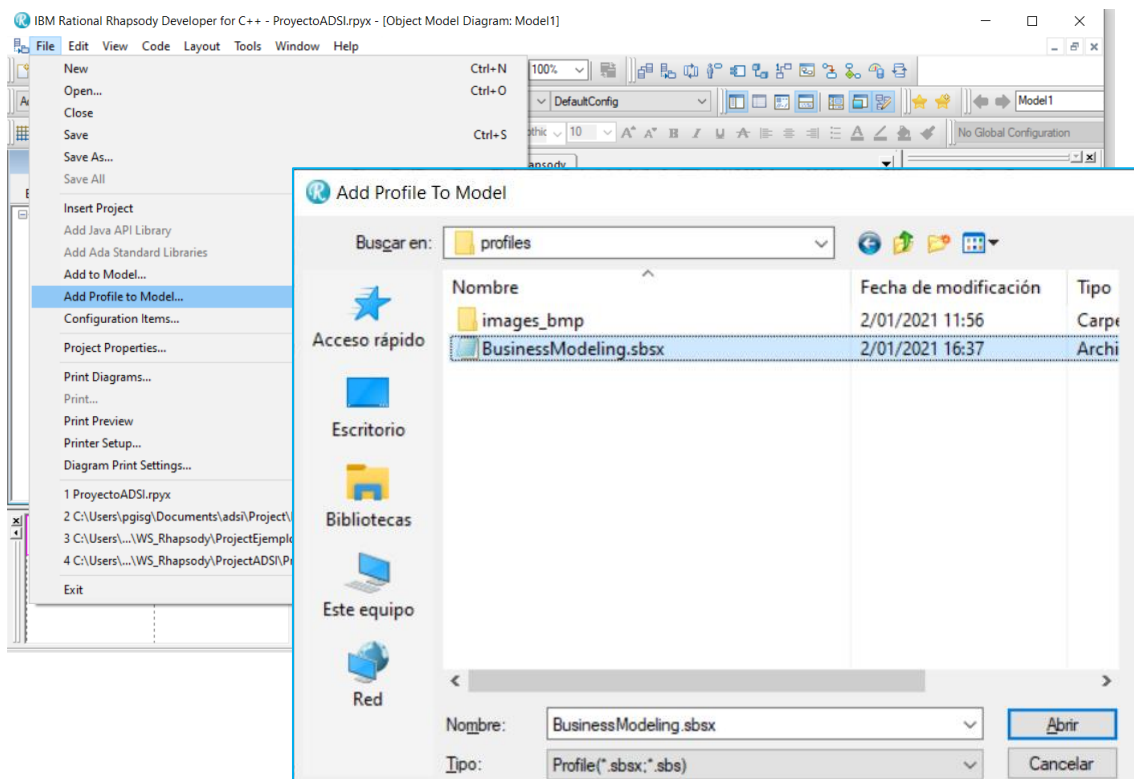


Figura 39: Agregar perfil con extensión *.sbsx a proyecto

Fuente. - Elaboración propia

Paso 5: A continuación, se visualizará el perfil cargado en la carpeta **Profiles** del proyecto.

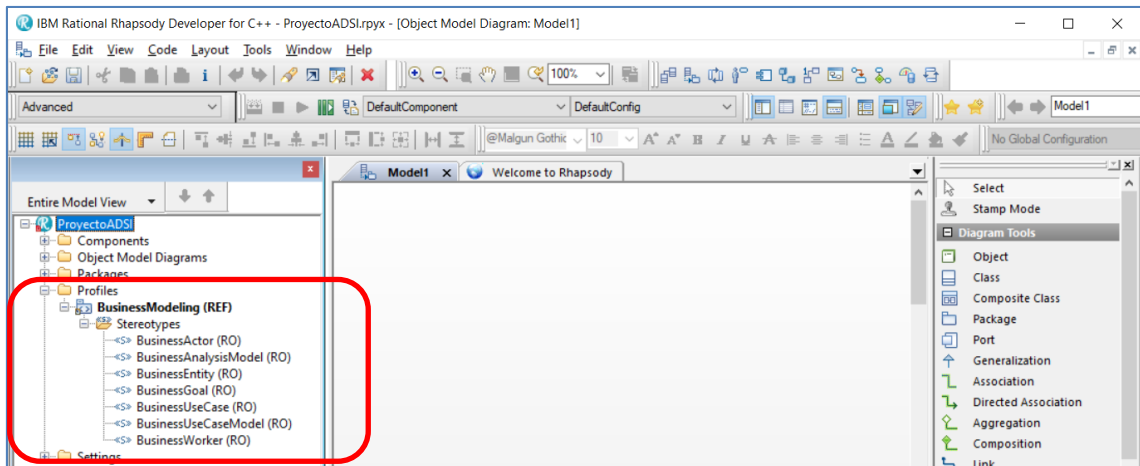


Figura 40: Proyecto con carpeta Profiles

Fuente. - Elaboración propia

Paso 6: Ahora, recobre el diagrama “Model1” por “Diagrama de Estructura del Proyecto”.

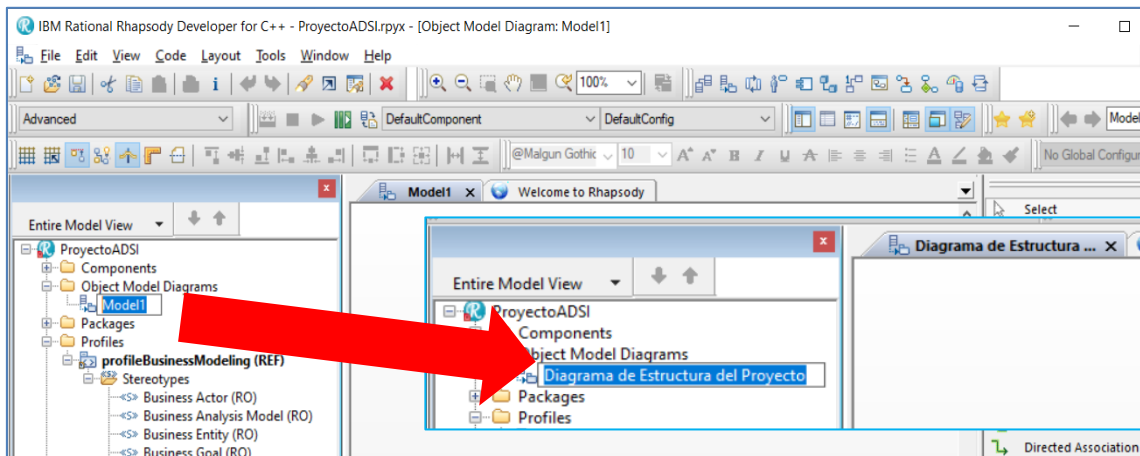


Figura 41: Diagrama de Estructura del Proyecto

Fuente. - Elaboración propia

Paso 7: Sobre el Diagrama de Estructura del Proyecto, cree 3 paquetes: MCUN (Modelo de Casos de Uso de Negocio), MAN (Modelo de Análisis de Negocio) y MCU (Modelo de Casos de Uso).

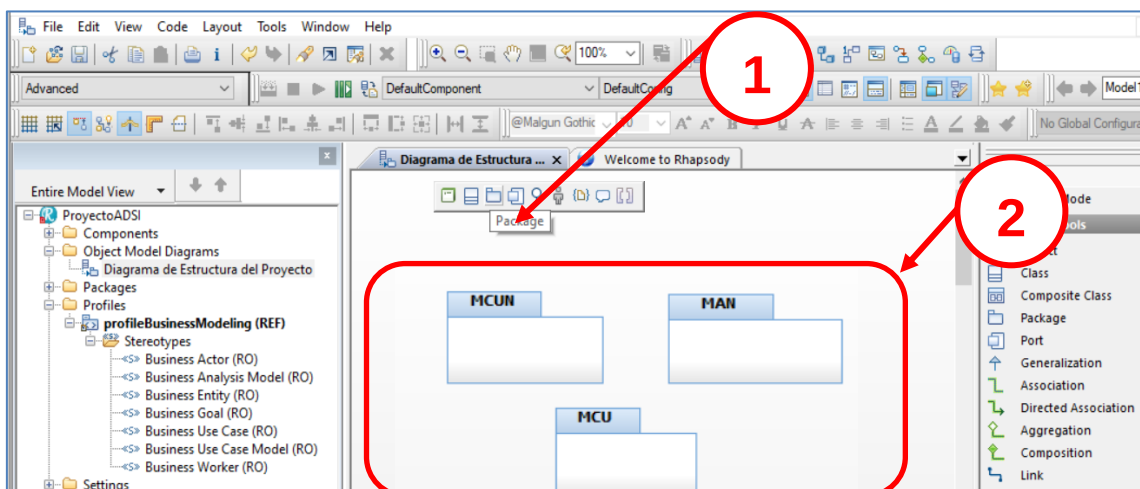


Figura 42: Diagrama de Estructura del Proyecto con los paquetes MCUN, MAN y MCU

Fuente. - Elaboración propia

Paso 8: A continuación, asigne el estereotipo de “Business Use Case Model” al paquete MCUN.

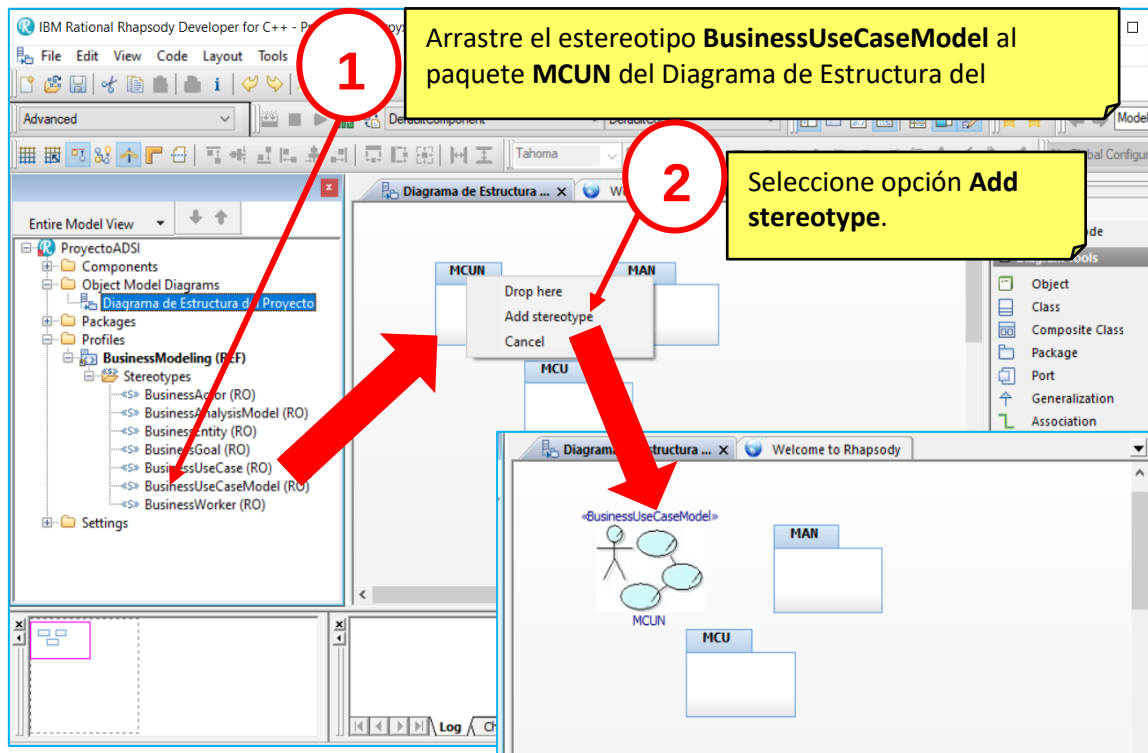


Figura 43: Aplicación del estereotipo **BusinessUseCaseModel** al paquete MCUN

Fuente. - Elaboración propia

Paso 9: Finalmente, asigne el estereotipo de “Business Analysis Model” al paquete MAN.

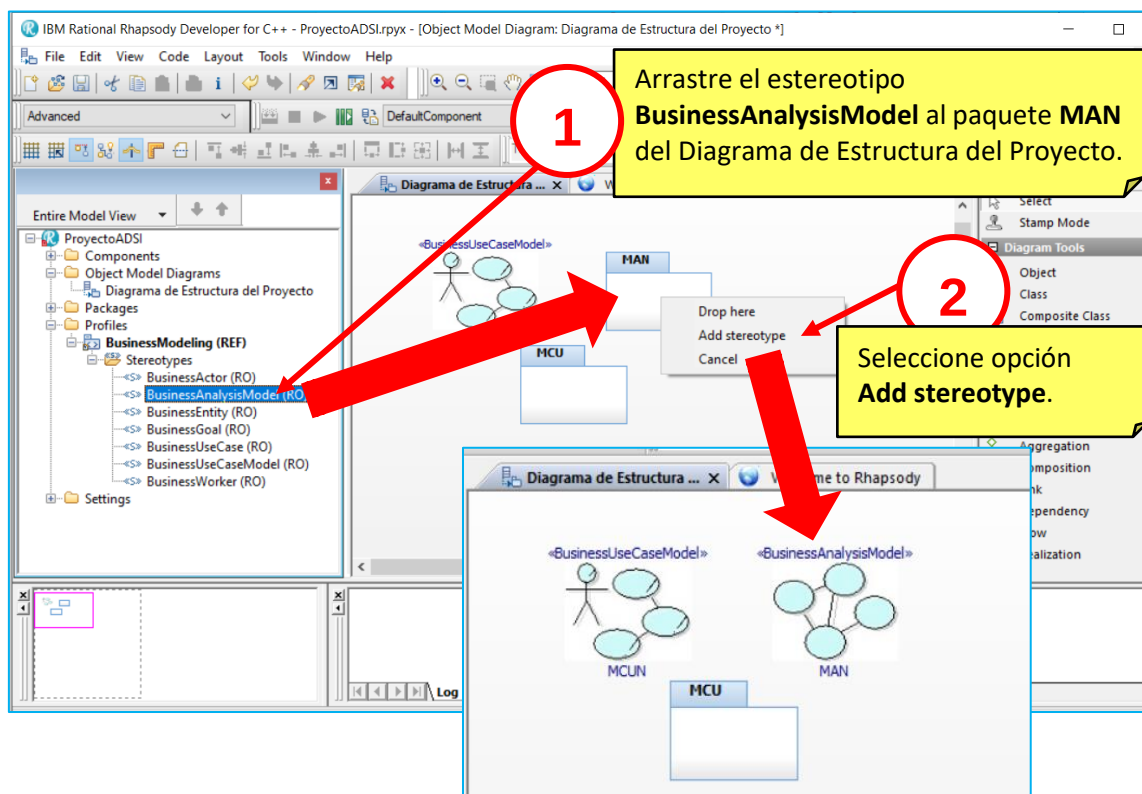


Figura 44: Aplicación del estereotipo **BusinessAnalysisModel** al paquete MAN

Fuente. - Elaboración propia

Resumen

1. La herramienta CASE de IBM conocida como IBM Engineering Systems Design Rhapsody presenta tres características importantes en comparación con IBM Rational Software Architect:
 - a. Permite crear matrices de trazabilidad entre artefactos de un modelo a otro.
 - b. Permite importar de una forma fácil elementos de un archivo CSV (actividades a sistematizar, requisitos y casos de uso) para generar matrices de trazabilidad en un proyecto.
 - c. Permite diseñar prototipos (GUI de casos de uso).
2. Rational Rhapsody es un entorno CASE que ofrece diversos idiomas para su instalación. Sin embargo, se sugiere que se mantenga la instalación en inglés a fin de estar familiarizado con la terminología que comúnmente se utilizan en ese idioma en el área de la Ingeniería de Software.
3. En Rational Rhapsody existe una opción para crear modelos sobre la base de un perfil que ofrece un toolkit para Ingeniería de Sistemas, el cual consiste en un set de modelos para un proyecto completo. Este toolkit se encuentra en el profile o perfil "Harmony SE".

Recursos

Pueden revisar los siguientes enlaces para ampliar los conceptos vistos en esta unidad:

- https://www.ibm.com/support/knowledgecenter/SSB2MU_8.4.0/com.ibm.rhp.uml.diagrams.doc/topics/rhp_t_dm_creating_rhap_projects.html
- https://www.ibm.com/support/knowledgecenter/SSB2MU_8.4.0/com.ibm.rhp.harmonyse.doc/topics/rhp_c_modeling_with_harmony_se_profile.html

2.3. MODELO DE CASOS DE USO DEL NEGOCIO – MCUN

2.3.1. Artefactos del modelo de casos de uso de negocio

En la siguiente tabla se muestra la lista de artefactos del Modelo de Casos de Uso de Negocio con una breve descripción. Estos artefactos serán los que se desarrollarán en el curso.

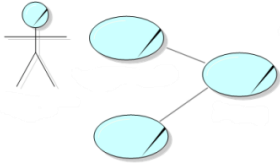
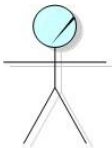
Artefacto	Descripción
 Modelo de Casos de Uso de Negocio	Es un paquete estereotipado como Business Use Case Model. Representa la vista externa del negocio. Es un modelo que describe la dirección e intención del negocio. La dirección es provista por los objetivos del negocio. Mientras que la intención es expresada por los diagramas que permiten ver cómo interactuar con el entorno.
 Objetivo de Negocio	Es una clase estereotipada como Business Goal. Es un requisito que debe ser satisfecho por el negocio. Describe el valor deseado de una medida en particular a futuro, y se utiliza para planear y administrar las actividades del negocio. El objetivo debe ser claro, medible, alcanzable, realista y sensible al tiempo. Se permite la relación de dependencia entre objetivos del negocio y la de soporte de un caso de uso del negocio.
 Caso de Uso de Negocio	Es un caso de uso estereotipado como Business Use Case Model. Define un conjunto de acciones que el negocio lleva a cabo y provee resultados de valor a quienes interactúan con él. Describe un proceso de negocio desde un punto de vista externo que percibe algún tipo de valor. Definen los límites de la organización. Categorías de CUN: Básicos o fundamentales (son los del core del giro de negocio), Estratégicas (son los CUN que coordinan la cadena de valor), y los de Apoyo o Soporte (son los que apoyan a la cadena de valor).
 Actor de Negocio	Es un actor estereotipado como Business Actor. Representa un rol que algo o alguien externo desempeña en relación con el negocio. Puede ser asociado a uno o más casos de uso del negocio.
 Situación de Negocio	Documento que incluye: • Los Objetivos del negocio y Alcance del proyecto. • Situación actual de la Organización. • Situación propuesta del Negocio que presenta los Casos de Uso de Negocio con Oportunidades de Automatización. • Reglas del negocio: Es una política o condición que debe ser satisfecha por el negocio. Ejemplo: "El pago de planillas se realizará los días 25 de cada mes y vía depósito en cuenta bancaria." • Glosario de términos del proyecto.

Figura 45: Tabla de Artefactos del Modelo de Casos de Uso de Negocio considerados en el curso ADSI

Fuente. - Elaboración propia

2.3.2. Reglas básicas de nomenclaturas

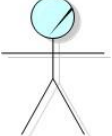
Artefacto	Reglas de nomenclatura
 Objetivo de Negocio	Su enunciado debe especificar claramente qué se va a lograr cuando el sistema se implemente en un proceso, para lo cual se debe indicar qué se va a medir, el indicador de medida, y con respecto de qué periodo de tiempo. Ejemplos: - Incrementar en un 80% el nivel de satisfacción de clientes a la categoría "A" con respecto del primer semestre del año 2020. Evaluemos el primer objetivo: ¿Qué se va a medir? El incremento del nivel de satisfacción de clientes a la categoría "A". ¿Cuál es el indicador de medida? en un 80%. ¿Con respecto de qué periodo? Primer semestre del año. Entonces, el enunciado es válido. - Disminuir el tiempo de atención de reserva de habitación en un 60% con respecto del segundo trimestre del año 2020. Evaluemos el segundo objetivo: ¿Qué se va a medir? La disminución del tiempo de atención de reserva de habitación. ¿Cuál es el indicador de medida? en un 60%. ¿Con respecto de qué periodo? segundo trimestre del año. Entonces, el enunciado es válido.
 Caso de Uso de Negocio	Su nombre empieza con un sustantivo que enuncie un conjunto de actividades, el cual debe ser enfocado a lo que la empresa realiza. Por ejemplo: - Contratación de personal - Selección de personal - Reserva de hospedaje - Afiliación de socios - Gestión de reserva de sala de reuniones Si el proceso que está analizando es muy complejo puede representarse con más de un caso de uso de negocio. Es iniciado por un solo actor, el cual es conocido como actor iniciador. Puede tener 0 o más participantes, los cuales son considerados como actores participantes.
 Actor de Negocio	El nombre debe ser un rol que cumple una persona según las actividades que realiza al iniciar o participar como agente externo de un proceso y que está esperando algo de valor de éste. Por ejemplo: - Cliente (inicia el CUN "Venta de productos") - Proveedor (participa en un CUN "Abastecimiento de insumos") - SolicitanteRequerimientoPersonal (inicia el CUN "Atención de Requerimientos de Personal") No es correcto colocar el nombre de una persona o de un área o departamento de la empresa. Por ejemplo: - Gina Ramos X - DepartamentoFinanzas X - Ventas X

Figura 46: Reglas de nomenclaturas del Modelo de Casos de Uso de Negocio.

Fuente. - Elaboración propia

2.3.3. Actividades para identificar los artefactos de la vista externa del negocio

En este apartado cubriremos el desarrollo de actividades relevantes que permiten obtener los artefactos principales del Modelo de Casos de Uso de Negocio, las cuales tienen como objetivo entender el negocio en estudio para delimitarlo y explorar el dominio del problema:

- Identificar objetivos del negocio y visión del proyecto para definir alcance del proyecto, los cuales, en nuestro caso serán descritos en el documento “Situación del Negocio”.
- Identificar los procesos a fin de tener claro las fronteras del negocio. Aquí se identifican los procesos que serán representados como casos de uso de negocio (CUN), y los actores de negocio involucrados. Tener en cuenta que, si un proceso es demasiado complejo, puede representarse con más de un CUN. Asimismo, es necesario priorizar los procesos que se enfocarán en el proyecto. Finalmente, con una herramienta CASE se crea un Diagrama General de Casos de Uso de Negocio para mostrar la interacción entre actores de negocio y casos de uso de negocio.
- Identificar las reglas del negocio y plasmarlo en Especificaciones de Reglas del Negocio. En nuestro caso lo incluiremos en el documento “Situación de Negocio”.
- Capturar un vocabulario común a través de un Glosario de términos del proyecto que lo incluiremos en “Situación de Negocio”.

Recuerde que AUP es una metodología iterativa e incremental. De modo que, las actividades no necesariamente tienen que realizarse de forma secuencial. En este sentido, el equipo puede optar por desarrollarlas en paralelo, identificando algunos elementos, hasta obtener una versión más refinada en cada iteración.

2.3.4. Identificación de artefactos del modelo de casos de uso de negocio para el caso fashion importaciones

CASO: FASHION IMPORTACIONES

Usted ha sido contratado para trabajar en el proyecto: Sistema de Venta de productos importados para mujeres emprendedoras. Esta compañía tiene como principal propósito captar y entrenar a emprendedoras para la venta de carteras, joyas y accesorios, para lo cual diseñan catálogos de ventas de forma digital para facilitar la venta a través de ellas. Su misión es ayudar a la mujer peruana a poder generar ingresos extras y alcanzar la independencia económica, apoyándolas y entrenándolas en las herramientas digitales, haciendo de su pasión por las carteras, joyas y accesorios su mejor negocio. Actualmente en el Perú, Fashion Importaciones tiene 3 sucursales con almacenes: Lima, Cajamarca y Chiclayo.

Con el fin de agilizar sus ventas y atención de pedidos, así como incrementar el número de emprendedoras, el Gerente de la empresa requiere un sistema que permita a sus emprendedoras registrar, pagar y hacer seguimiento de sus pedidos de los productos correspondientes a un catálogo vigente. Asimismo, requieren definir una ruta óptima de entregas de estos pedidos a fin de minimizar los costos de combustible de sus móviles. A continuación, se describen los procesos.

El proceso de afiliación de una emprendedora inicia cuando una postulante se contacta con la empresa para acceder a la capacitación. Una Recepcionista atiende a la postulante y le solicita su nombre completo, DNI, dirección, números de contacto y correo electrónico. A continuación, la recepcionista le brinda las fechas de capacitación presencial o virtual. Si la postulante está de

acuerdo con alguna fecha, el Asesor registra su participación en la fecha indicada. Al término de la capacitación, la postulante es evaluada sobre los temas que ha sido entrenada. En caso de aprobar la evaluación, la postulante recibe una constancia de la capacitación y puede acceder al catálogo digital. Asimismo, está lista para realizar sus pedidos.

El proceso de venta de productos inicia cuando una emprendedora se contacta con la empresa para realizar su pedido el cual tiene asignado los datos de la emprendedora y de los productos solicitados. Un Asesor de Ventas toma el pedido y lo registra como “GENERADO” y confirma el monto total para que la emprendedora realice el pago a una cuenta bancaria. Una vez, realizado el pago del pedido, la emprendedora envía la constancia de pago al Asesor de ventas, quien cambia de estado del pedido a “PAGADO”. A continuación, genera el Comprobante de Pago (CDP) y lo envía a la emprendedora vía correo electrónico.

La atención de pedidos inicia cuando la Supervisora de ventas entrega la lista de pedidos de las emprendedoras al área de despacho. El asistente de despacho elabora un cronograma de salidas de la semana junto con la ruta a seguir para entregar los pedidos. El asistente de despacho entrega al transportista el cronograma de salidas, la hoja de ruta y una guía de remisión de los pedidos; en este momento los pedidos están “EN RUTA”. Una vez que el transportista entrega los productos a la emprendedora, le solicita que firme la copia de la guía de remisión. Finalmente, el encargado de despacho recibe las copias de las guías de remisión firmadas confirmando que el pedido ha sido “ENTREGADO” y genera un informe de despacho, el cual es enviado a la Gerente de Ventas.

Realice las siguientes actividades para crear el Modelo de Casos de Uso de Negocio (MCUN):

- Elabore la estructura del Proyecto en una Herramienta CASE.
- Elabore el Diagrama General de Casos de Uso de Negocio.
- Identifique los Casos de Uso de Negocio
- Identifique los Actores de Negocio
- Identifique los Objetivos de Negocio

Solución del caso en Rational Rhapsody

Debe seguir los pasos descritos en la sección “2.2.4. Creación de un proyecto con la estructura de Modelado” para crear el proyecto “ProyectoFashionImportaciones”:

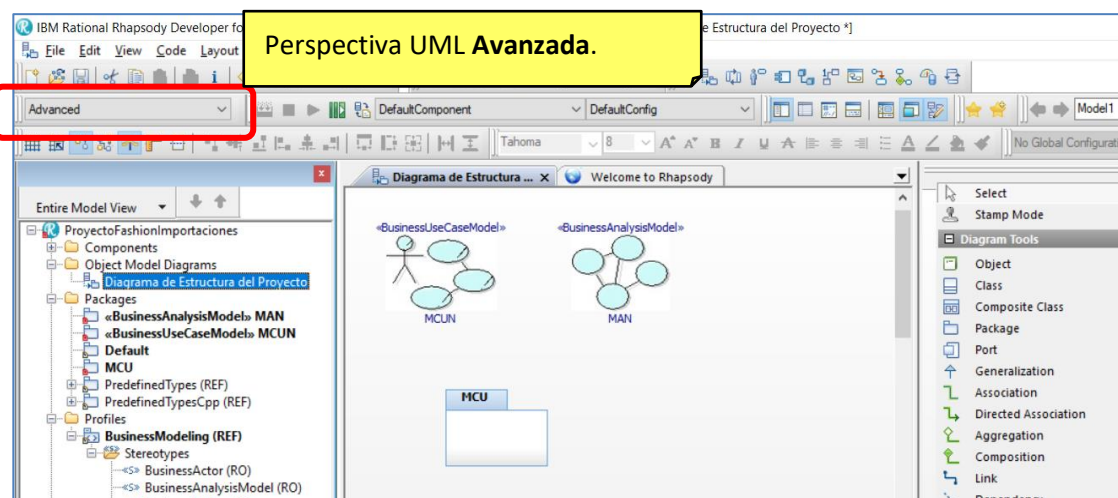


Figura 47: Estructura de Proyecto del caso “Fashion Importaciones”.

Fuente. - Elaboración propia

Como se muestra en la figura anterior, el entorno debe tener activada la perspectiva UML avanzada a fin de contar con la sección completa de Diagramas en la barra de herramientas. Asimismo, note que la barra de herramientas con los componentes a dibujar se encuentra en el panel derecho del diagrama.

A continuación, crearemos los artefactos para el MCUN:

Paso 1: Cree el Diagrama General de Casos de Uso de Negocio dentro del paquete MCUN. Para ello, seleccione MCUN desde el explorador de proyecto; luego, de clic sobre el **Diagrama de Caso de Uso** ubicado en la sección de diagramas de la barra de herramientas.

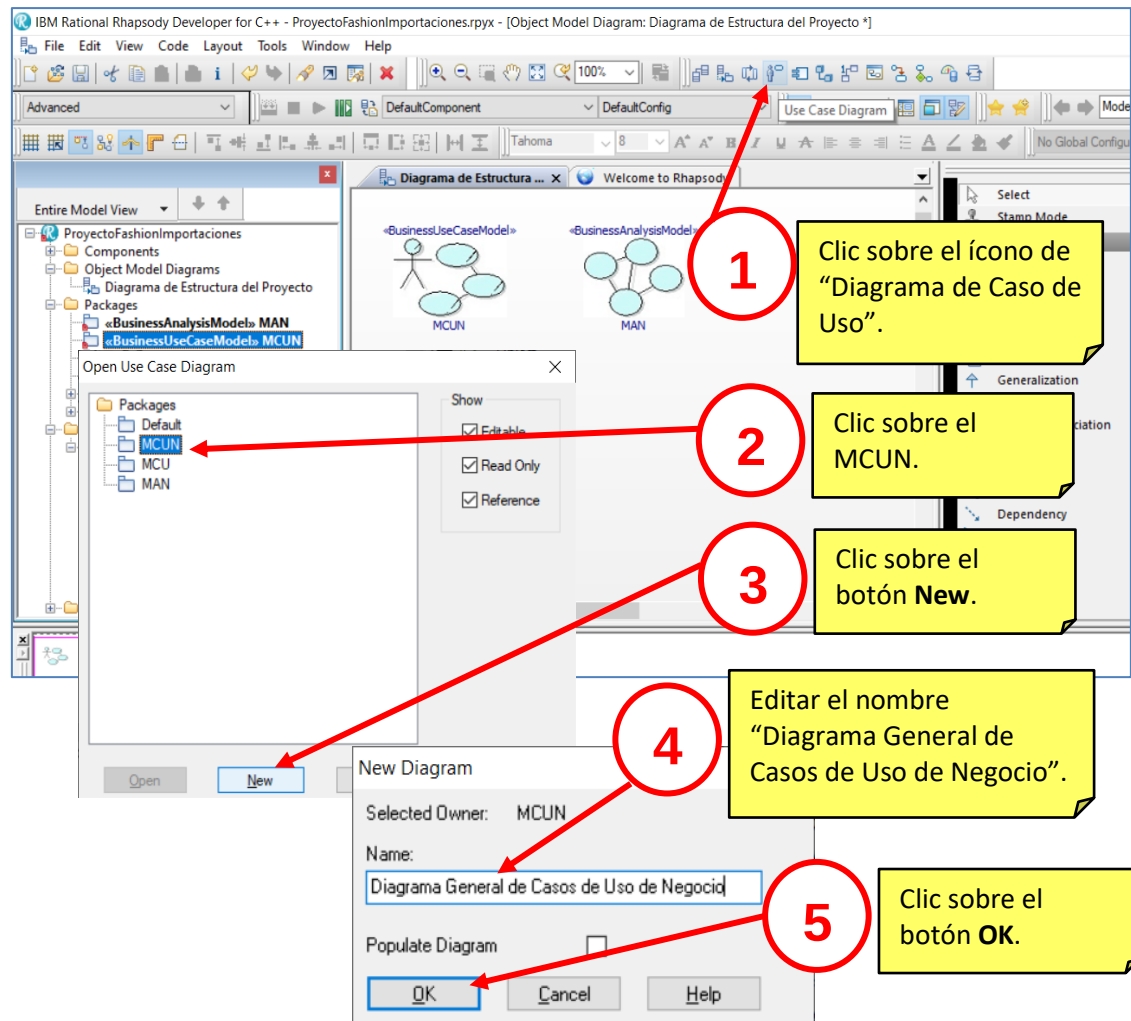


Figura 48: Diagrama General de Casos de Uso de Negocio del caso "Fashion Importaciones".

Fuente. - Elaboración propia

Paso 2: Ahora, dibuje los casos de uso de negocio del caso.

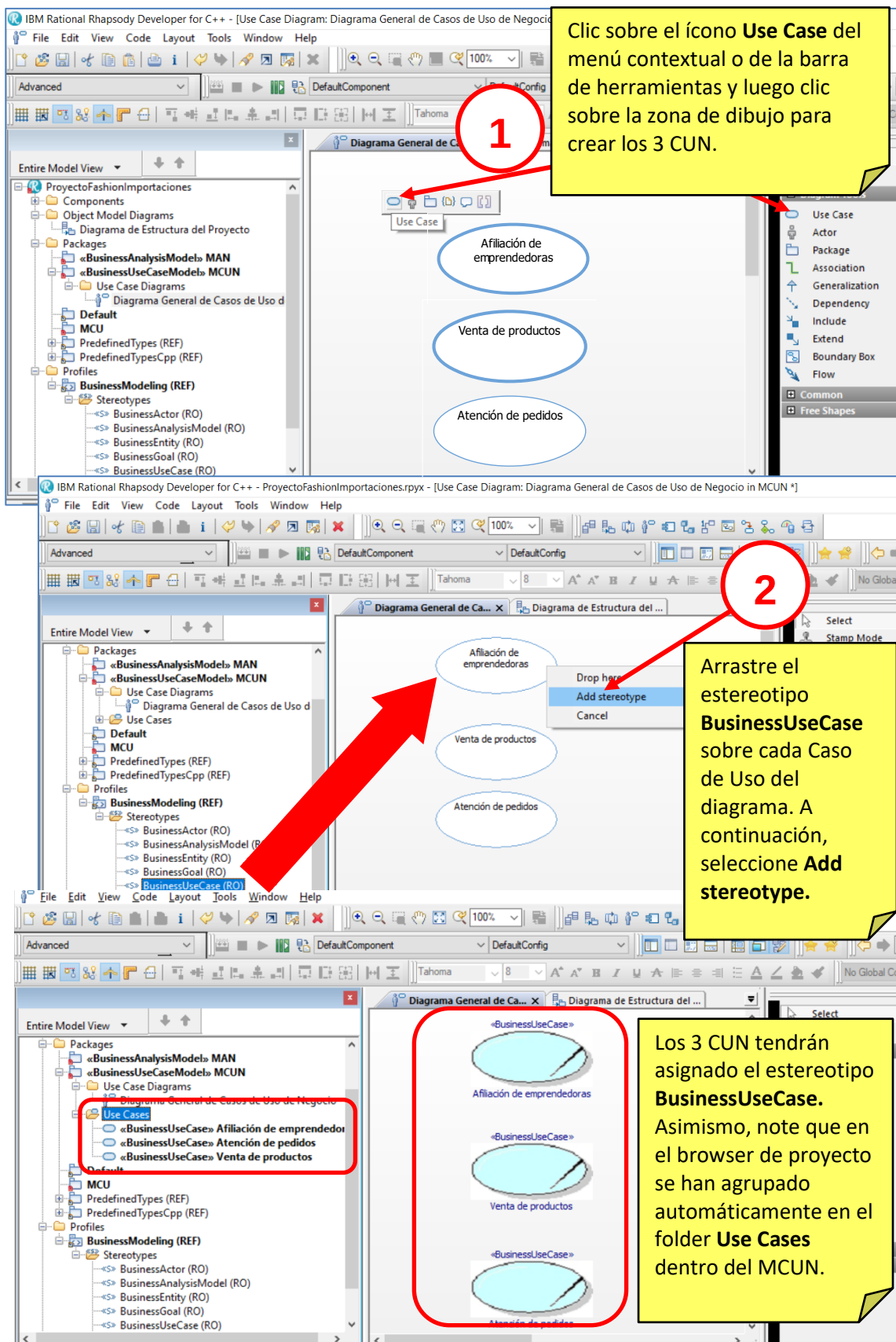


Figura 49: Casos de Uso de Negocio del caso "Fashion Importaciones".

Elaboración propia

Paso 3: Ahora, dibuje los actores de negocio del caso.

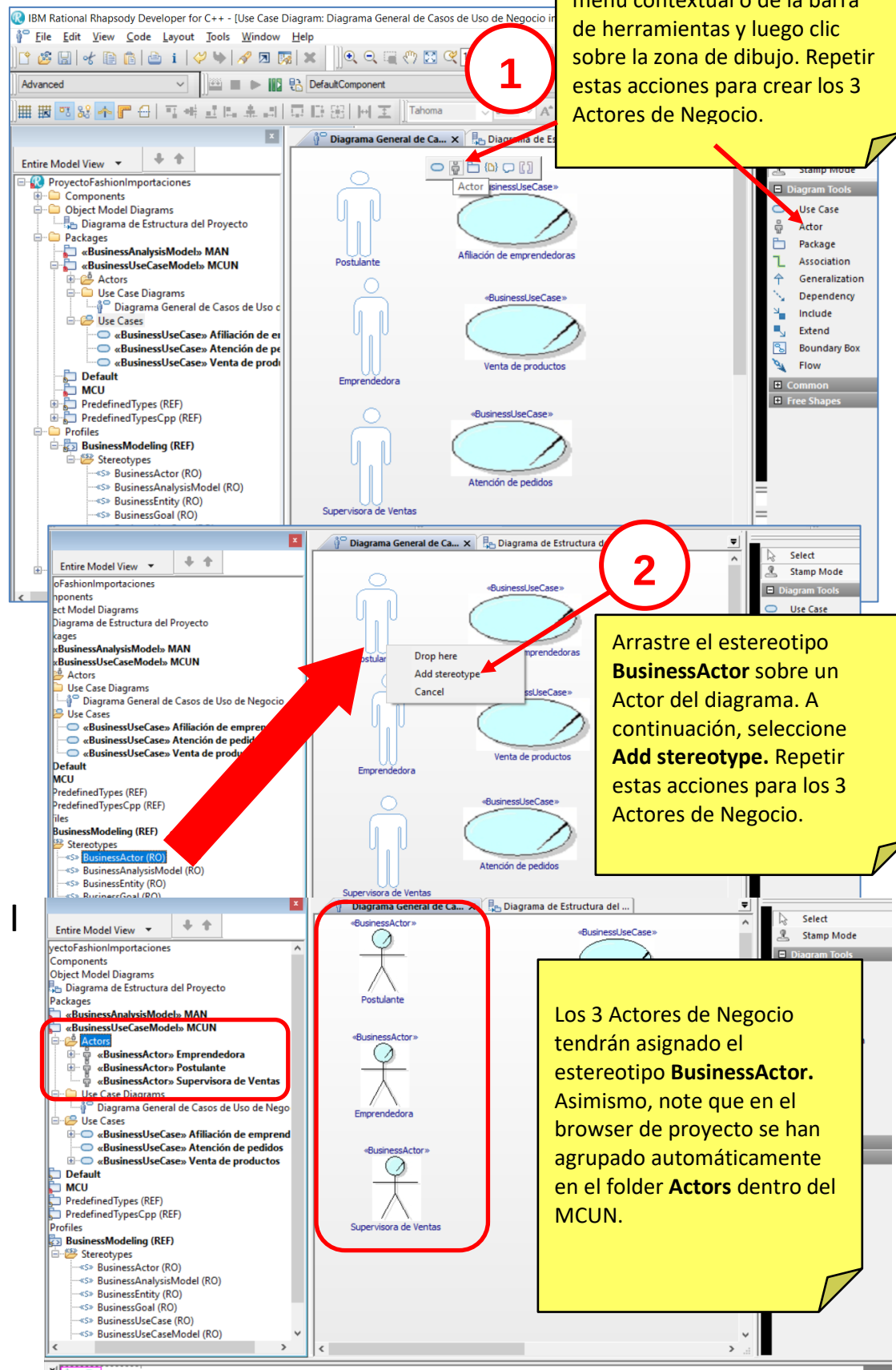


Figura 50: Actores de Negocio del caso "Fashion Importaciones".

Fuente. - Elaboración propia

Paso 4: Puede desactivar la opción “Show Stereotype” para no visualizar el nombre del estereotipo de los componentes creados.

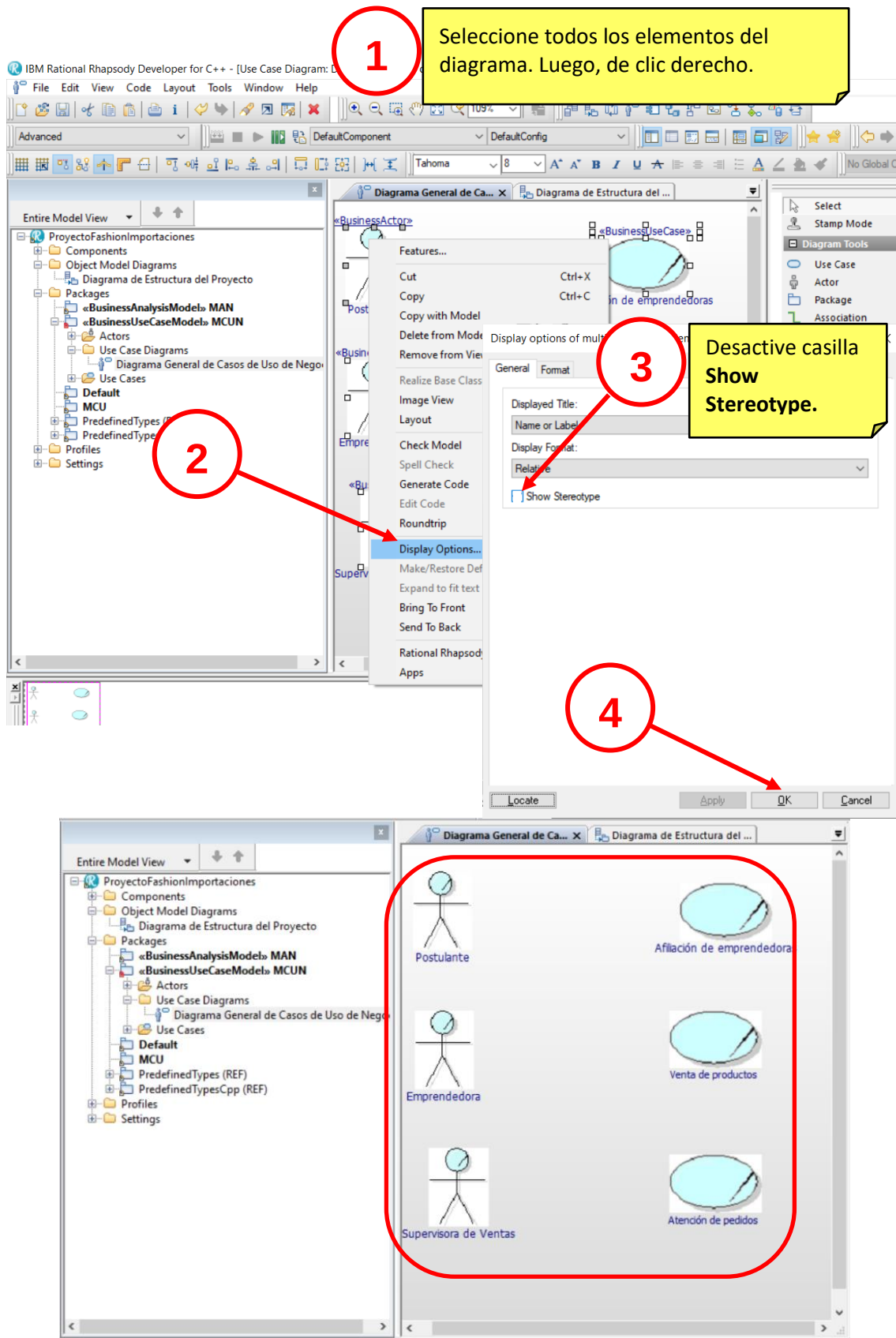


Figura 51: Artefactos con nombre de estereotipo desactivado

Fuente: - Elaboración propia

Paso 5: A continuación, agregue las asociaciones dirigidas entre el Actor de Negocio y el CUN. Antes de crear las asociaciones en el Diagrama de Casos de Uso, debe configurar ciertas propiedades desde el menú **Window**.

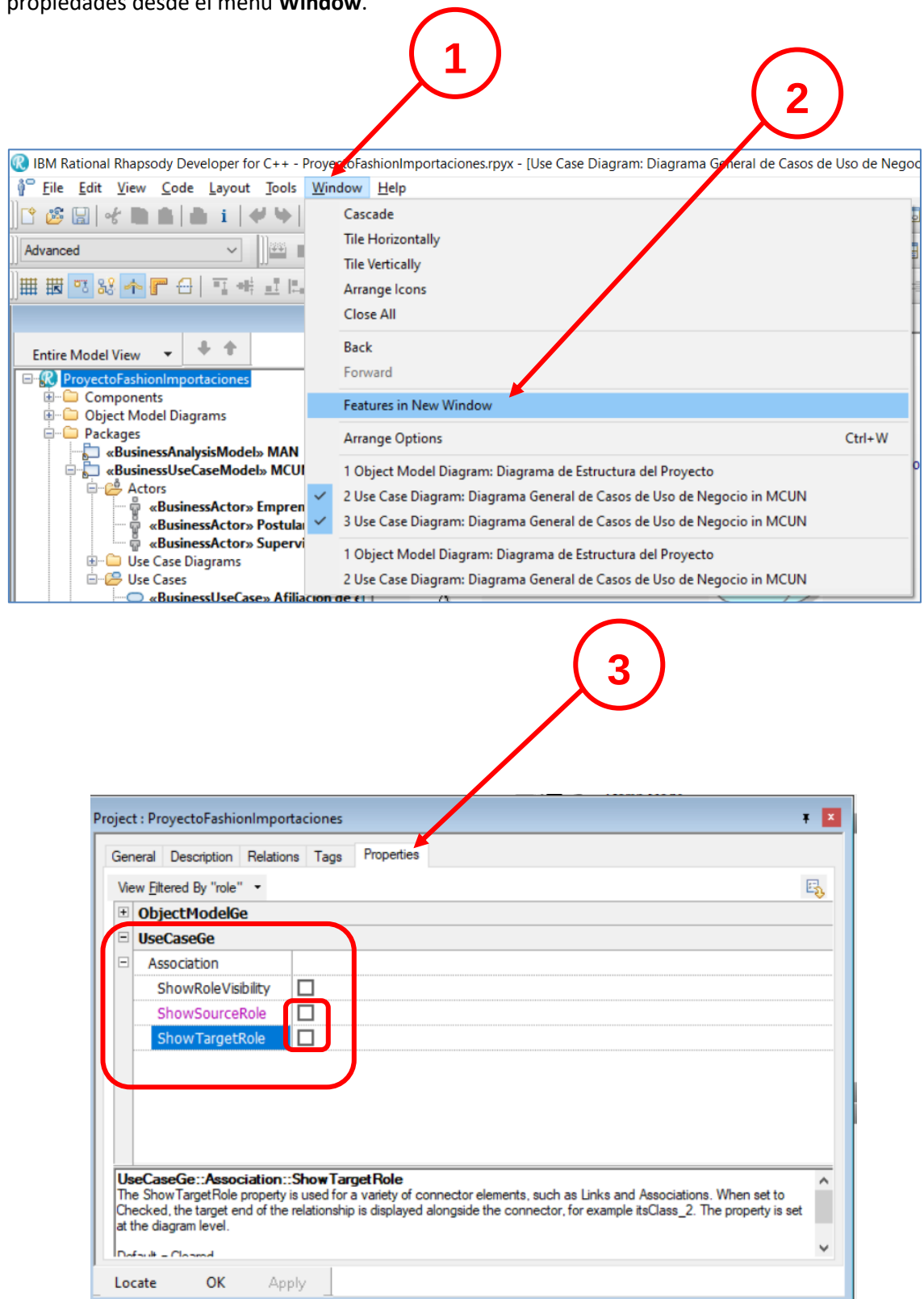


Figura 52: Configuración de Asociaciones dirigidas en Diagramas de Casos de Uso

Fuente. - Elaboración propia

Paso 6: Ahora, agregue las asociaciones dirigidas configuradas en el paso anterior. Estas relaciones deben ser entre el Actor de Negocio y el Caso de Uso del Negocio; no entre actores ni tampoco entre casos de uso.

De clic en el siguiente orden:
(1) Actor, (2) Association, y
(3) Caso de Uso.

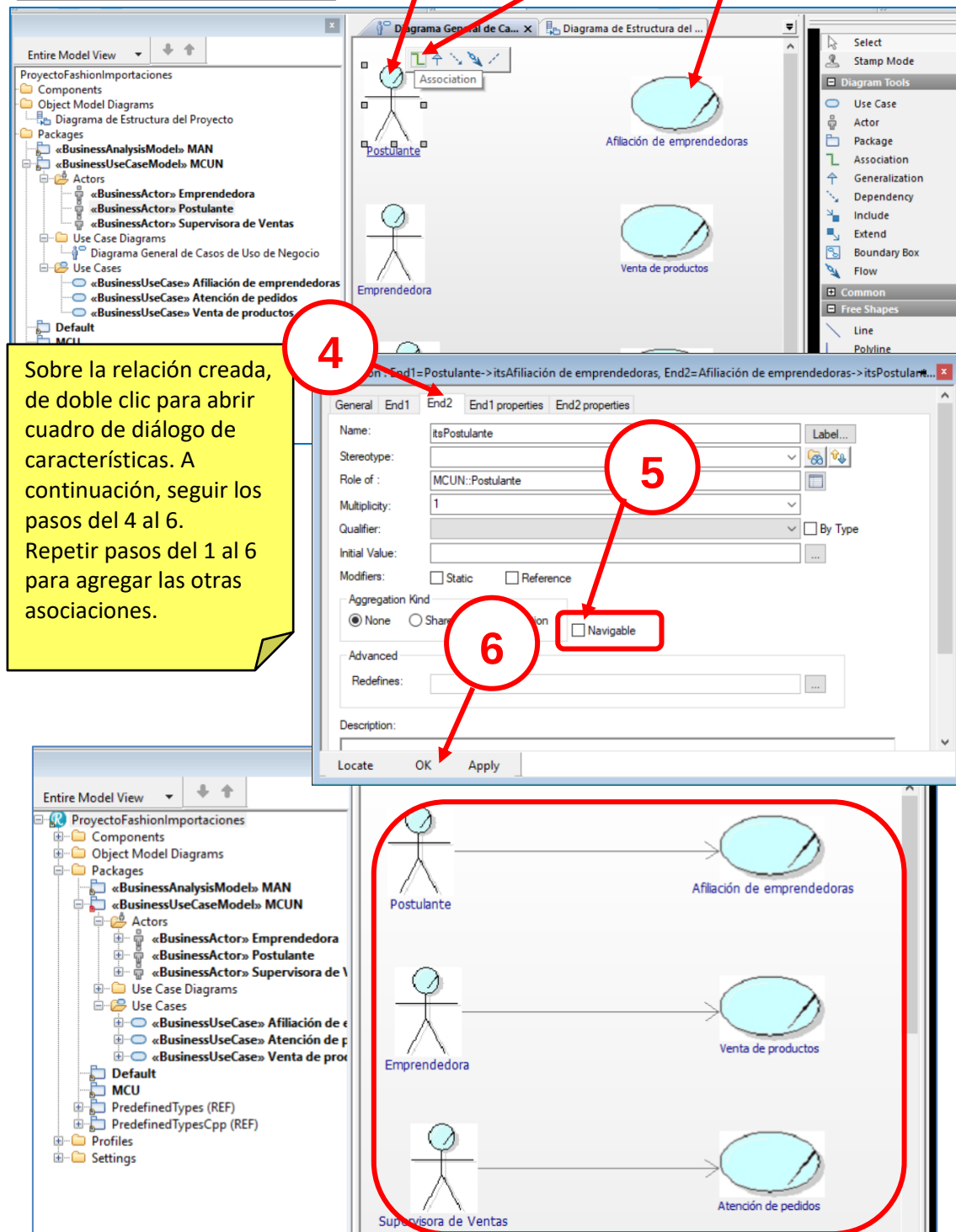


Figura 53: Asociaciones dirigidas entre actores de negocio y casos de uso de negocio

Fuente. - Elaboración propia

Paso 7: Ahora, identifique los objetivos del negocio. Antes de crear los objetivos, debe crear un Diagrama de Objetivo de Negocio Vs. Casos de Uso de Negocio. Para ello, utilice el Diagrama **Object Model Diagram**, el cual es como un diagrama de formato libre debido a que permite agregar diversos tipos de componentes.

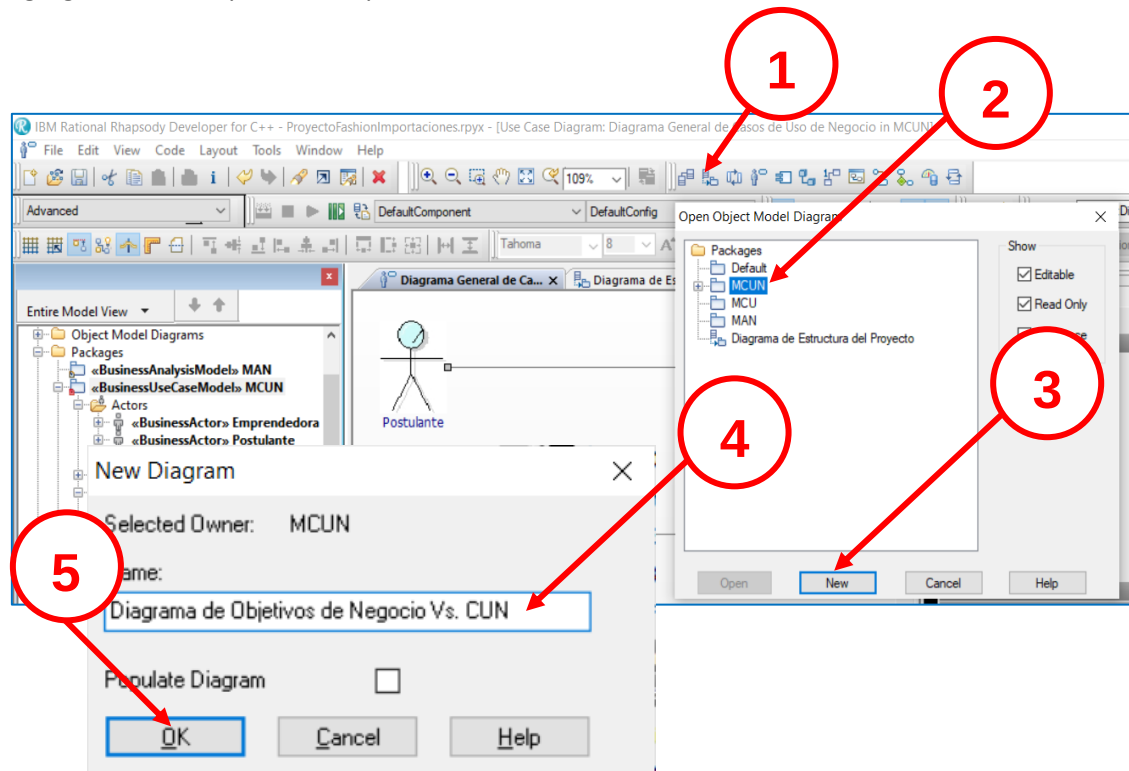


Figura 54: Creación del Diagrama de Objetivos de Negocio Vs. Casos de Uso de Negocio
Fuente. - Elaboración propia

Paso 8: A continuación, se agregará los objetivos de negocio, los cuales son clases estereotipadas. De modo que, agregaremos clases como objetivos. Luego, le asignamos el estereotipo **BusinessGoal**.

Del tercer párrafo del caso, se pueden identificar los objetivos que Fashion Importaciones desea alcanzar con el sistema a implementar. Sin embargo, es importante completar dichos enunciados siguiendo las características SMART (Específico, Medible, Alcanzable, Realista, y Tiempo definido). Suponemos los siguientes objetivos, luego de un análisis junto con el cliente que nos ha contratado:

ON1: Incrementar el número de emprendedoras en un 80% con respecto del último semestre del año 2020.

ON2: Disminuir en un 50% el tiempo de registro de las ventas de productos con respecto del año 2020.

ON3: Disminuir el tiempo de atención de pedidos en un 50% con respecto del año 2020.

ON4: Minimizar el costo de combustible en un 40% con respecto del último mes del año 2020.

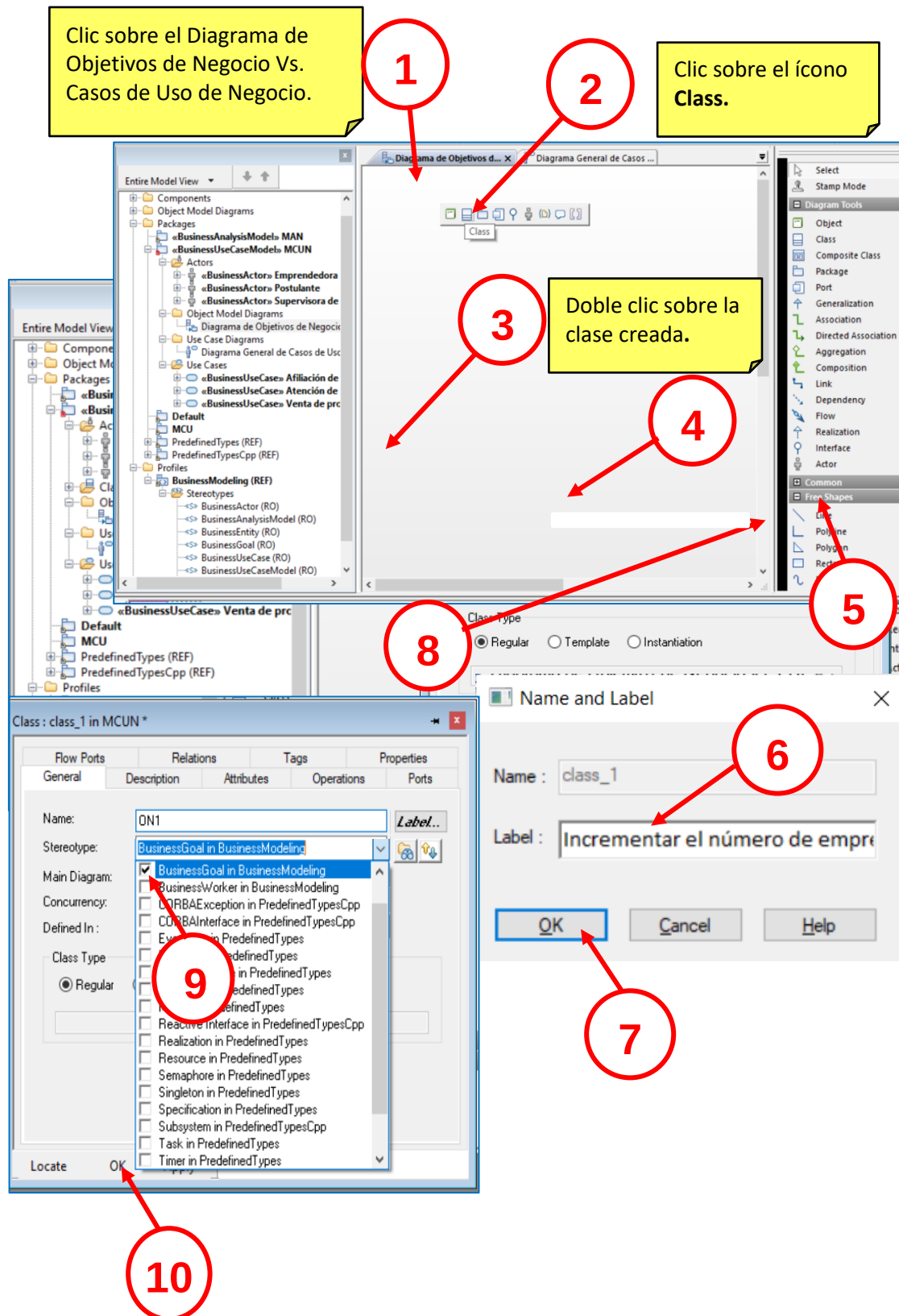


Figura 55: Creación del Objetivo de Negocio
Fuente. - Elaboración propia

Paso 9: Ahora, cambie el **Display Options** del Objetivo de Negocio a fin de visualizar su enunciado. El enunciado del objetivo fue editado como Label en el paso anterior.

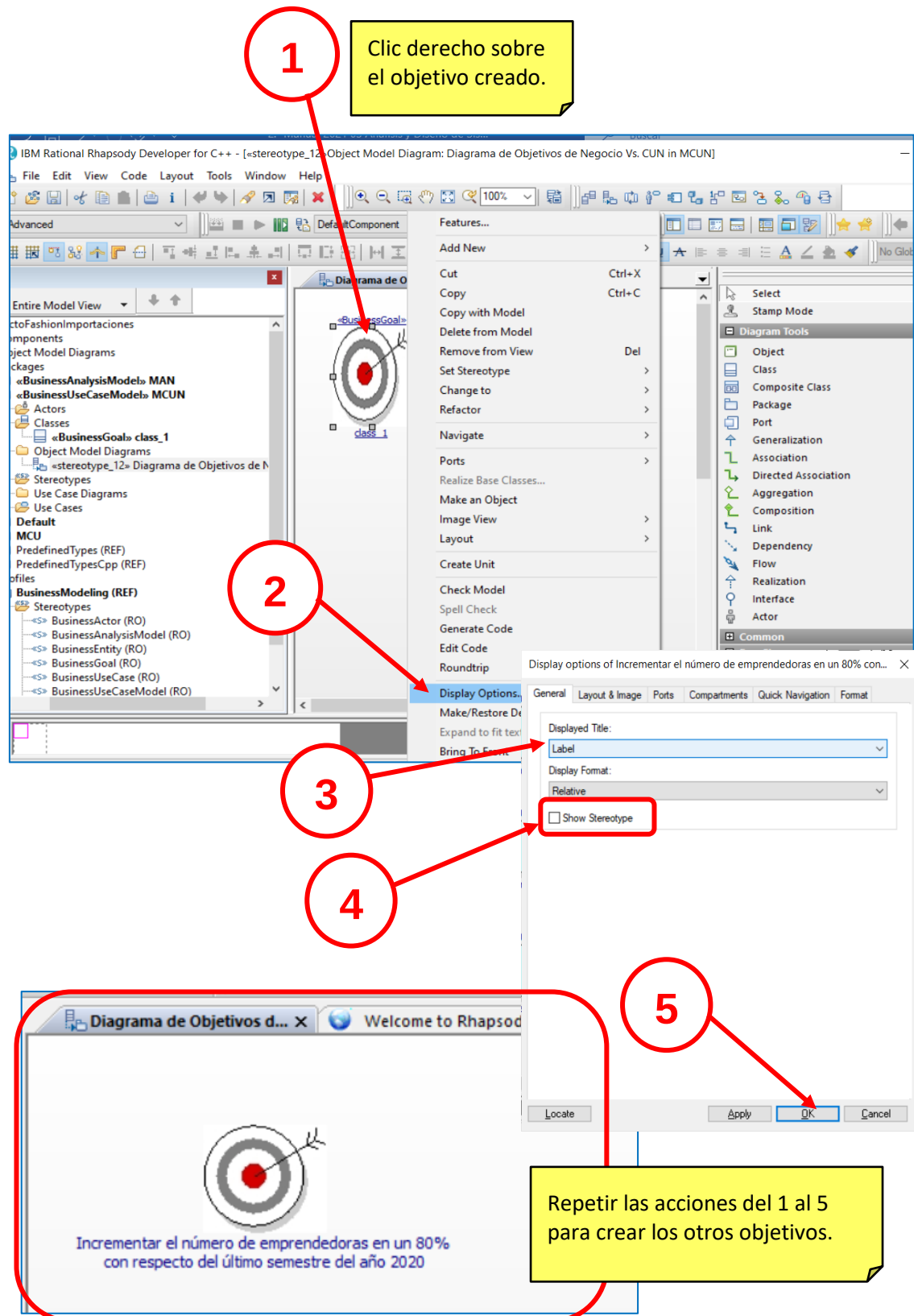


Figura 56: Cambio de opciones para mostrar el enunciado del Objetivo de Negocio
Fuente. - Elaboración propia

Paso 10: Finalmente, complete el Diagrama de Objetivos de Negocio Vs. Casos de Uso de Negocio. Para ello, arrastre el folder **Use Cases**, el cual contiene los 3 Casos de Uso de Negocio, del browser del proyecto al Diagrama. Luego, agregue las relaciones de dependencia desde el CUN al Objetivo de Negocio.

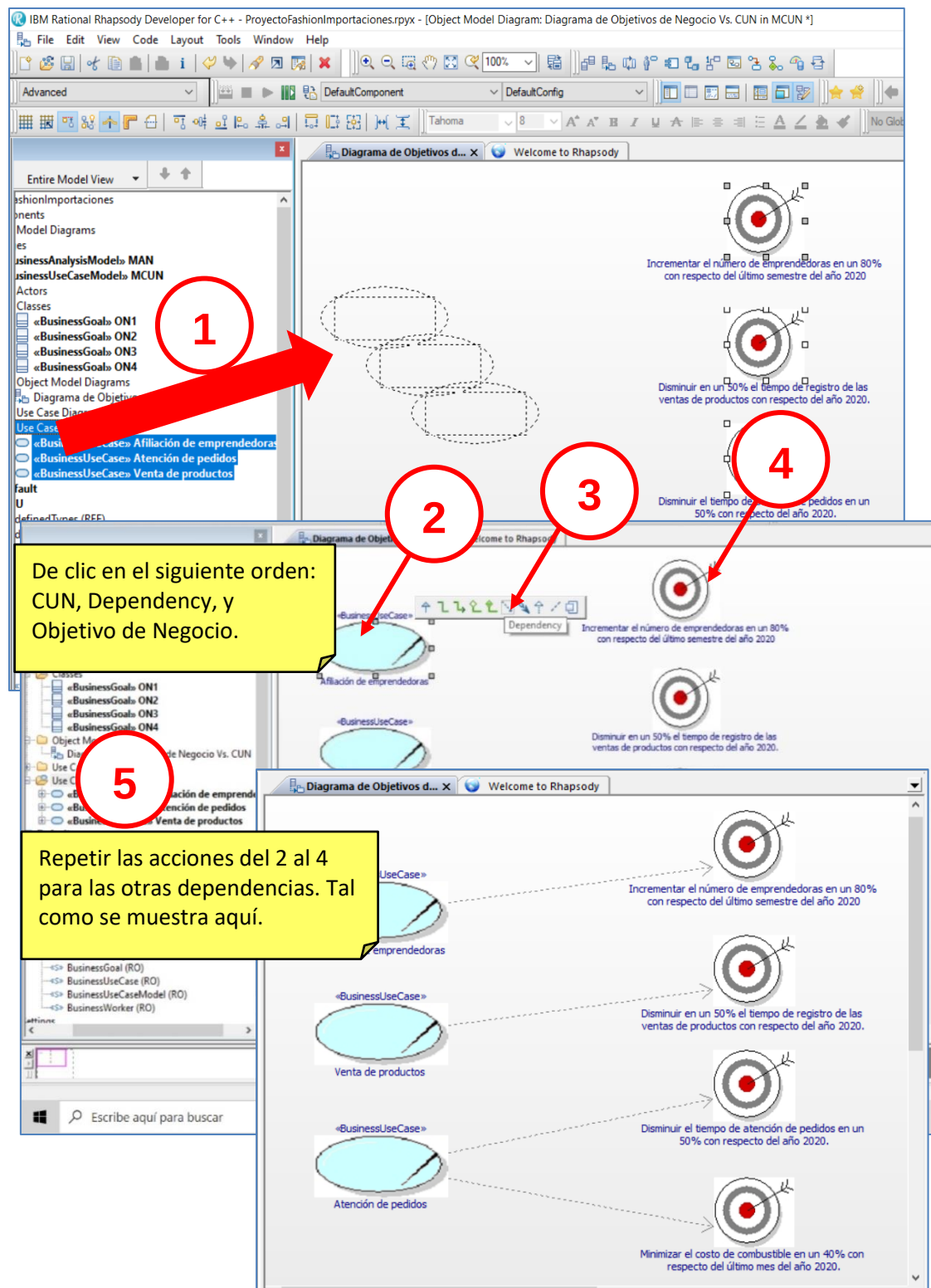


Figura 57: Diagrama del Objetivos de Negocio Vs. CUN de Fashion Importaciones

Fuente. - Elaboración propia

Resumen

1. El Modelo de Casos de Uso de Negocio representa la vista externa del negocio.
2. El Modelo de Casos de Uso de Negocio (MCUN) es un modelo que describe la dirección e intención del negocio. La dirección es provista por los objetivos del negocio. Mientras que la intención es expresada por los diagramas que permiten ver cómo interactuar con el entorno, tal es el caso del Diagrama General de Casos de Uso de Negocio.
3. EL MCUN presenta los siguientes artefactos relevantes: Objetivos de Negocio, Casos de Uso de Negocio, y Actores de Negocio. Asimismo, en el curso se incluye el documento Situación de Negocio.
4. El enunciado de un Objetivo de Negocio debe ser SMART (Específico, Medible, Alcanzable, Realista, y con un Tiempo definido). En este sentido, debe especificar claramente qué se va a lograr cuando el sistema se implemente en el proceso, para lo cual se debe indicar qué se va a medir, el indicador de medida, y con respecto de qué periodo de tiempo.
5. El nombre de un Caso de Uso de Negocio empieza con un sustantivo que enuncie un conjunto de actividades, el cual debe ser enfocado a lo que la empresa realiza.
6. El nombre de un actor debe ser un rol externo al negocio, que inicia o participa en un proceso. Asimismo, espera un resultado de valor del proceso.

Recursos

Pueden revisar los siguientes enlaces para ampliar los conceptos vistos en esta unidad:

- http://www.michael-richardson.com/processes/rup_classic/#extend.bus_model/disciplines/rup_business_modeling_discipline_553E20F6.html
- https://www.ibm.com/support/knowledgecenter/SSB2MU_8.3.0/com.ibm.rhp.uml.diagrams.doc/topics/rhp_t_dm_creating_rhap_projects.html

2.4. MODELO DE ANÁLISIS DEL NEGOCIO – MAN

2.4.1. Artefactos del modelo de análisis de negocio

En la siguiente tabla se muestra la lista de artefactos del Modelo de Análisis de Negocio con una breve descripción. Estos artefactos serán los que se desarrollarán en el curso.

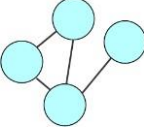
Artefacto	Descripción
 Modelo de Análisis de Negocio	Es un paquete estereotipado como Business Analysis Model. Representa la vista interna del negocio. Es un modelo que describe la realización de los casos de uso del negocio. Es una abstracción de cómo los trabajadores del negocio y las entidades de negocio se relacionan y de cómo colaboran para realizar los casos del uso del negocio.
 Entidades de Negocio	Es una clase estereotipada como Business Entity. Ente significativo y persistente manipulado por actores del negocio y trabajadores del negocio.
 Trabajadores de Negocio	Es una clase estereotipada como Business Worker. Un trabajador del negocio es un rol interno al negocio. Colabora con trabajadores de otro sector, es notificado de acontecimientos del negocio y manipula entidades de negocio para realizar sus responsabilidades.

Figura 58: Tabla de Artefactos del Modelo de Análisis de Negocio considerados en el curso ADSI

Fuente. - Elaboración propia

2.4.2. Reglas básicas de nomenclaturas

Artefacto	Reglas de nomenclatura
 Entidades de Negocio	El nombre debe ser un sustantivo que haga referencia a un documento, ficha o registro de datos, o reportes. Puede escribir siglas o acrónimos de la entidad, en caso sea un término utilizado en el negocio. Ejemplos: - CDP (Comprobante de Pago). - RegistroClientes - RegistroProductos - FichaCompetenciasGenerales
 Trabajadores de Negocio	El nombre debe ser un rol que cumple una persona según las responsabilidades asignadas para un proceso. Por ejemplo: - Vendedor - AsistenteCompras - EncargadoCompras

Figura 59: Reglas de nomenclaturas del Modelo de Casos de Uso de Negocio.

Fuente. - Elaboración propia

2.4.3. Actividades para identificar los artefactos de la vista interna del negocio

En este apartado cubriremos el desarrollo de actividades relevantes que permiten obtener los artefactos principales del Modelo de Análisis de Negocio. El objetivo es detallar los procesos de negocio (vista interna del negocio) a fin de lograr un mayor entendimiento del negocio.

A continuación, realizaremos las siguientes actividades:

- Refinar la identificación de las reglas del negocio y plasmarlo en Especificaciones de Reglas del Negocio. En nuestro caso lo incluiremos en el documento “Situación de Negocio”.
- Refinar la captura de un vocabulario común a través de un Glosario de términos del proyecto que lo incluiremos en “Situación de Negocio”.
- Detallar los procesos de negocio, para lo cual por cada proceso se construye dos diagramas: Diagrama de Clases (Diagrama de estructura) y Diagrama de Actividades (Diagrama de comportamiento). En el Diagrama de Clases se identifica a los Trabajadores de Negocio y Entidades de Negocio que son manipulados por los trabajadores. En el Diagrama de Actividades se representa el flujo de trabajo del proceso a fin de entender cómo se desarrolla actualmente dicho proceso. Se puede considerar crear 2 diagramas de actividades: uno que muestre el flujo actual del proceso y el segundo que muestre el flujo que se desea cuando el sistema se implemente. Esta actividad se desarrollará en una herramienta CASE.
- Identificar oportunidades de automatización el cual lo representaremos resaltando las actividades a automatizar desde un segundo Diagrama de Actividades por cada Caso de uso de negocio. Esta actividad se desarrollará en una herramienta CASE.

2.4.4. Identificación de artefactos del modelo de análisis de negocio para el caso fashion importaciones

En esta sección, continuaremos analizando el caso Fashion Importaciones visto en la sección anterior, Modelo de Casos de Uso de Negocio. Para efectos de ejemplo, se detallará uno de los procesos: Venta de productos.

Analizaremos la descripción del proceso de venta para crear los Diagrama de Clases y Diagrama de Actividades:

“El proceso de venta de productos inicia cuando una emprendedora se contacta con la empresa para realizar su pedido el cual tiene asignado los datos de la emprendedora y de los productos solicitados. Un Asesor de Ventas toma el pedido y lo registra como “GENERADO” y confirma el monto total para que la emprendedora realice el pago a una cuenta bancaria. Una vez, realizado el pago del pedido, la emprendedora envía la constancia de pago al Asesor de ventas, quien cambia de estado del pedido a “PAGADO”. A continuación, genera el Comprobante de Pago (CDP) y lo envía a la emprendedora vía correo electrónico.”

Asimismo, se debe tener en cuenta los requerimientos que se describen a continuación a fin de considerar un segundo Diagrama de Actividades que plasme el flujo requerido cuando se implante el sistema en dicho proceso:

“Con el fin de agilizar sus ventas y atención de pedidos, así como incrementar el número de emprendedoras, el Gerente de la empresa requiere un sistema que permita a sus emprendedoras registrar, pagar y hacer seguimiento de sus pedidos de los productos correspondientes a un catálogo vigente.”

Por consiguiente, se realizará las siguientes actividades para crear el Modelo de Análisis de Negocio (MAN) e identificar sus componentes:

- Elabore la estructura del Proyecto en una Herramienta CASE.
- Elabore el Diagrama de Clases para el Caso de Uso de Negocio “Venta de productos”.
- Identifique los Trabajadores de Negocio
- Identifique las Entidades de Negocio
- Elabore el Diagrama de Actividades para el Caso de Uso de Negocio “Venta de productos”. (Situación actual)
- Elabore un segundo Diagrama de Actividades para el Caso de Uso de Negocio “Venta de productos”. (Flujo propuesto para atender requerimiento del Gerente de Fashion Importaciones)

Solución del caso en Rational Rhapsody

Debido a que la estructura de este proyecto ha sido creada en la sesión anterior, abriremos este proyecto desde el entorno CASE:

Paso 1: Abra el proyecto FashionImportaciones.

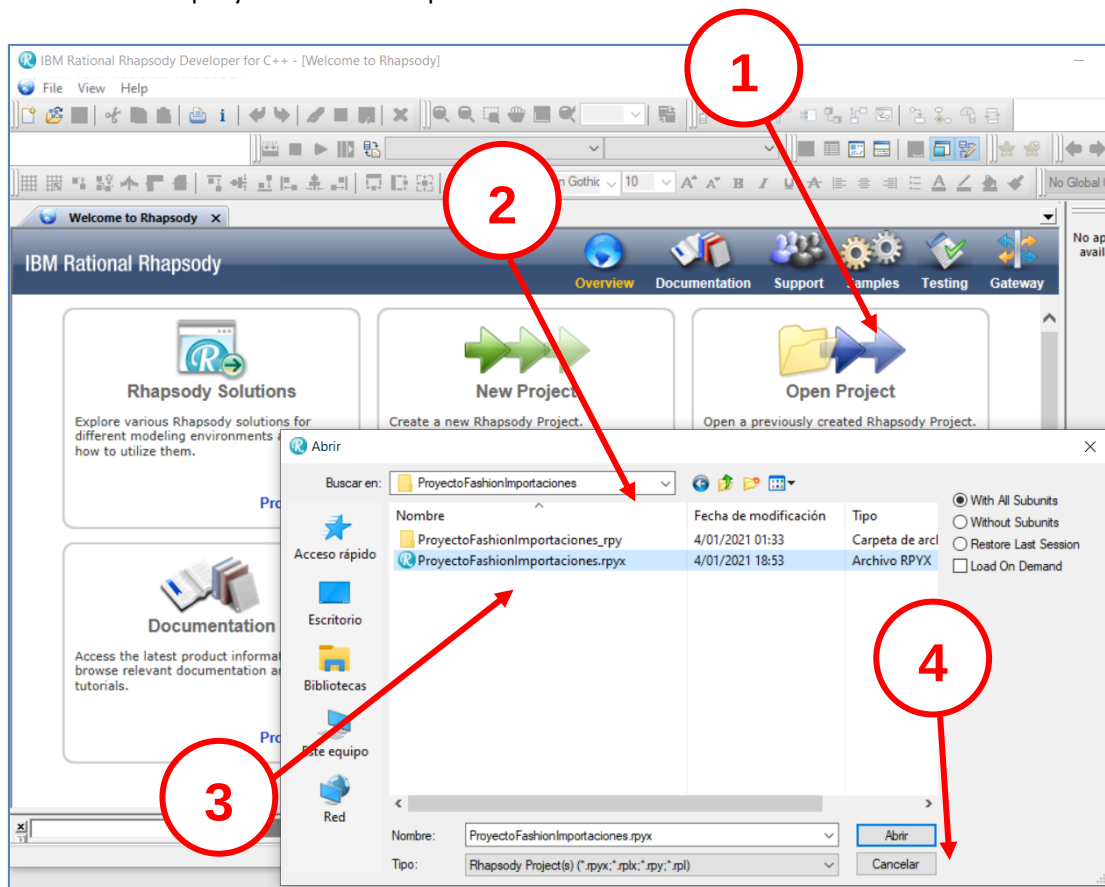


Figura 60: Pasos para abrir un proyecto creado.

Fuente. - -Elaboración propia

Paso 2: A continuación, se mostrará el proyecto FashionImportaciones.

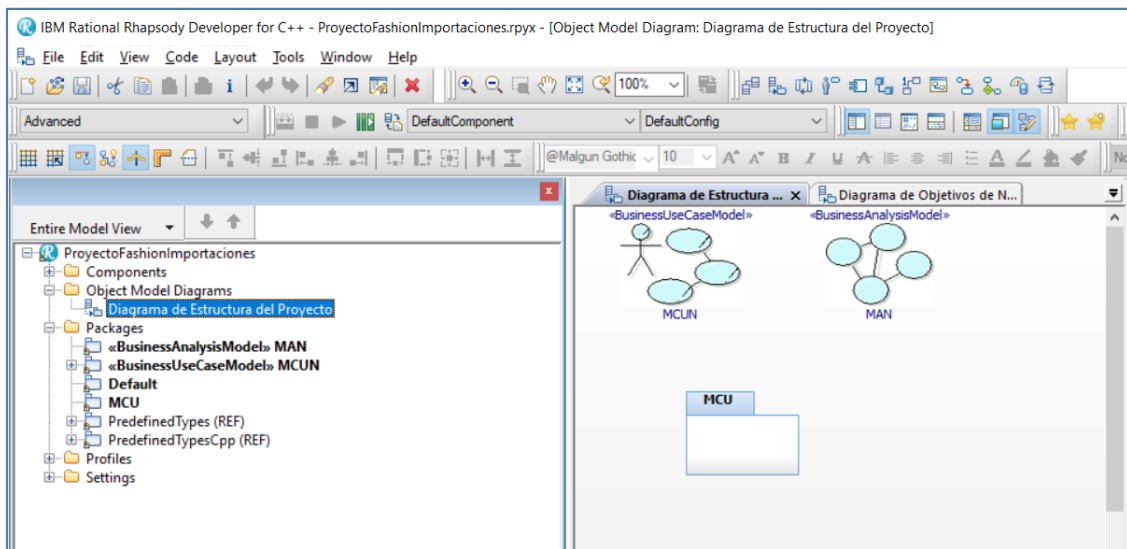


Figura 61: Proyecto “Fashion Importaciones”

Fuente. - Elaboración propia

Paso 3: Ahora, cree el Diagrama de Clases.

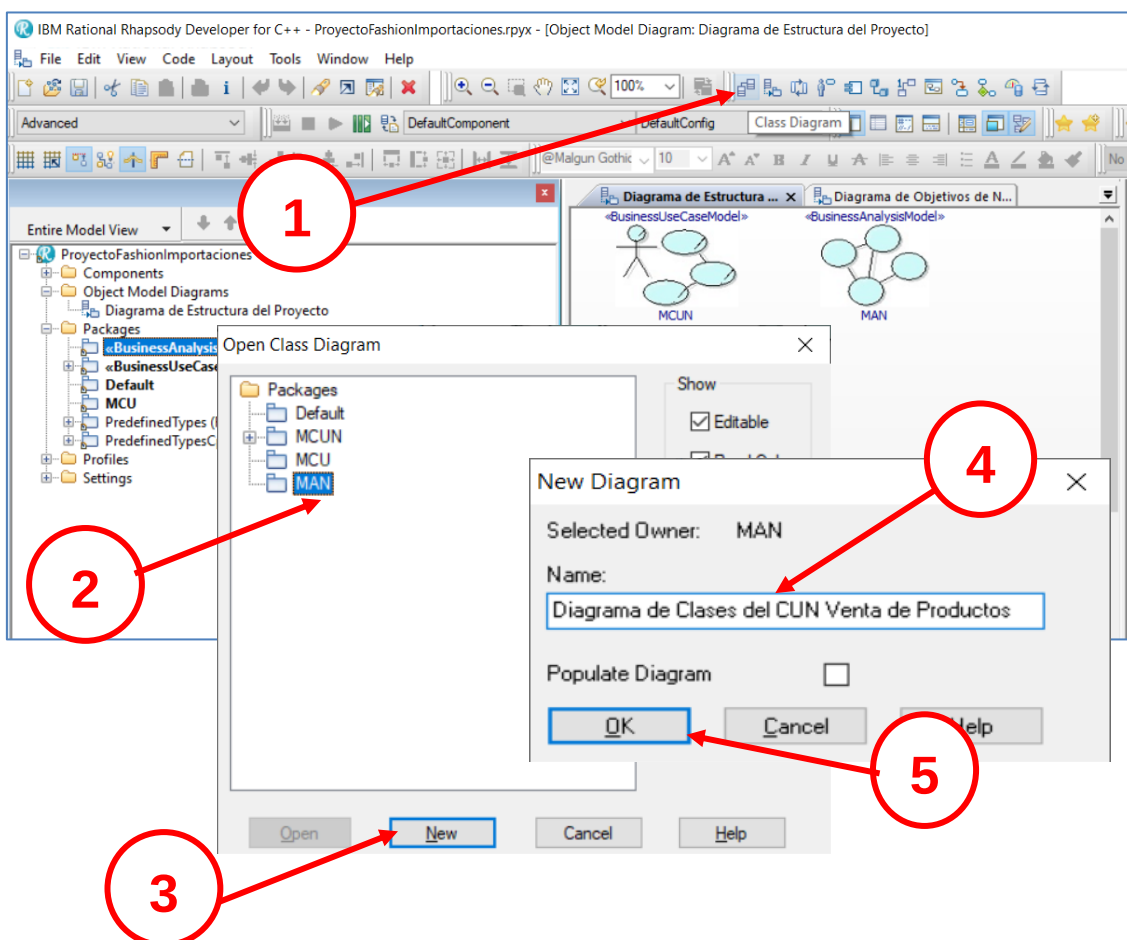


Figura 62: Creación del Diagrama de Clases

Fuente. - Elaboración propia

Paso 4: Sobre el Diagrama de Clases identifique a los Trabajadores de Negocio.

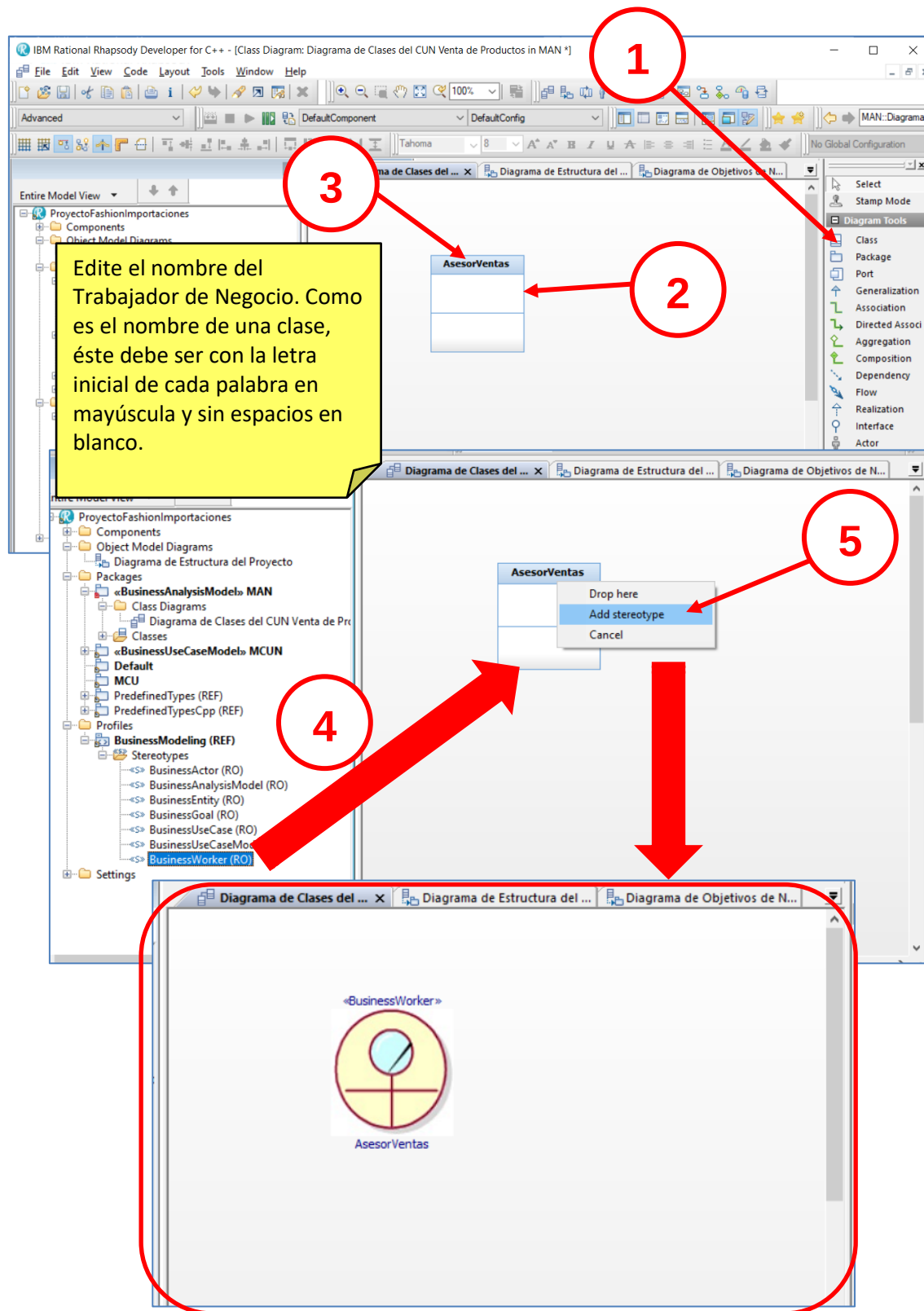


Figura 63: Creación de Trabajadores de Negocio sobre Diagrama de Clases
Fuente. - Elaboración propia

Paso 5: Ahora cree las Entidades de Negocio que son manipulados por el Trabajador de Negocio identificado.

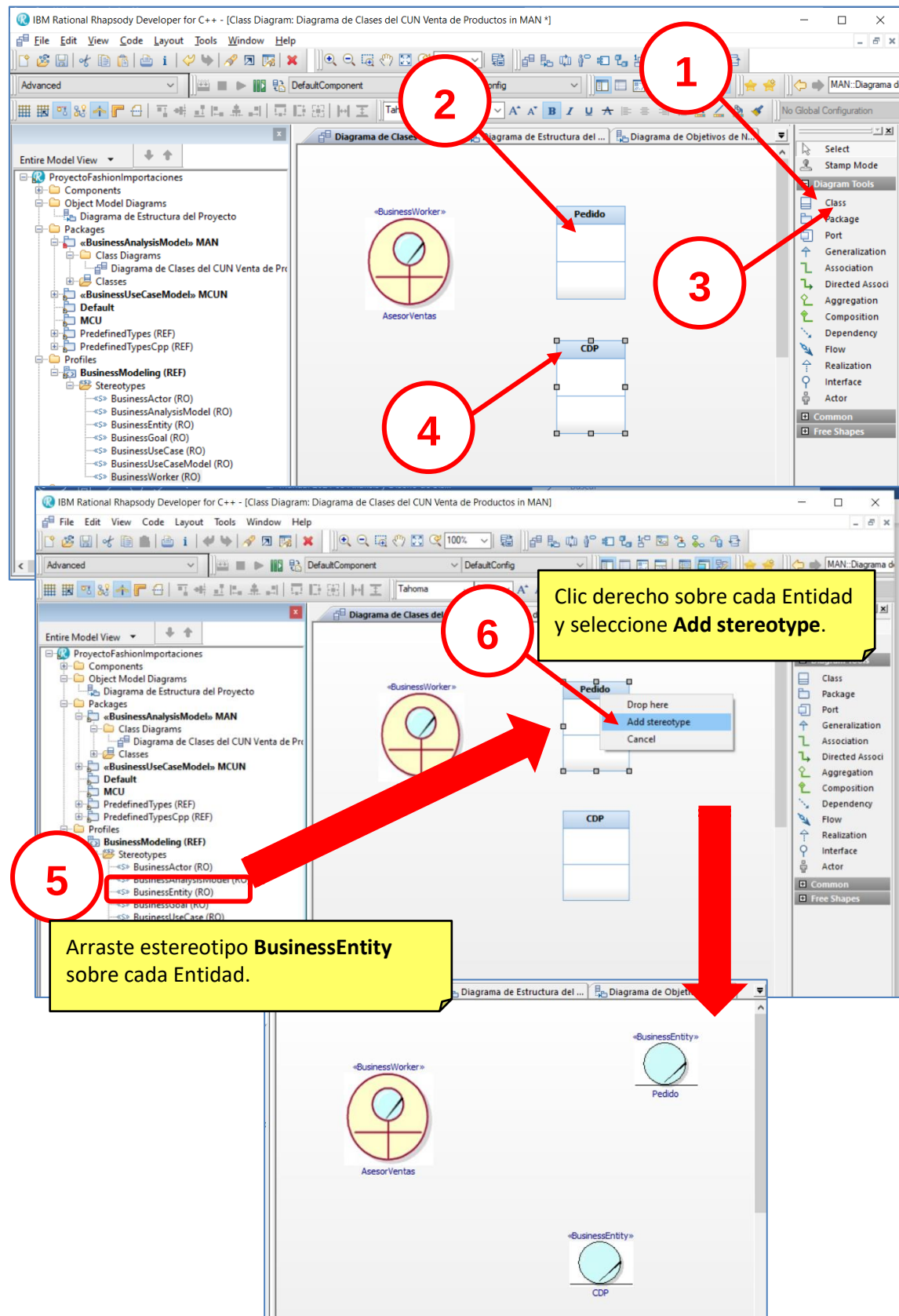


Figura 64: Creación de Entidades de Negocio sobre Diagrama de Clases
Fuente. - Elaboración propia

Paso 6: Ahora agregue las relaciones de asociación entre el Trabajador de Negocio y las Entidades de Negocio.

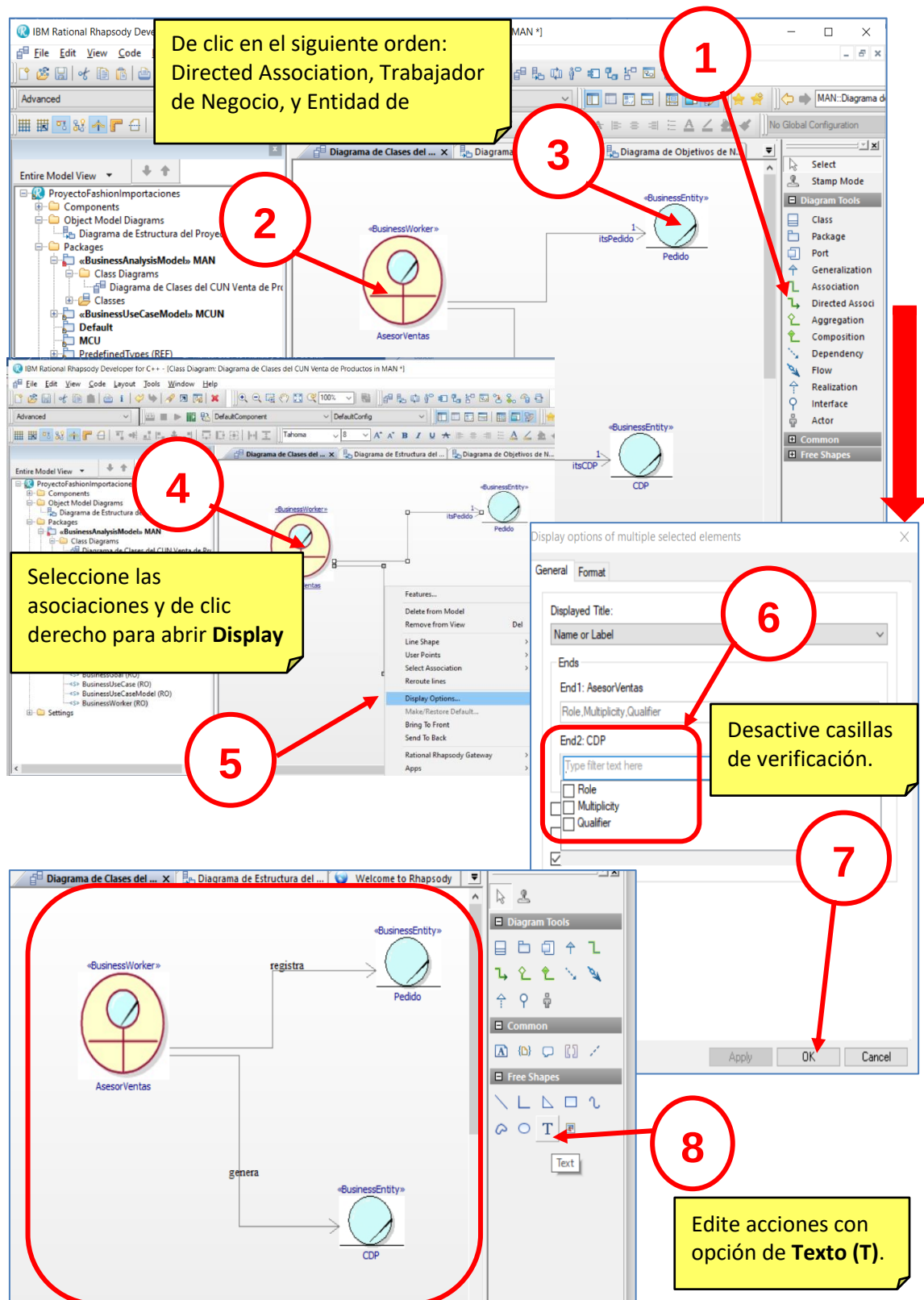


Figura 65: Creación de Asociaciones entre el Trabajador de Negocio y Entidades de Negocio

Fuente: - Elaboración propia

Paso 7: Ahora, cree el Diagrama de Estado para la Entidad de Negocio Pedido. Según el caso, el pedido tiene los siguientes estados: Generado, Pagado, En Ruta, Entregado.

Nota: Se crea Diagramas de estado para las entidades que tengan más de un estado.

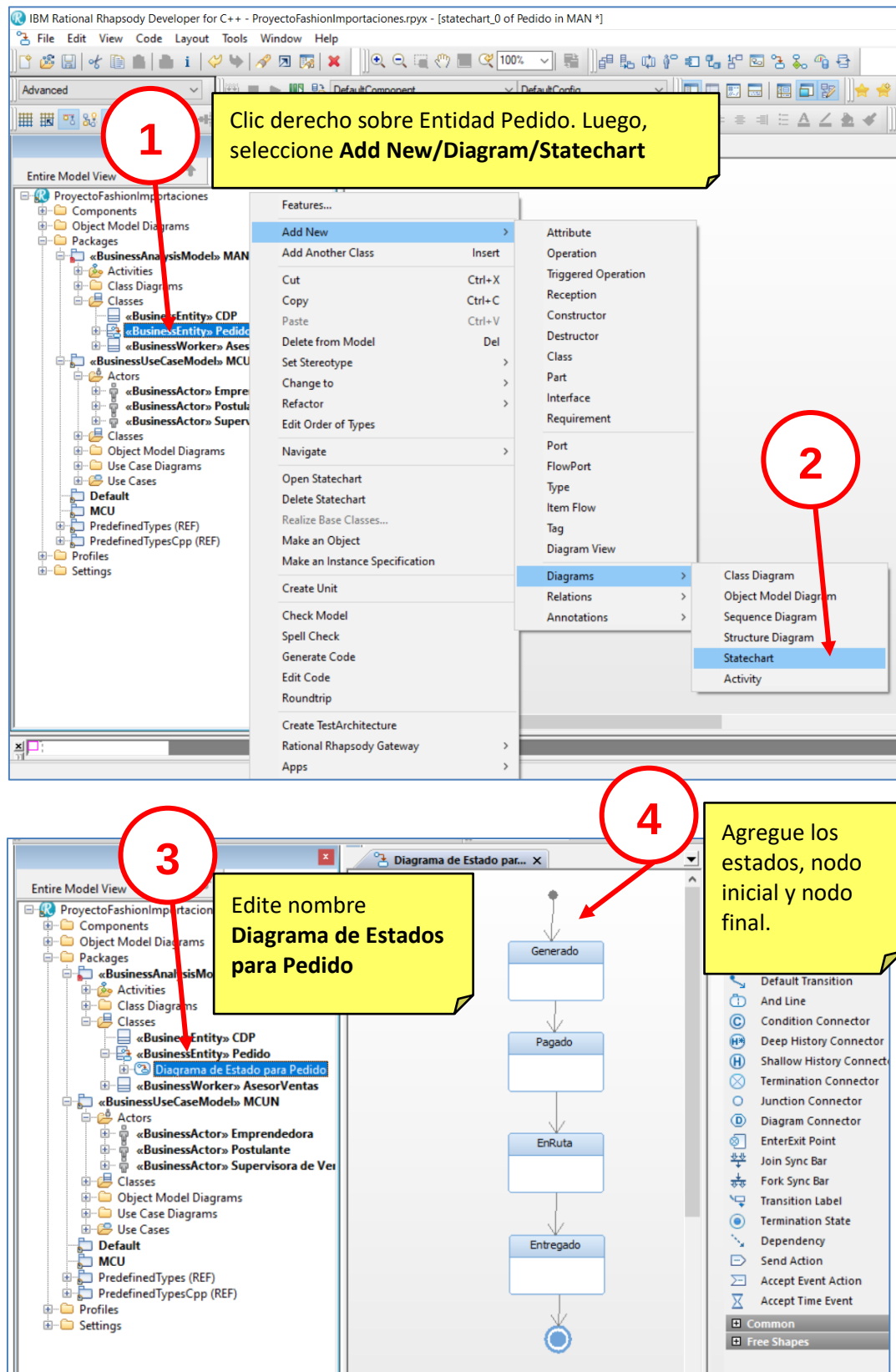


Figura 66: Creación de Diagrama de Estado

Fuente. - Elaboración propia

Paso 8: Ahora cree el Diagrama de Actividades. Antes de empezar a agregar acciones y otros elementos del diagrama de actividades, debe agregar dos componentes: Swimlane Frame y Swimlane Divider.

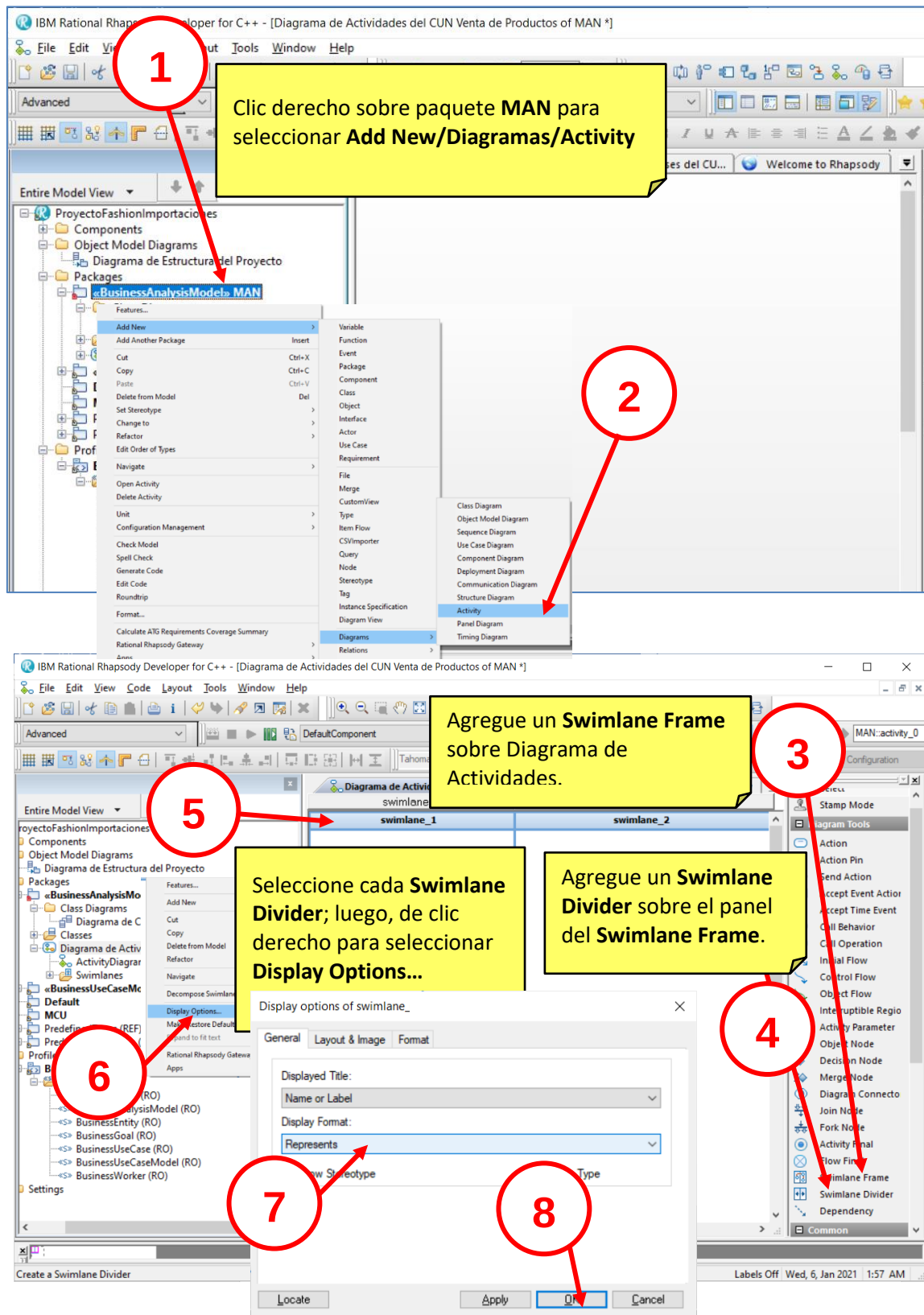


Figura 67: Swimlane Frame and Swimlane Divider sobre Diagrama de Actividades

Fuente: - Elaboración propia

Paso 9: A continuación, arrastre los roles que participan en el flujo del proceso hacia cada cabecera del swimlane.

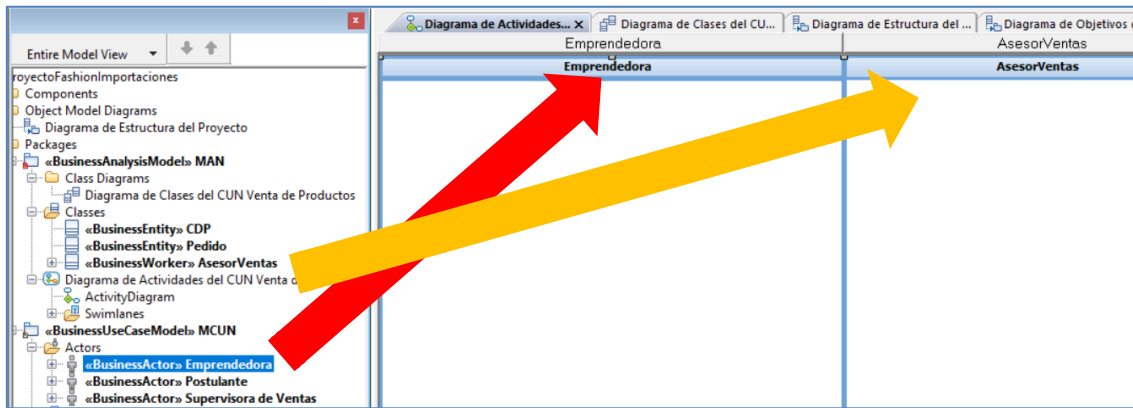


Figura 68: Roles en cabecera de Swimlanes

Fuente. - Elaboración propia

Paso 10: Ahora, agregue el flujo inicial, las acciones por cada rol y el nodo final; arrastre las entidades para representarlos en el Diagrama. Dependiendo del flujo de proceso a representar, utilizará otros componentes adicionales.

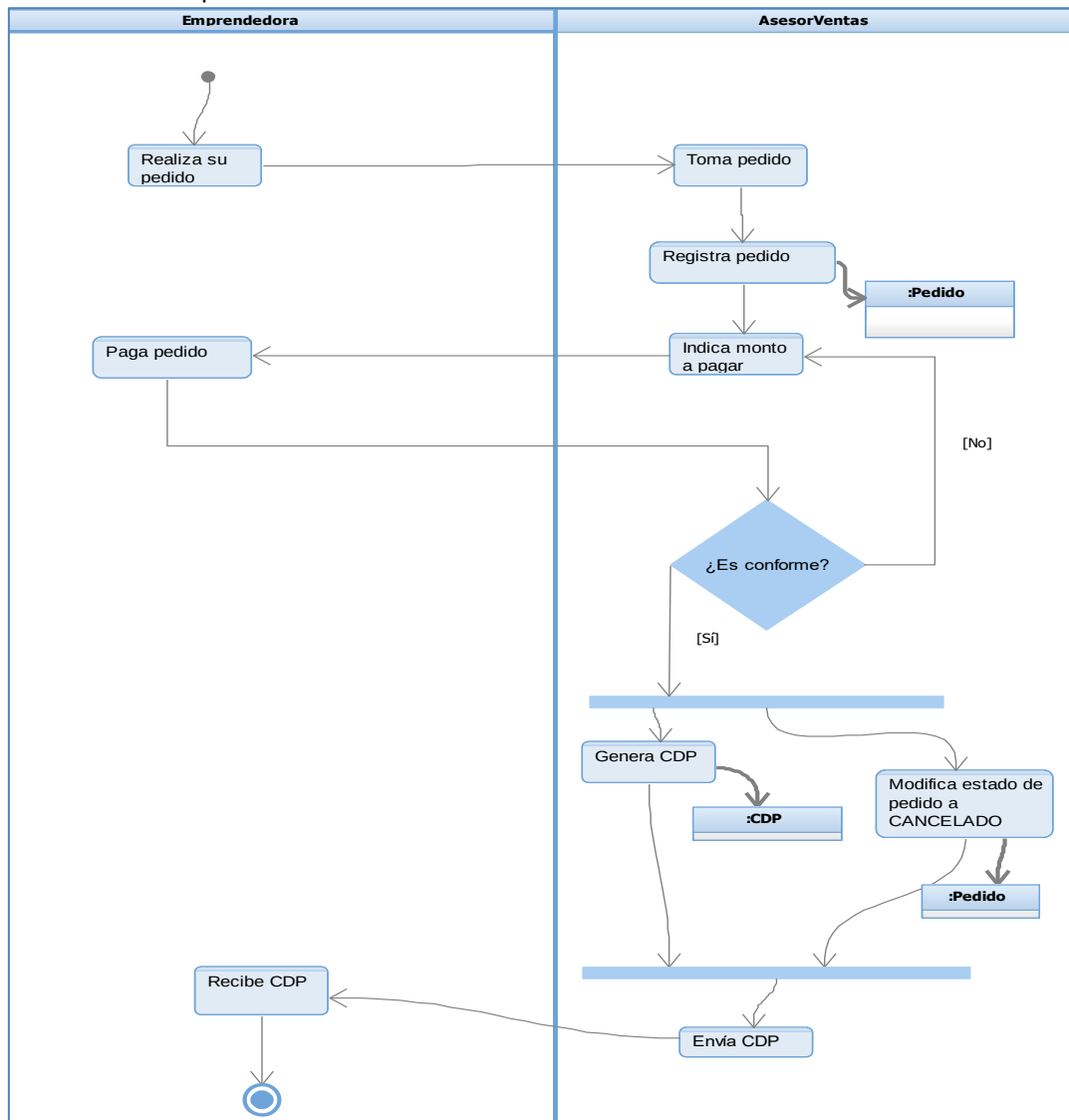


Figura 69: Diagrama de Actividades de la situación actual del CUN Venta de Productos

Fuente. - Elaboración propia

Paso 11: Finalmente, puede crear un segundo Diagrama de Actividades para mostrar una propuesta del flujo del proceso cuando el sistema se implemente.

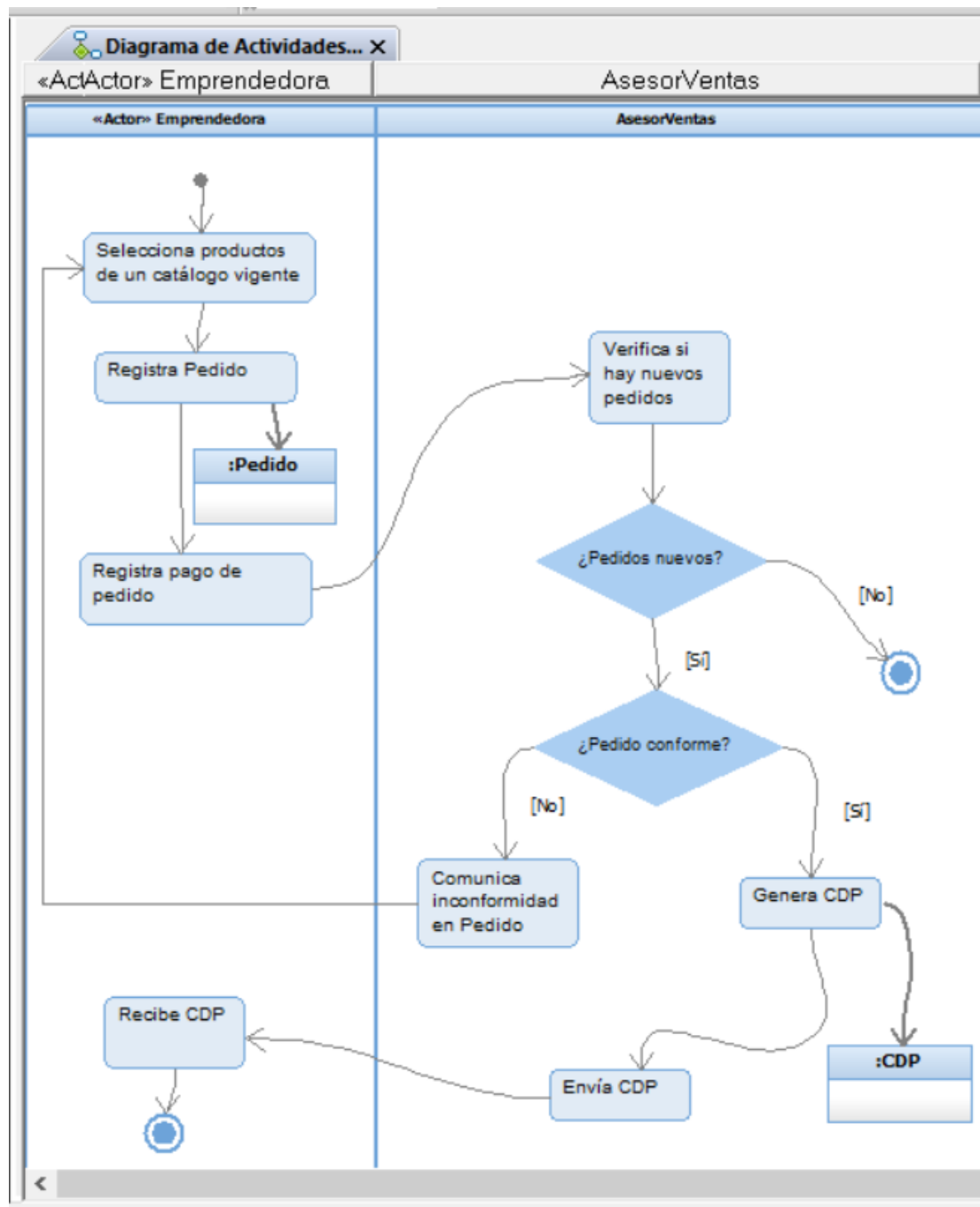


Figura 70: Diagrama de Actividades Propuesto para el CUN Venta de Productos
Fuente. - Elaboración propia

Resumen

1. El Modelo de Análisis de Negocio representa la vista interna del negocio.
2. El Modelo de Análisis de Negocio (MAN) es un modelo que describe el detalle de los casos de uso del negocio. Es una abstracción de cómo los trabajadores del negocio y las entidades de negocio se relacionan y de cómo colaboran para realizar los casos del uso del negocio representado a través de un Diagrama de Actividades.
3. El MAN presenta los siguientes artefactos relevantes: Trabajadores de Negocio, Entidades de Negocio, y por cada CUN se crea un Diagrama de Clases y un Diagrama de Actividades para representar el flujo actual del proceso. Además, puede crearse un segundo Diagrama de Actividades para proponer un flujo del proceso cuando el sistema se implemente.
4. El nombre de un Trabajador de Negocio debe ser un sustantivo que exprese un rol interno al negocio, según las responsabilidades o funciones asignados en el proceso.
5. El nombre de una Entidad de Negocio debe ser un sustantivo que haga referencia a un documento, ficha o registro de datos, o reportes. Puede escribir siglas o acrónimos de la entidad, en caso sea un término utilizado en el negocio.

Recursos

Pueden revisar los siguientes enlaces para ampliar los conceptos vistos en esta unidad:

- http://www.michael-richardson.com/processes/rup_classic/#extend.bus_model/disciplines/rup_business_modeling_discipline_553E20F6.html
- https://www.ibm.com/support/knowledgecenter/SSB2MU_8.3.0/com.ibm.rhp.uml.diagrams.doc/topics/rhp_t_dm_creating_rhap_projects.html

2.5. CASOS PROPUESTOS DE MCUN Y MAN

2.5.1. Caso: Play and Learning Zone S.A.C.

Play and Learning Zone S.A.C. es una empresa que surge en el centro de la ciudad de Lima en el año 2017 como iniciativa de una mujer de negocios llamada Abigail Padilla, persona emprendedora, líder y con gran calidad humana que desea contribuir con la formación complementaria de los niños, para lo cual su foco de atención está en los juegos. Así fue como, desde sus inicios, su empresa ofrece un excelente servicio de alquiler de juegos y de coaching escolar para niños en etapa escolar.

Para adquirir los juegos, la empresa los solicita a sus proveedores con los que trabajó desde un inicio. El Gerente General inicia este proceso cuando brinda las directrices de compra (presupuesto y fechas de compra) al personal del área de compras. Un asistente es el que coordina la compra con los proveedores. Finalmente, el proveedor entrega los productos solicitados.

Recientemente Play and Learning Zone S.A.C. ha diseñado un servicio adicional, talleres para niños de 5 a 12 años, en 3 áreas: Arte, Ciencias, y Robótica. Estos talleres son ofrecidos solamente en colegios, o a domicilio donde expertos profesionales contratados por la empresa dictan los talleres de cada área de forma presencial. Ahora, la emprendedora, motivada por seguir creciendo en el negocio ha impulsado el servicio de ofrecer talleres para dictarlos de forma virtual.

La contratación de personal es iniciada por el Jefe Académico cuando brinda la proyección de requerimientos de docentes por taller a su asistente. El asistente elabora una ficha de requerimiento de personal y el perfil profesional del docente a fin de publicarlo en la página web de la empresa. El asistente registra los datos de un nuevo perfil profesional de docente en caso no se tenga mapeado. Los postulantes interesados envían su CV. El asistente entrevista a los postulantes. Finalmente, la asistente comunica a los postulantes que pasaron la entrevista, que se acerquen a firmar su contrato de docente.

Con la información obtenida, implemente el MCUN y MAN.

2.5.2. Caso: Impulse Center

El gerente general de la empresa “IMPULSE CENTER” ha solicitado los servicios al área de sistemas para que sistematice el proceso de atención de incidentes o requerimientos de Mesa de ayuda. Para llevar a cabo esta sistematización el analista realiza el levantamiento de información entrevistando a las personas que trabajan dentro de este proceso.

A continuación, un resumen de cómo trabaja esta área: El usuario de cualquier departamento de la empresa llama por teléfono a Mesa de ayuda para reportar un requerimiento o una incidencia en su equipo o con algún aplicativo. Un asesor telefónico atiende la llamada, quien recopila la información relevante de la incidencia: nombre del usuario afectado, sede, departamento, teléfono de contacto, correo electrónico, fecha de registro, descripción del requerimiento, o de la incidencia indicando el equipo o aplicación afectada.

Una vez finalizada la llamada, el asesor registra la información como “Solicitud de atención de requerimiento o incidencia” con un número de ticket de atención y en estado PENDIENTE en un archivo Excel compartido con el equipo de Mesa de ayuda. En caso el problema sea resuelto por el asesor telefónico, éste registra un informe técnico con la solución brindada y la solicitud cambia al estado RESUELTO; de lo contrario, lo deriva a un analista técnico. El analista técnico tiene la facultad de asignar el caso a un analista especializado de su equipo. Una vez asignado, la solicitud de atención cambia de estado a ASIGNADO. Cuando el analista especializado toma el caso, cambia el estado de la solicitud de atención a EN PROCESO. Al finalizar la solución de la incidencia, el analista especializado cambia de estado de la solicitud a RESUELTO y genera un informe técnico indicando el número de ticket de atención respectivo.

A fin de entender con más detalle los roles que participan durante el proceso, el analista de sistemas entrevista al jefe de Mesa de Ayuda, quien monitorea la carga de incidencias, el cumplimiento de los SLA (Service Level Agreement) del servicio brindado y la capacidad de atención. En la entrevista obtuvo la siguiente información adicional:

- En el área se maneja los siguientes niveles técnicos de usuarios:
 - Nivel 0: el usuario
 - Nivel 1: asesor telefónico
 - Nivel 2: analista técnico con equipo multidisciplinario, quien tiene la facultad de asignar el caso a un analista especializado de su equipo.
 - Nivel 3: analista especializado, quien atiende el requerimiento o incidencia registrada.
- Un requerimiento puede ser la creación de un usuario de aplicación, instalación de software, creación de cuenta de correo, etc.
- Una incidencia de equipo puede ser un problema técnico en una laptop, desktop, impresora, celular, etc.
- Una incidencia de un aplicativo puede ser de una aplicación de negocio o de ofimática.

Con la información obtenida, implemente el MCUN y MAN.

2.5.3. Otros casos propuestos

Para los siguientes casos, elabore un Modelo de Casos de Uso de Negocio (MCUN) y Modelo de Análisis de Negocio (MAN):

CASO: IMPRENTA LASER DATA

La imprenta “Laser Data”, es una empresa dedicada a la confección de tarjetas y tiene como objetivos satisfacer en un 100% todos los requerimientos de partes matrimoniales, quince años, cumpleaños, bautizo, primera comunión, etc. “Laser Data”, le presta mucha atención a los detalles que las personas buscan, ofreciendo así productos de calidad, acompañado del mejor servicio de atención del pedido y a buenos precios. “Laser Data”, cuenta con una gran variedad de partes, recuerdos y accesorios para todo tipo de ocasión.

Como parte de la estrategia de marketing, “Laser Data” cuenta con socios de negocio para cada ocasión, por ejemplo, en los matrimonios, tiene convenios con empresas que organizan las bodas, revistas para novios, tiendas de regalos, agencias de viajes, etc. Así, cuando una pareja desea casarse y va a uno de los socios de negocio, también llega a contactarse con “Laser Data” por las buenas recomendaciones.

Al contactarse el cliente o el socio con la empresa, este le envía las especificaciones de la tarjeta y cantidad al responsable de pedidos (RP) que toma el pedido, quien lo envía al ejecutivo de cuenta (EC) quien recibe y entrega al diseñador gráfico (DG) el cual genera la versión de la tarjeta de acuerdo a los requerimientos; devolviendo luego al EC para que verifique el resultado en consulta con el cliente quien da la autorización de impresión, por lo que el EC enviara la muestra al equipo de prensa (EP) para que proceda a imprimir el material solicitado.

El responsable del área de diseño se encarga de tener las últimas tendencias en los modelos para las tarjetas o partes, teniendo siempre el catálogo de tarjetas actualizado y dejando para la personalización que el cliente finalmente pueda hacer sobre la tarjeta.

CASO: DON PASTEL

En una empresa dedicada a la elaboración de pasteles y tortas, se realizan las actividades de acuerdo con el detalle siguiente:

El jefe de ventas entrega las órdenes de pedidos al jefe de producción quien las recibe y prepara la orden de producción, el jefe de producción proporciona la orden de producción al operador para que genere su requerimiento de materia prima e insumo en base a la orden de producción, dicho requerimiento es entregado al almacenero de productos en proceso, que se encarga del retiro de las materias primas e insumos, procede a pesar los ingredientes necesarios para producir la torta, luego entrega los ingredientes al operador, donde verifica la conformidad de la recepción firma la atención al requerimiento, en caso contrario no firma documento alguno y solicita que se complete la atención a su requerimiento. Una vez obtenido los materiales e insumos el operador inicia su trabajo agregando los ingredientes a la máquina de batido, haciendo funcionar la máquina, y monitorea el proceso, al término traslada la mezcla a la máquina dosificadora, activándola y verificando que no se presenten problemas en el preparado. Finalmente, el operador organiza las tortas preparadas en las cajas, generando un reporte de producción, entregando los productos y el reporte al almacenero de productos terminados quien los recepciona dando su conformidad a la producción.



DISCIPLINA DE MODELADO – PARTE II

LOGRO DE LA UNIDAD DE APRENDIZAJE

Al término de la unidad, el alumno sustenta el proyecto realizado durante el curso relacionado a la elaboración de la documentación del Modelado de Negocio, Captura de Requisitos, y Prototipos (GUIs) de un caso de estudio utilizando una Herramienta CASE. Además, el alumno es capaz de elaborar prototipos utilizando herramientas de prototipado (GUIs).

TEMARIO

3.1 Tema 8 : Captura de requisitos

- 3.1.1 : Importancia de la captura de requisitos
- 3.1.2 : Dificultades de la captura de requisitos
- 3.1.3 : Artefactos de la captura de requisitos
- 3.1.4 : Reglas básicas de nomenclaturas
- 3.1.5 : Actividades para realizar la captura de Requisitos
- 3.1.6 : Requisitos FURPS+
- 3.1.7 : Técnicas para capturar requisitos
- 3.1.8 : Experiencia de usuario (UX Design)
- 3.1.9 : Captura de requisitos a solicitud del cliente
- 3.1.10 : Identificación de actividades para sistematizar a partir del diagrama de actividades

3.2 Tema 9 : Modelo de casos de uso

- 3.2.1 : Artefactos del modelo de casos de uso
- 3.2.2 : Reglas básicas de nomenclaturas
- 3.2.3 : Identificación de artefactos del modelo de casos de uso para el caso fashion importaciones

3.3 Tema 10 : Estructurando el modelo de casos de uso

- 3.3.1 : Objetivos
- 3.3.2 : Tipos de relaciones
- 3.3.3 : Casos de uso abstractos y concretos
- 3.3.4 : Especificación de casos de uso - ECU
- 3.3.5 : Priorización de casos de uso

ACTIVIDADES PROPUESTAS

- Los alumnos identifican los requisitos funcionales de un caso.
- Los alumnos identifican los requisitos técnicos o no funcionales de un caso.
- Los alumnos identifican los artefactos claves del modelo de casos de uso de cada caso.

3.1. CAPTURA DE REQUISITOS

La Ingeniería de Requisitos es un área de la Ingeniería de Software que abarca las actividades de descubrir, documentar y mantener una serie de requerimientos para un sistema y el uso del término “ingeniería” implica que se debe usar una serie de técnicas sistemáticas, repetibles para asegurar que los requerimientos sean completos, consistentes y relevantes. (Sommerville, 2011)

En AUP la identificación de requisitos es construida a través de un proceso iterativo durante el cual las discusiones entre los desarrolladores del sistema y los clientes (y/o usuarios finales) llevan a una especificación de requisitos en la que todos estén de acuerdo.

3.1.1. Importancia de la captura de requisitos

La importancia de la captura de requisitos radica en lo siguiente:

- Permite estimar costos, tiempos y recursos.
- Disminuye los costos y retrasos del proyecto.
- Mejora la comunicación entre clientes y desarrolladores.
- Evita rechazos de usuarios finales.

3.1.2. Dificultades de la captura de requisitos

Para aplicar un enfoque ágil en el modelado de requisitos es necesario estar en una situación en la que es posible tener éxito, y para muchos equipos de proyecto, lamentablemente, no se da este caso. Muy a menudo los esfuerzos de modelado de requisitos se ven socavados por su entorno, es común descubrir que la cultura de una organización no es propicio para esfuerzos de desarrollo de software eficaces o los stakeholders del proyecto no entienden las implicaciones de sus decisiones. A continuación, se mencionan las dificultades comunes que enfrentan los equipos de desarrollo cuando se trata de modelado de requisitos (Ambler, 2006).

- Acceso limitado o sin acceso a los stakeholders del proyecto. Se percibe falta de participación del usuario.
- Los stakeholders están dispersos geográficamente
- Los stakeholders del proyecto no saben lo que quieren
- Los stakeholders del proyecto cambian de opinión. En este sentido, los requisitos pueden variar con el tiempo.
- Prioridades conflictivas de los stakeholders del proyecto. Inevitablemente, cuando intervienen diversos participantes, los requisitos entrarán en conflicto.
- Muchos stakeholders quieren participar
- Los stakeholders prescriben soluciones tecnológicas
- Los stakeholders no pueden ver más allá de la situación actual
- Los stakeholders no entienden los artefactos del modelado
- Los desarrolladores no entienden el dominio del problema
- Los stakeholders se centran excesivamente en un tipo de requisito
- Los stakeholders del proyecto requieren una formalidad significativa con respecto a los requisitos
- Los desarrolladores no entienden los requisitos. Los requisitos pueden estar incompletos y no son obvios.

3.1.3. Artefactos de la captura de requisitos

El conjunto de artefactos de la captura de requisitos, sirven como entrada y referencia para el análisis, diseño, implementación y pruebas del sistema.

La propuesta del curso, para una solución de mediana envergadura, es crear los artefactos proporcionados en la siguiente tabla:

Artefacto	Descripción
 Especificación de Software	La Especificación de Software es un documento que enfoca la organización completa de los requisitos del proyecto: funcionales y no funcionales (técnicos). Un requisito es una condición o capacidad que el sistema debe cumplir. Los requisitos funcionales definen el comportamiento del sistema. Derivan o se traducen en casos de uso. Los requisitos técnicos o no funcionales especifican propiedades significativas del sistema como, por ejemplo: facilidad de uso, de seguridad y de rendimiento. Esta categoría de requisito se especifica para un caso de uso, para un grupo de casos de uso o para todo el sistema. En este documento también se incluye la lista de casos de uso y actores, y la Especificación de los casos de uso.
 Caso de Uso	Es un proceso específico del sistema con identidad propia, el cual define una secuencia de acciones que el sistema realiza para un actor en particular. Es derivado por uno o más requisitos funcionales.
 Actor	Representa un rol (humano, hardware o software) externo al sistema con el que se establece intercambio directo de información. Puede ser asociado a uno o más casos de uso.
 Matriz de Actividades Vs. Requisitos	Es una matriz que muestra la trazabilidad entre Actividades de Negocio que se pueden sistematizar y Requisitos funcionales. Asimismo, se incluye la trazabilidad entre Requisitos funcionales y Casos de Uso.
 Matriz de Requisitos Vs. Casos de Uso	Es una matriz que muestra la trazabilidad entre Requisitos y Casos de uso.

Figura 71: Artefactos de la Captura de Requisitos

Fuente. - Elaboración propia

3.1.4. Reglas básicas de nomenclaturas

En la siguiente tabla se mencionan las reglas de nomenclatura de dos de los artefactos que se identifican en la captura de requisitos: requisito funcional y requisito técnico o no funcional. Más adelante, en el siguiente apartado se indicarán las reglas de nomenclaturas para actor y caso de uso.

Artefacto	Reglas de nomenclatura
Requisito funcional	El enunciado de un requisito funcional debe incluir un código o identificador, un nombre y una breve descripción o especificación. El nombre del requisito funcional puede seguir las mismas reglas de nomenclatura que se utilizan para un caso de uso, el cual se explica en el siguiente apartado. Ejemplos: RF001-Generar CDP. El sistema permitirá al cajero ingresar el Comprobante de Pago (CDP) asignando los datos del cliente y de los productos que compró. RF002-Buscar cliente. El sistema permitirá al usuario buscar los datos del cliente. RF002-Buscar productos. El sistema permitirá al usuario buscar los datos de productos.
Requisito técnico o No funcional	El enunciado de un requisito técnico o no funcional debe incluir un código o identificador, un nombre y una breve descripción o especificación. Ejemplos: RNF001-Disponibilidad del sistema. El sistema estará disponible el 99,99 % del tiempo durante cualquier período de 24 horas. RNF002-Tiempo mínimo de búsqueda de un seminario. Una búsqueda de seminarios se producirá en menos de tres segundos el 95 por ciento del tiempo. RNF003- Tiempo máximo de búsqueda de un seminario. Una búsqueda de seminarios se llevará a producir en no más de diez segundos el 99 por ciento del tiempo.

Figura 72: Reglas de Nomenclatura de artefactos de la Captura de requisitos

Fuente. - Elaboración propia

3.1.5. Actividades para realizar la captura de requisitos

Antes de mencionar las actividades para realizar la captura de requisitos del sistema, es preciso recordar que nuestros esfuerzos se deben orientar con un enfoque ágil.

Es muy fácil que el modelado de casos de uso se vuelva anti-ágil. Para evitar que esto suceda es necesario centrarse en la creación de artefactos que son “lo suficientemente bueno”, no es

necesario ser perfecto. Algunos proyectos se desvían porque la gente pensaba que los requisitos tenían que ser redactados perfectamente. Tenga en cuenta que “No está escribiendo la Carta Magna”. Recuerde siempre el principio del modelado ágil “Maximizar la inversión de los stakeholders y solo hacer las cosas que agreguen valor. Dado que en un entorno ágil pasará rápidamente a escribir código en función de esos requisitos, descubrirá que, si no comprende completamente lo que se requiere, trabajará estrechamente con su stakeholder para hacerlo y construirá algo que satisfaga sus necesidades reales. Es desarrollo de software, no desarrollo de documentación. (Ambler, 2006)

Las actividades orientadas a la captura de requisitos son las siguientes:

- Identificar oportunidades de automatización. En esta actividad se identifican los primeros requisitos funcionales. Pueden ser brindados directamente por los usuarios (los usuarios mencionan sus requerimientos o historias de usuario, luego, el equipo de desarrollo los especifica como requisitos funcionales) o identificados a partir de un diagrama de actividades del flujo de un proceso.
- Identificar casos de uso. Se identifican los casos de uso y sus actores (roles de usuario que utilizarán el sistema) a partir de los requisitos funcionales especificados por el equipo de desarrollo.
- Identificar requisitos técnicos. Esta actividad tiene como objetivo especificar los requisitos técnicos o no funcionales en el documento de Especificación de Software. Las fuentes para obtenerlos son diversas, es decir, cualquier stakeholder del proyecto podría iniciar con la indicación de algún requisito técnico. Sin embargo, los desarrolladores son los responsables de especificarlos apropiadamente, los cuales pueden ser restricciones para todo el sistema, para un solo caso de uso o para un conjunto de casos de uso.
- Diseñar interfaces de usuario. Es importante que el analista diseñe prototipos de interfaces gráficas de usuario para los primeros casos de uso a implementar. Después de crearse estos prototipos como parte de la especificación de los casos de uso, debe ser evaluado por las partes interesadas para comprobar que satisface sus necesidades. Por su experiencia, (Ambler, 2006) sugiere evaluar los prototipos diseñados con las siguientes preguntas, las cuales proporcionan comentarios significativos para la mejora de los diseños: ¿Qué tiene de bueno el prototipo de interfaz de usuario?, ¿Qué tiene de malo el prototipo de interfaz de usuario?, y ¿Qué falta en el prototipo de interfaz de usuario? **Realizar esta actividad ayudará a definir otros casos de uso (incluidos y/o extendidos)**
- Explorar facilidad de uso. Esta actividad **permitirá identificar requisitos especiales asociados a cada interfaz gráfica de usuario del sistema.**
- Priorizar requisitos. Es clasificar los casos de uso según su importancia para establecer el orden de implementación de estos. Más adelante se brinda mayor explicación.

3.1.6. Requisitos FURPS+

Este es un tipo de clasificación de requisitos especificado en la documentación de RUP. Se utiliza el acrónimo FURPS (por las siglas en inglés) para describir las principales categorías de requisitos:

- Funcionalidad (Functionality)
- Facilidad de Uso (Usability)
- Confiabilidad (Reliability)
- Rendimiento (Performance)
- Soporte (Supportability)

El símbolo "+" en FURPS+ hace referencia a que se deben incluir otros requisitos, tales como:

- Restricciones de diseño
- Requisitos de implementación
- Requisitos de interfaz
- Requisitos físicos

Funcionales: Los requerimientos funcionales deben incluir: Conjunto de características, Capacidades, y Seguridad.

Por ejemplo, para un Sistema de Ventas:

- R1: El sistema debe permitir mostrar descripción y precio de productos.
- R2: El sistema debe permitir registrar venta de productos.
- R3: El sistema debe permitir reducir stock cuando se realiza la venta.
- R4: El sistema debe permitir identificar al cajero utilizando un usuario y una clave.

Facilidad de Uso: Los requerimientos relacionados a la facilidad de uso deben incluir subcategorías tales como: Factores Humanos, Estéticos, Consistencia de Interfaz de Usuario, Ayuda en línea o "context-sensitive", Asistentes ("wizards"), Documentación de Usuario, y Materiales de Capacitación/Entrenamiento. Por ejemplo:

- R1: El sistema deberá proporcionar ayudas en línea para orientar al usuario en el uso de sus interfaces.
- R2: El sistema debe tener interfaces gráficas de administración y de operación en idioma español y en ambiente 100% Web, a través de navegadores de Internet.
- R3: Maximizar eficiencia mediante la navegación con teclado.

Confiabilidad: Dentro de los requerimientos relacionados a la confiabilidad podemos mencionar: Frecuencia de fallos, Capacidad de recuperación a fallos, Posibilidades de predicción del programa, Precisión, y Tiempo medio de fallos. Por ejemplo:

- R1: El sistema debe registrar los pagos a crédito autorizados que se hagan a las cuentas por cobrar en un plazo de 24 horas, aun cuando se produzcan fallas de energía o del equipo.
- R2: La cuenta del usuario se bloqueará por un lapso de 30 minutos luego de 4 intentos fallidos para evitar vulnerabilidades en la seguridad del sistema.

Rendimiento: Los requerimientos de rendimiento están relacionados a las condiciones impuestas a requisitos funcionales y son tales como: Velocidad, Eficiencia, Disponibilidad, Tiempo de Respuesta, Tiempo de Recuperación, y Uso de recursos. Por ejemplo:

- R1: El tiempo máximo para mostrar el reporte de cuentas por cobrar mediante un histograma es de 10 segundos.
- R2: El sistema debe estar disponible al 100% o muy cercano a esta disponibilidad durante el horario hábil laboral de la empresa a nivel nacional, es decir, de lunes a viernes de 8:00 a.m. a 06:15 p.m., con excepción de los días festivos.

Soporte: Los requerimientos de soporte están relacionados en la capacidad que tiene el software de ser modificado fácilmente para adecuar mejoras o cambios e incluyen: Adaptabilidad, Facilidad de mantenimiento, Compatibilidad, Configurabilidad, Facilidad de instalación, e Internacionalización. Por ejemplo:

- R1: El sistema debe operar de manera independiente del navegador que se utilice (Microsoft Internet Explorer 6.0 o superior, Netscape 6.0 o superior, Mozilla FireFox).

- R2: El sistema deberá estar orientado a que las actualizaciones sólo se hagan en el sitio del servidor.

Restricciones de diseño: Los requisitos de diseño son también llamados restricciones de diseño, pues especifican o restringen el diseño de un sistema. Por ejemplo:

- R1: El sistema deberá considerar en su arquitectura un modelo tres capas, donde se definen tres componentes lógicos de manera independiente: servicios de presentación o interfaz de usuario, servicios de funcionalidad y servicios de datos.
- R2: El sistema de matrícula deberá considerar la utilización de un servicio web con la RENIEC para verificar los datos del alumno.

Requisitos de implementación: Los requerimientos de implementación especifican restricciones de codificación o de construcción del sistema: Estándares requeridos, Lenguajes de implementación, Políticas para la integridad de Bases de Datos, Límite de recursos, y Ambientes de Operación. Por ejemplo:

- R1: El sistema debe desarrollarse en ASP.NET C# con framework 4.5.
- R2: La SGBD utilizado por el sistema será MySQL.
- R3: El sistema debe ser publicado en un servidor IIS 7.5.

Requisitos de Interfaz: Los requerimientos de interfaz especifican: Elementos externos con el que el sistema debe interactuar, y Restricciones o formatos, tiempos u otros factores usados en tales interacciones. Por ejemplo:

- R1: El sistema deberá proporcionar, para los diferentes reportes solicitados, salidas en documentos electrónicos (Word, Excel y PDF Acrobat).
- R2: En una visión preliminar de impresión se consideraría que todos los textos estarán relacionados con un visor de PDF, las estadísticas y resultados de consultas estarán relacionados con Excel 2007 o superior.

Requisitos Físicos: Los requisitos físicos especifican características físicas con las que el sistema debe contar (material, forma, tamaño y peso). Pueden especificarse los requisitos de hardware. Por ejemplo:

- R1: Para que un cliente de la aplicación pueda ejecutar procesos, en línea, considerados en el sistema, la estación de trabajo del usuario deberá cumplir con los siguientes requisitos mínimos:

Requisitos Físicos
▪ Procesador 1.0 GHz. ▪ Memoria 128 MB. ▪ Disco duro 10 GB. ▪ Sistema Operativo Windows XP o Linux. ▪ Navegador internet Explorer 6.0 o posterior, Mozilla Firefox 2.X, Netscape Navigator 6.X o posterior con plugins para Macromedia Flash y Java. ▪ Conexión a Internet. mínimo 56Kbps.

3.1.7. Técnicas para capturar requisitos

Las técnicas de recolección de información de la ingeniería de requisitos surgen como medio para mejorar la comunicación entre usuarios o clientes y el equipo de trabajo que desarrolla el software. Esta técnica es importante por dos razones:

1. En ocasiones los desarrolladores no conocen todos los detalles del trabajo de la organización para la cual están desarrollando el sistema.
2. Los usuarios no saben que información es necesaria y relevante para el desarrollo de un sistema.

A continuación, se muestra un listado de herramientas para la captura de requisitos en las etapas que estas normalmente son aplicadas:

Técnicas	Extracción	Análisis	Especificación	Validación
Entrevistas y/o cuestionarios	X			
Sistemas existentes	X	X		
BrainStorming	X	X		
Observación	X			
Prototipo Bosquejado	X	X	X	
Prototipo Tangible / Usable	X		X	X
FODA		X		
Cadena de Valor		X		
Modelo de Clase Conceptual		X	X	
Diagrama de Pescado	X	X	X	
Glosario	X	X	X	X
Casos de Uso	X	X	X	X
CheckList	X			X

Figura 73: Tabla con las herramientas para captura de requisitos

Fuente. - Elaboración propia

3.1.8. Experiencia de usuario (UX Design)

¿Qué es UX Design o Diseño de experiencia de usuario?

UX Design es una filosofía de diseño que se enfoca en crear productos que resuelvan necesidades de los usuarios. Este tipo de diseño se puede dar en cualquier tipo de producto: un software, una televisión o un automóvil.

El objetivo del Diseño de experiencia de usuario es crear una experiencia única y adaptada al cliente. Por ejemplo, con respecto a las páginas web de una empresa, este enfoque se ocupa de facilitar a los usuarios la navegación de las páginas del sitio web para que encuentren lo que quieren y lo que ofrece la empresa de la manera más sencilla posible. Por tanto, es necesario identificar qué necesidades tienen los usuarios, alinearlos con los objetivos del negocio y tener en cuenta las limitaciones tecnológicas o técnicas de la empresa.

Fases del Proceso de diseño UX

Al igual que todas las actividades digitales dirigidas hacia el usuario, UX Design se enfoca en realizar análisis e investigaciones utilizando técnicas online y offline, para recoger datos valiosos sobre los usuarios. Estos datos son esenciales para el diseño de un sitio web y sus contenidos.

Después de haber identificado y analizado al público relevante, llega el momento de la elaboración de la experiencia del usuario que uno desea crear:

Planificar: A través de una fase de brainstorming se generan ideas para aprovechar las oportunidades y resolver los problemas que surgen en la fase del análisis de la investigación del usuario. Recopilamos todas las ideas, comentarios y sugerencias del equipo de diseño, que surgen con total libertad, recogiendo incluso las propuestas más banales y disruptivas, para evaluar todas las posibilidades. Posteriormente, se examinan y seleccionan aquellas que se consideran relevantes para el proyecto. Esta es la fase post-it. Al organizar los post-its en un tablero de anuncios, se puede crear un mapa visual que abrirá el debate para seleccionar las ideas más válidas.

Procedemos entonces a crear un Sitemap basado en las ideas surgidas en la fase anterior, para resaltar la importancia del contenido y la estructura de navegación de un sitio web. Estos mapas a menudo también se producen para aplicaciones móviles y son útiles para mostrar cómo se organizarán los contenidos, se dividirán en secciones y páginas individuales, y cómo el usuario puede moverse de una sección a otra con facilidad. En general, también se establece un primer diagrama del flujo de la ruta de conversión del usuario.

Se crea entonces el Wireframe de la nueva web, un prototipo digital, y trabajamos en la entrega al cliente del mock-up. Estas son generalmente imágenes estáticas, que se pueden transformar en presentaciones interactivas, para que el cliente entienda la navegabilidad y la interacción con el usuario.

Presentación y Pruebas de entrega: Una vez obtenida la aprobación de los clientes, se crea un prototipo basado en la web, interactivo y navegable para confirmar la funcionalidad y la satisfacción mediante la prueba.

Prueba de Usabilidad: La UX está enfocada en el usuario. Por esta razón, es imposible probar el prototipo de la web sin probarlo primero en usuarios reales.

Existen muchas técnicas para evaluar el éxito de un proyecto de diseño de UX, que aplicándolas puedes crear un informe de usabilidad, que se compartirá con el cliente, generalmente compuesto por:

- Información sobre las pruebas realizadas: que se probó, dónde, cuándo y con qué equipo.
- Metodología: cómo se realizó la encuesta, qué actividades se solicitaron a los usuarios, qué datos fueron recogidos, quiénes eran los participantes y cuáles eran sus características demográficas.
- Análisis de los resultados: la presentación de los datos recopilados consiste en gráficos, infografías y posibles comentarios de los usuarios.
- Resultados y recomendaciones: una lectura de lo que surge a partir del análisis de datos, incluyendo los aspectos más apreciados y negativos, junto con una propuesta para la resolución de problemas, la optimización del diseño y la interfaz de usuario.

Análisis de Resultados: Después de haber hecho el producto final y de lanzarlo, es necesario monitorear los resultados de la web, verificar los datos de uso y la composición del grupo de usuarios para obtener información sobre cómo mejorar la usabilidad.

3.1.9. Captura de requisitos a solicitud del cliente

Si el equipo de desarrollo tiene un conocimiento de la estructura de la organización, de las metas, de la visión y de los clientes/usuarios o si solo se está añadiendo una nueva característica a un sistema existente, entonces AUP no recomienda que se empiece con un Modelado del negocio. En ese caso, se empieza con la identificación de todas las necesidades de stakeholders o historias de usuario¹⁰ (user stories en inglés) para obtener y analizar requisitos del sistema.

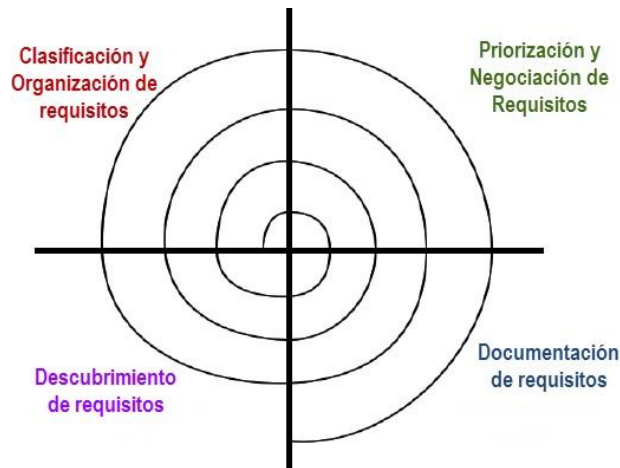


Figura 74: Obtención y análisis de requisitos

Fuente. - Elaboración propia

3.1.10. Identificación de Actividades para sistematizar a partir del diagrama de actividades

Mediante la utilización del modelo del negocio como entrada, específicamente del Diagrama de Actividades de cada Caso de Uso de Negocio, el analista puede emplear una técnica sistemática para crear un modelo de casos de uso tentativo.

Es importante documentar el seguimiento de estos elementos: actividades a informatizar, requisitos funcionales y casos de uso. Más aún, si se trata de seguir requisitos funcionales de casos de uso, el cual es un proceso complicado por el hecho de que existe una relación muchos a muchos entre ellos. Un caso de uso puede tratar muchos requisitos funcionales y un requisito funcional puede estar presente en varios casos de uso diferentes. Para ello, podemos utilizar herramientas de ingeniería de requisitos como el Requisite Pro y DOORS, pero también se puede realizar manualmente mediante hojas de cálculo, como la siguiente plantilla:

Matriz de Actividades vs Requisitos Funcionales del Sistema <Nombre_de_Sistema>						
Proceso de Negocio	Actividades de Negocio	Responsable de Negocio	Requisito		Caso de Uso	Prioridad
			R01			
			R02			
			R03			

Figura 75: Plantilla de Matriz de Actividades vs Requisitos Funcionales

Fuente. - Elaboración propia

¹⁰ Historias de usuario, Una historia de usuario es una definición de muy alto nivel de un requisito, que contiene la información suficiente para que los desarrolladores puedan producir una estimación razonable del esfuerzo para implementarlo. Son necesidades de stakeholders que hacen referencia a una amplia variedad de tipos de requisitos: funcionales y técnicos.

La identificación de Actividades para sistematizar a partir del Diagrama de Actividades conlleva a la realización de las siguientes actividades:

- Creación de un Diagrama de Actividades que muestre el flujo propuesto para un Caso de Uso de Negocio al implementar el sistema.
- Resaltar con otro color, las actividades candidatas a ser informatizadas. Serán aquellas referidas a alguna transacción SQL como INSERT, DELETE, UPDATE, SELECT. Por ejemplo, “Ingresar datos de cliente” (INSERT), “Eliminar datos de productos” (DELETE), “Cambiar contraseña” (UPDATE), “Consultar cotizaciones aceptadas” (SELECT).
- Luego, dichas actividades resaltadas se colocan en la columna de Actividades de Negocio en la matriz de Actividades Vs. Requisitos.
- A continuación, a partir de dichas actividades se obtienen los requisitos funcionales. Para ello, se empieza con un verbo en infinitivo, y a continuación sigue el dato a manipular. Por ejemplo: Registrar productos, Evaluar cotización, Generar CDP, Eliminar clientes, etc. En general puede seguir las mismas nomenclaturas de casos de uso.
- Finalmente, a partir de los requisitos se crean los casos de uso. Los actores se identificarán a partir de los roles de negocio que realizan las actividades del negocio que se tienen en la matriz. Para estos artefactos (casos de uso y actores) se sigue las nomenclaturas mencionadas en la figura 79.

Como ejemplo se creará la matriz de Actividades Vs. Requisitos del caso Fashion Importaciones para el proceso de Venta de productos:

Paso 1: Se crea un segundo Diagrama de Actividades para mostrar una propuesta del flujo del procesode “Venta de productos” cuando el sistema se implemente.

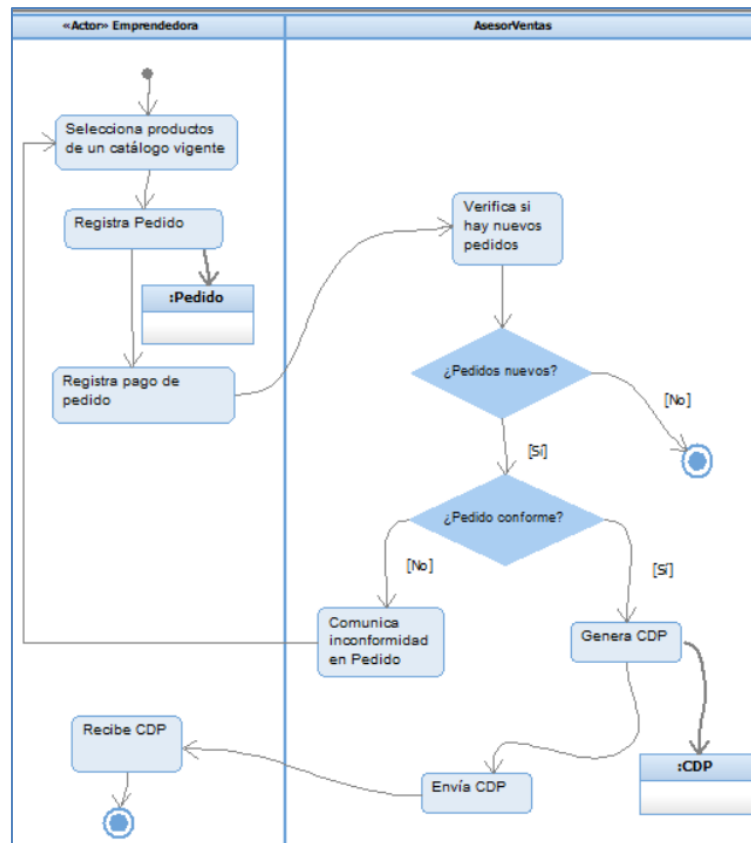


Figura 76: Diagrama de Actividades Propuesto para el CUN Venta de Productos

Fuente. - Elaboración propia

Paso 2: Ahora, en este segundo Diagrama de Actividades se resalta las actividades a sistematizar.

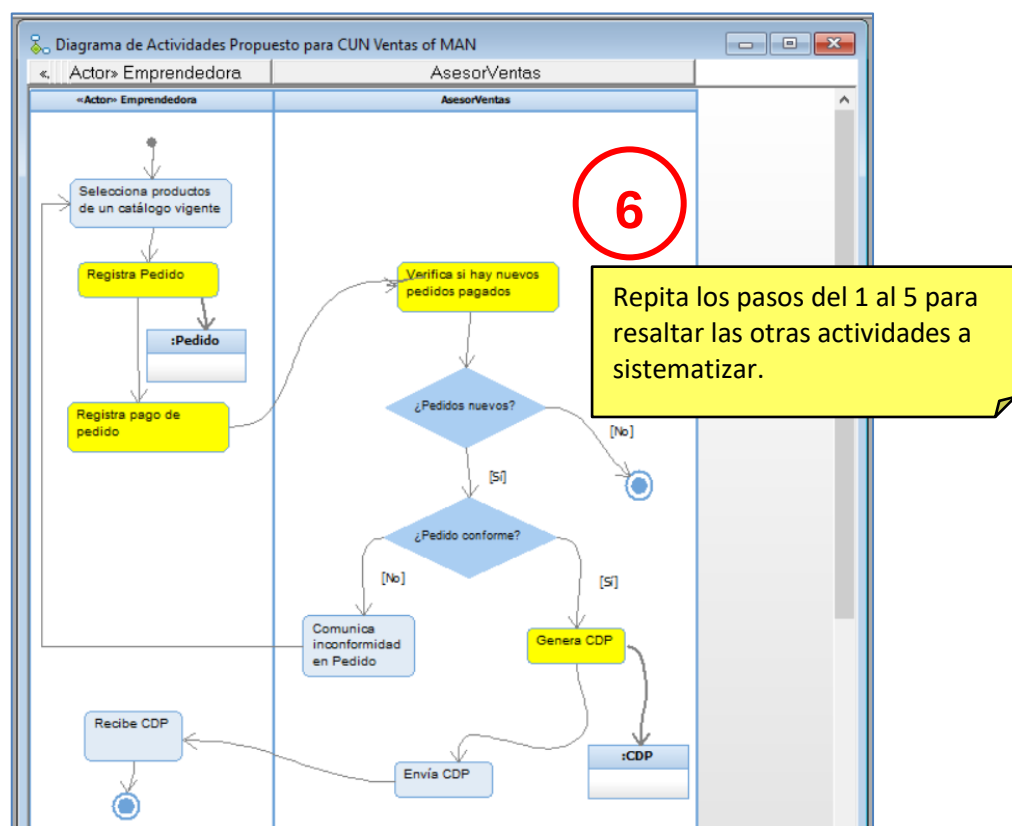
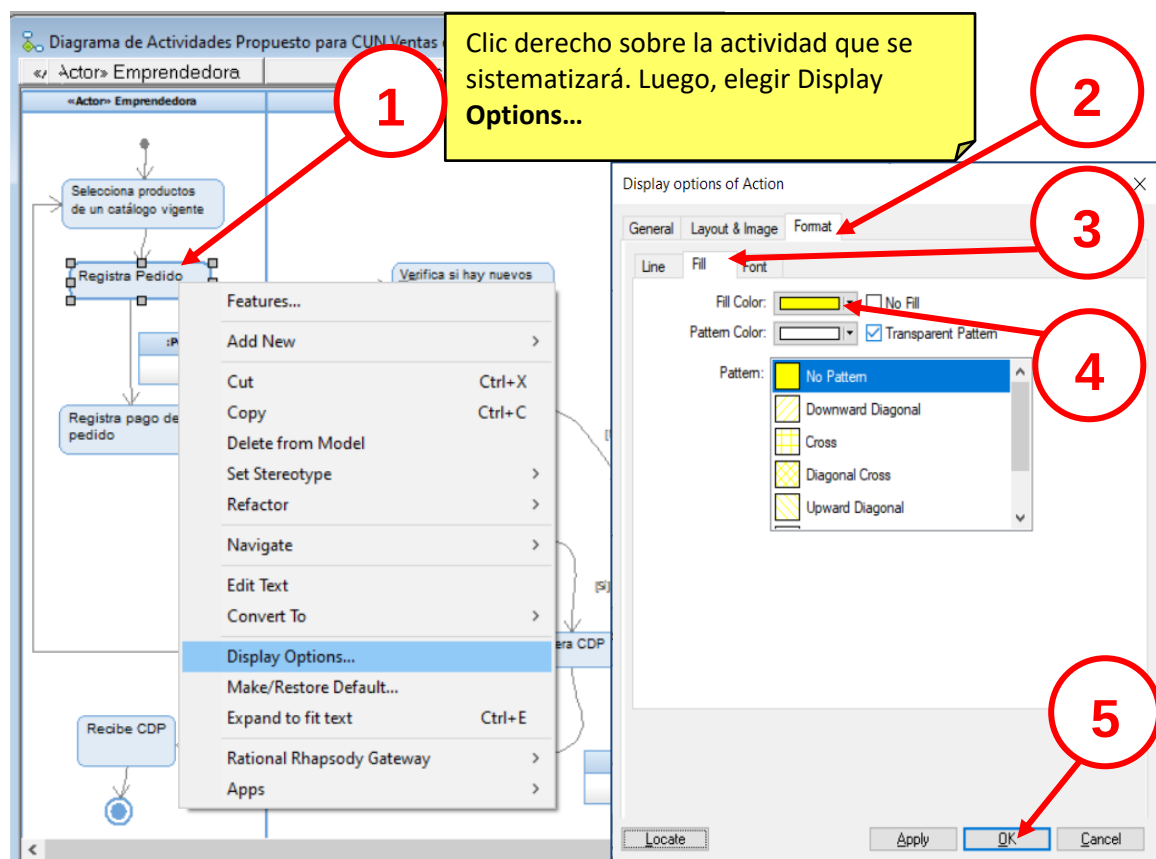


Figura 77: Actividades a sistematizar resaltadas con otro color
Fuente. - Elaboración propia

Paso 3: Finalmente se construye la matriz con las actividades a informatizar. Además, se identifican los requisitos, casos de uso, y actores.

Note que se ha agregado el RF003 “Notificar pedido pagado” que no tiene trazabilidad con alguna actividad en columna “Actividad de Negocio”, el cual puede darse en un proyecto real cuando el analista brinda una propuesta para dar mejor soporte y agilidad al proceso y que ayude la labor del AsesorVentas. Asimismo, se observa que los requisitos RF002 y RF003 derivan al Caso de Uso CU02 “Pagar pedido”.

Finalmente, observe que se ha agregado los requisitos RF006 y RF007 que son identificados en todo proyecto de sistema por temas de seguridad. Estos requisitos derivan a los casos de uso CU05 y CU06 que son funcionalidades que también los podemos notar en nuestra Intranet de la institución al ingresar como Docente o Alumno.

Matriz de Actividades y Requisitos del Sistema Fashion Importaciones							
Proceso de Negocio	Actividad del Negocio	Responsable del Negocio	Requisito		Caso de Uso		Actores
CUN 01 - Venta de productos	Registra pedido	Emprendedora	RF001	Generar pedido	CU01	Generar pedido	Emprendedora
	Registra pago de pedido	Emprendedora	RF002	Pagar pedido	CU02	Pagar pedido	Emprendedora
			RF003	Notificar pedido pagado			
	Verifica si hay nuevos pedidos pagados	AsesorVentas	RF004	Consultar pedidos pagados	CU03	Consultar pedidos pagados	Asesor de Ventas
	Genera CDP	AsesorVentas	RF005	Generar CDP	CU04	Generar CDP	Asesor de Ventas
			RF006	Ingresar al sistema	CU05	Ingresar al sistema	Usuario
			RF007	Cambiar contraseña	CU06	Cambiar contraseña	Usuario

Figura 78: Matriz de Actividades Vs. Requisitos del caso Fashion Importaciones

Fuente. - Elaboración propia

Resumen

1. Es muy importante involucrar a los stakeholders claves para la captura de requisitos del sistema. En este sentido, debemos coordinar con ellos concientizándolos sobre la importancia de la captura de requisitos:
 - Permite estimar costos, tiempos y recursos
 - Disminuye los costos y retrasos del proyecto
 - Mejora la comunicación entre clientes y desarrolladores
 - Evita rechazos de los usuarios finales
2. Los artefactos de la captura de requisitos sirven como entrada y referencia para el análisis, diseño, implementación y pruebas de software.
3. Los artefactos de la captura de requisitos son los siguientes: Especificación de Software, requisitos funcionales, requisitos técnicos, casos de uso, actores, Matriz de Actividades Vs. Requisitos, y Matriz de Requisitos Vs. Casos de Uso.
4. El enunciado de un requisito funcional y requisito no funcional debe incluir un código o identificador, un nombre y una breve descripción o especificación.
5. Los requisitos FURPS+ hacen referencia a las siguientes categorías de requisitos:
 - Funcionalidad (Functionality)
 - Facilidad de Uso (Usability)
 - Confiabilidad (Reliability)
 - Rendimiento (Performance)
 - Soporte (Supportability)
 - El símbolo "+" incluye otros requisitos como: restricciones de diseño, requisitos de implementación, requisitos de interfaz, y requisitos físicos.
6. Los requisitos pueden obtenerse directamente de las necesidades o solicitudes de usuarios. En este caso, si se tiene claro el negocio o si solo se está agregando una nueva característica a un sistema existente, no es necesario realizar el modelado de negocio.
7. Asimismo, pueden identificarse requisitos funcionales a partir de los diagramas de actividades de cada proceso. Se utiliza una Matriz de Actividades Vs. Requisitos para documentar el seguimiento de actividades a informatizar, requisitos funcionales y casos de uso.

Recursos

Pueden revisar los siguientes enlaces para ampliar los conceptos vistos en este tema:

- <http://agilemodeling.com/essays/requirementsChallenges.htm#NoAccessToStakeholders>
- <https://www.thepowermba.com/es/marketing/disenio-ux-para-tu-web/>

3.2. MODELO DE CASOS DE USO

3.2.1. Artefactos del modelo de casos de uso

Existen 2 artefactos relevantes en el modelo de casos de uso: actores y casos de uso, los cuales fueron definidos en el apartado anterior. El objetivo de un modelo de casos de uso es mostrar la forma en que el sistema es usado por los usuarios. De modo que, se visualiza quién interactúa en el sistema (actores) y qué deberá hacer el sistema (casos de uso).

3.2.2. Reglas básicas de nomenclaturas

En la siguiente tabla se mencionan las reglas de nomenclaturas de un actor y un caso de uso.

Artefacto	Reglas de nomenclatura
 Actor	El nombre de un actor debe ser un rol. Por lo tanto, su nombre debe ser algún rol que proviene del modelado de negocio: de un actor de negocio o de un trabajador de negocio. También puede representar un sistema con el que se comunicará el sistema que se implementará. De modo que, su nombre puede ser el de un sistema. Finalmente, puede representar a un “Timer” cuando se ejecuta una funcionalidad definida por un tiempo específico.
 Caso de Uso	En general, debe empezar con un verbo en infinitivo a continuación el dato que manipula. Para mantenimientos de datos (INSERT, DELETE, UPDATE), dependiendo del tipo de dato, se utilizará el siguiente nombre. Si es un dato maestro: “Mantener <dato maestro>”, por ejemplo, “Mantener clientes”. Si es un dato transaccional: “Administrar/Gestionar/Evaluar <dato transaccional>”, por ejemplo, “Gestionar cotizaciones”. Para ingresar datos (INSERT), se tiene dos situaciones. Si es documento Interno para el CUN y el Actor es el responsable de la creación del dato transaccional, se utiliza “Generar <dato>”. Si el documento externo para el CUN o el Actor no es el responsable directo de la creación del dato transaccional se utiliza “Registrar <dato>”. Para consultar datos (SELECT). Si el resultado de la consulta es cargado a otro caso de uso. Además, SIEMPRE presenta criterios de búsqueda, se utiliza “Buscar <dato>”. Si el resultado de la consulta NO es cargado a otro caso de uso. Es opcional los criterios de búsqueda; se utiliza “Consultar <dato>”.

Figura 79: Nomenclaturas de artefactos del Modelo de Casos de Uso

Fuente. - Elaboración propia

3.2.3. Identificación de artefactos del modelo de casos de uso para el caso fashion importaciones

En esta sección, seguiremos analizando el caso Fashion Importaciones descrito en la Unidad 2, sección Modelo de Casos de Uso de Negocio (pág 75). Para efectos de ejemplo, se identificará los requisitos funcionales y casos de uso de uno de los procesos: Venta de productos.

Paso 1: Agregue los paquetes: PaqueteRequisitos y MCU (Modelo de Casos de Uso) en el fólдер “Packages”.

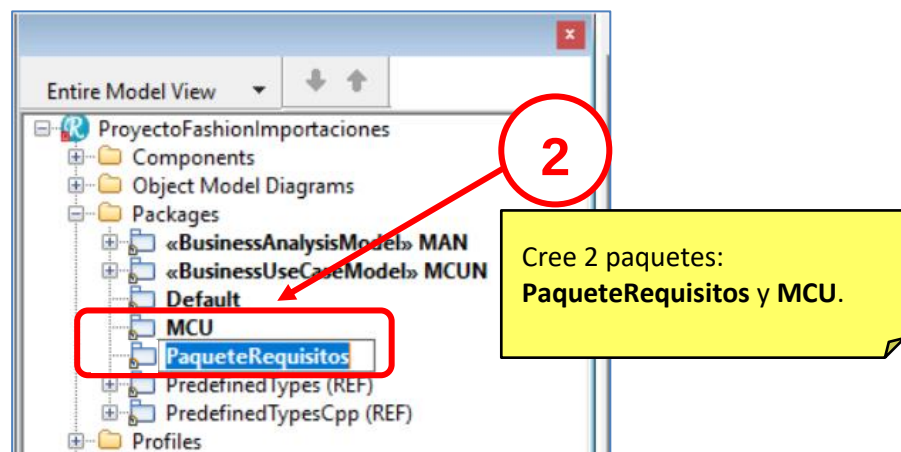
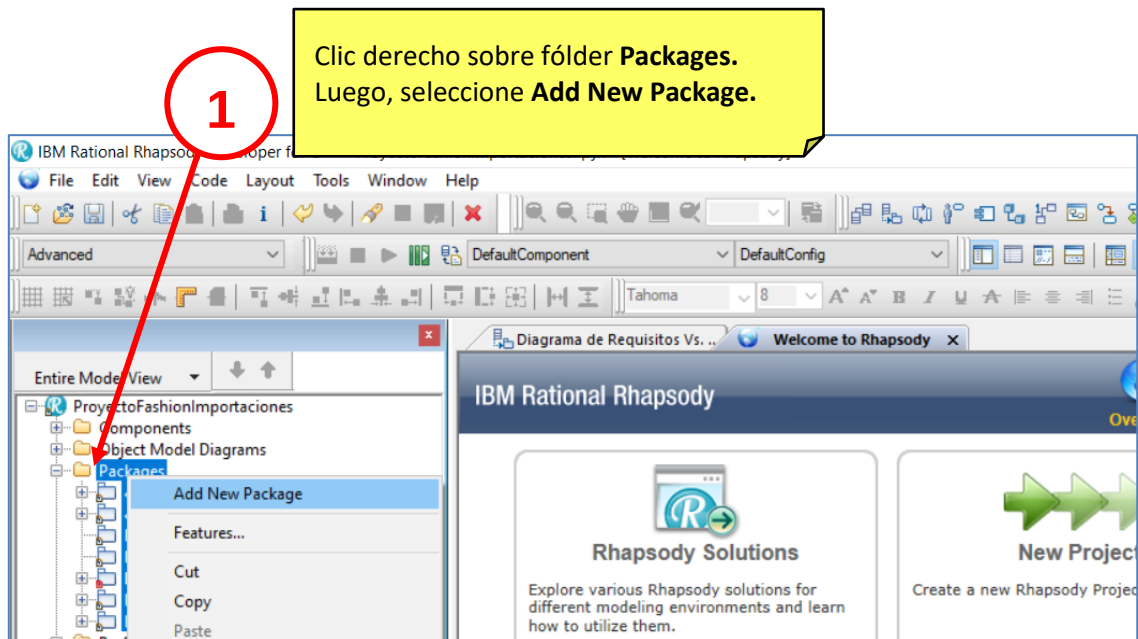


Figura 80: Creación de los paquetes PaqueteRequisitos y MCU
Fuente. - Elaboración propia

Paso 2: Ahora, cree los requisitos funcionales que identificó en la matriz de Actividades Vs. Requisitos, en la sección anterior.

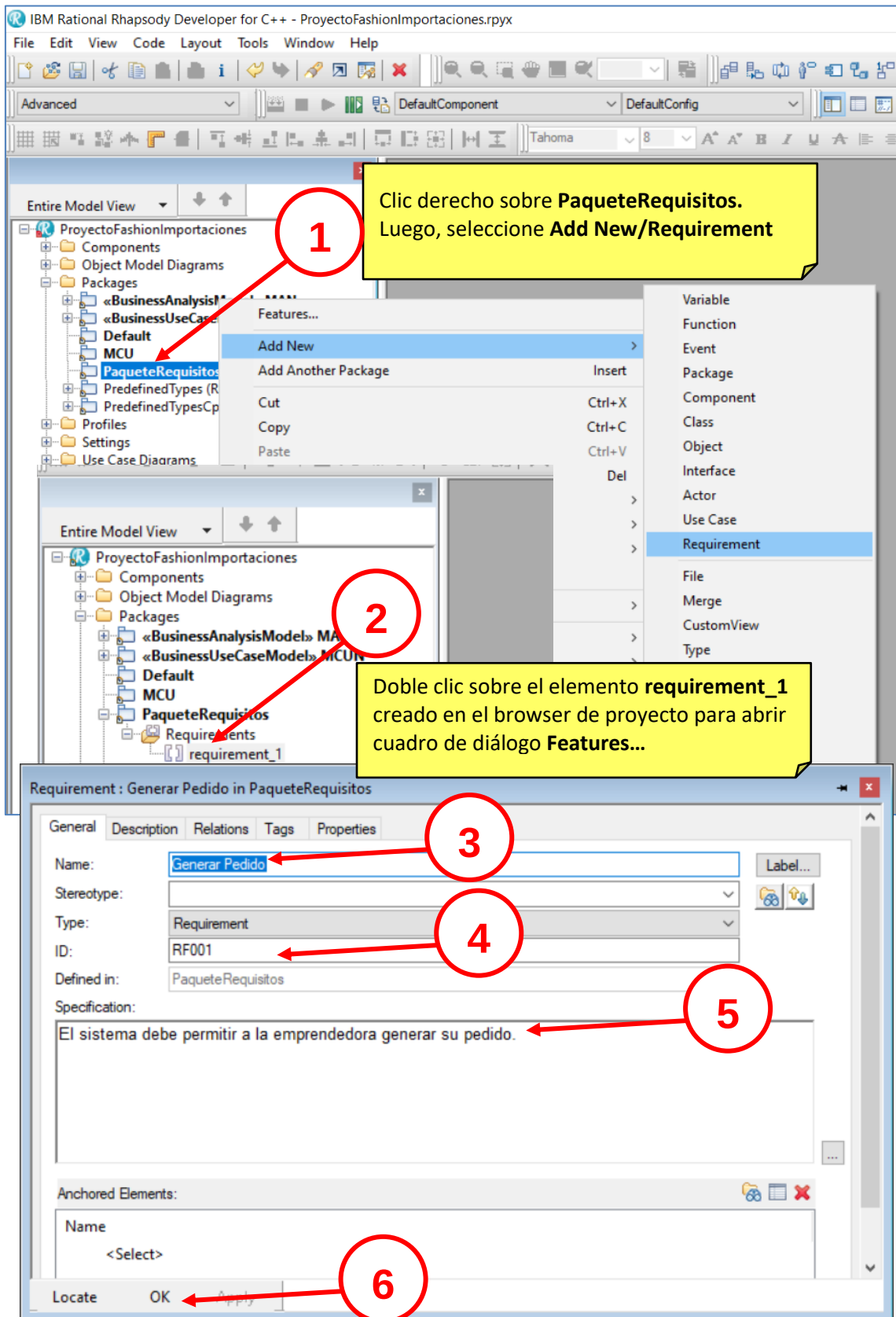


Figura 81: Creación de los requisitos
Fuente. - Elaboración propia

Paso 3: A continuación, agregue los otros requisitos con sus respectivos nombres, ID y especificación:

The figure shows four overlapping screenshots of the 'Requirement' dialog box in a software tool, illustrating the creation of four requirements for the 'Fashion Importaciones' case. Each dialog box has tabs for 'General', 'Description', 'Relations', 'Tags', and 'Properties'. The 'General' tab is active, showing fields for Name, Stereotype, Type, ID, Defined in, and Specification.

- Requirement 1: Pagar Pedido**
 - Name: Pagar Pedido
 - Stereotype: (empty)
 - Type: Requirement
 - ID: RF002
 - Defined in: PaqueteRequisitos
 - Specification: El sistema debe permitir a la emprendedora pagar su pedido.
- Requirement 2: Notificar pedido pagado**
 - Name: Notificar pedido pagado
 - Stereotype: (empty)
 - Type: Requirement
 - ID: RF003
 - Defined in: PaqueteRequisitos
 - Specification: El sistema debe notificar a asesora de ventas que hay un pedido pagado.
- Requirement 3: Consultar pedidos pagados**
 - Name: Consultar pedidos pagados
 - Stereotype: (empty)
 - Type: Requirement
 - ID: RF004
 - Defined in: PaqueteRequisitos
 - Specification: El sistema debe permitir a la asesora de ventas consultar pedidos pagados.
- Requirement 4: Generar CDP**
 - Name: Generar CDP
 - Stereotype: (empty)
 - Type: Requirement
 - ID: RF005
 - Defined in: PaqueteRequisitos
 - Specification: El sistema debe permitir a la asesora de ventas generar CDP.

At the bottom of the dialog boxes, there are buttons for 'Locate', 'OK', and 'Apply'.

Figura 82: Creación de los requisitos del caso Fashion Importaciones

Fuente. - Elaboración propia

Paso 4: Luego, cree un Diagrama de Modelo de Objetos “Diagrama de Requisitos Vs. Casos de Uso” y agregue los requisitos creados.

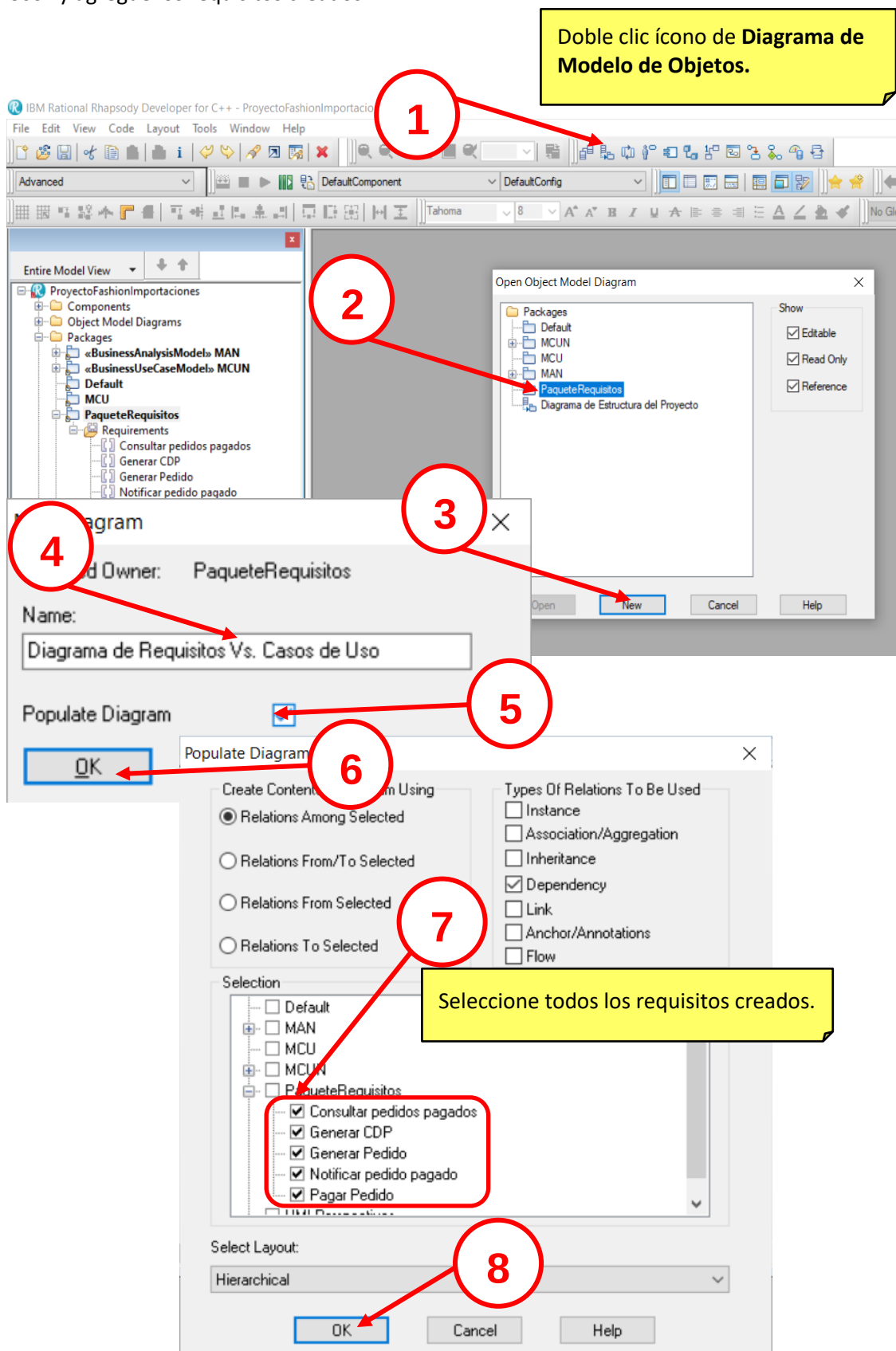


Figura 83: Creación del Diagrama de Requisitos Vs. Casos de uso

Fuente. - Elaboración propia

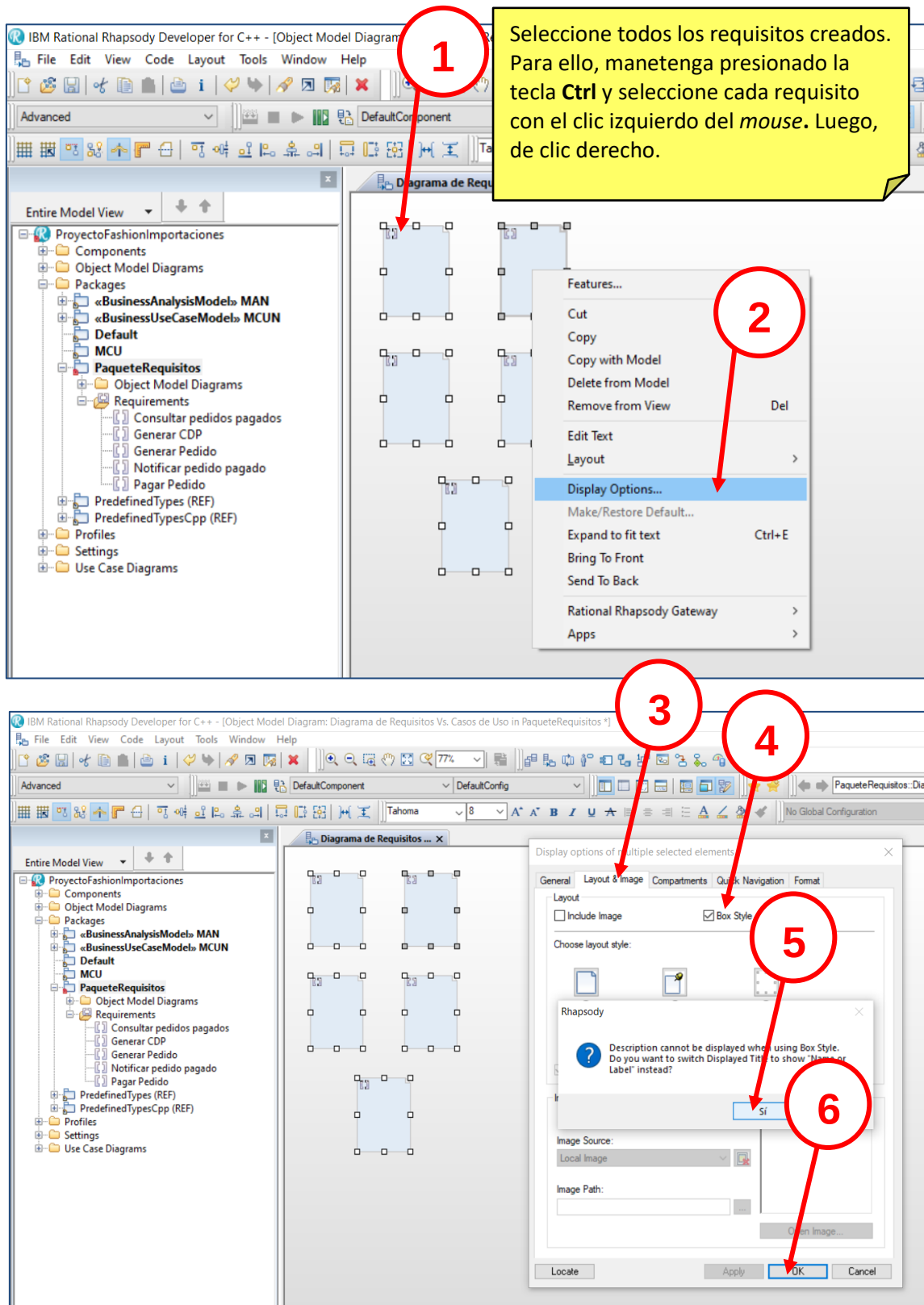
Paso 5: Cambiar opciones de “Display Options...” de los requisitos.

Figura 84: Configuración del **Display Options** de los requisitos
Fuente. - Elaboración propia

Paso 6: Ordenar los requisitos del Diagrama.

1

Seleccione todos los requisitos. Para ello, mantenga presionado la tecla **Ctrl** y seleccione cada requisito con el clic izquierdo del *mouse*. Luego, cambie de tamaño, seleccionando los cuadros de la selección que están con líneas entrecortadas, tanto en ancho como en alto apropiados para visualizar contenido.

2

Ahora, visualizará el nombre, ID y especificación de cada requisito.

The screenshot shows the IBM Rational Rhapsody interface. On the left, the 'Entire Model View' tree shows the project structure, including 'ProyectoFashionImportaciones', 'Components', 'Object Model Diagrams', 'Packages', and 'PaqueteRequisitos'. The 'Diagrama de Requisitos Vs. Casos de Uso' diagram is displayed in the center. It contains five requirements, each in a box with a red header and a blue body. The requirements are: 'Generar Pedido' (ID RF001), 'Pagar Pedido' (ID RF002), 'Notificar pedido pagado' (ID RF003), 'Consultar pedidos pagados' (ID RF004), and 'Generar CDP' (ID RF005). The diagram is titled 'Diagrama de Requisitos Vs. Casos de Uso' and is part of the 'PaqueteRequisitos' package.

Figura 85: Requisitos Funcionales del caso Fashion Importaciones en el Diagrama de Requisitos Vs. Casos de Uso.

Fuente. - Elaboración propia

Paso 6: A continuación, cree el Diagrama General de Casos de Uso y agregue los casos de uso con sus respectivos actores que se identificó en la Matriz de Actividades Vs. Requisitos.

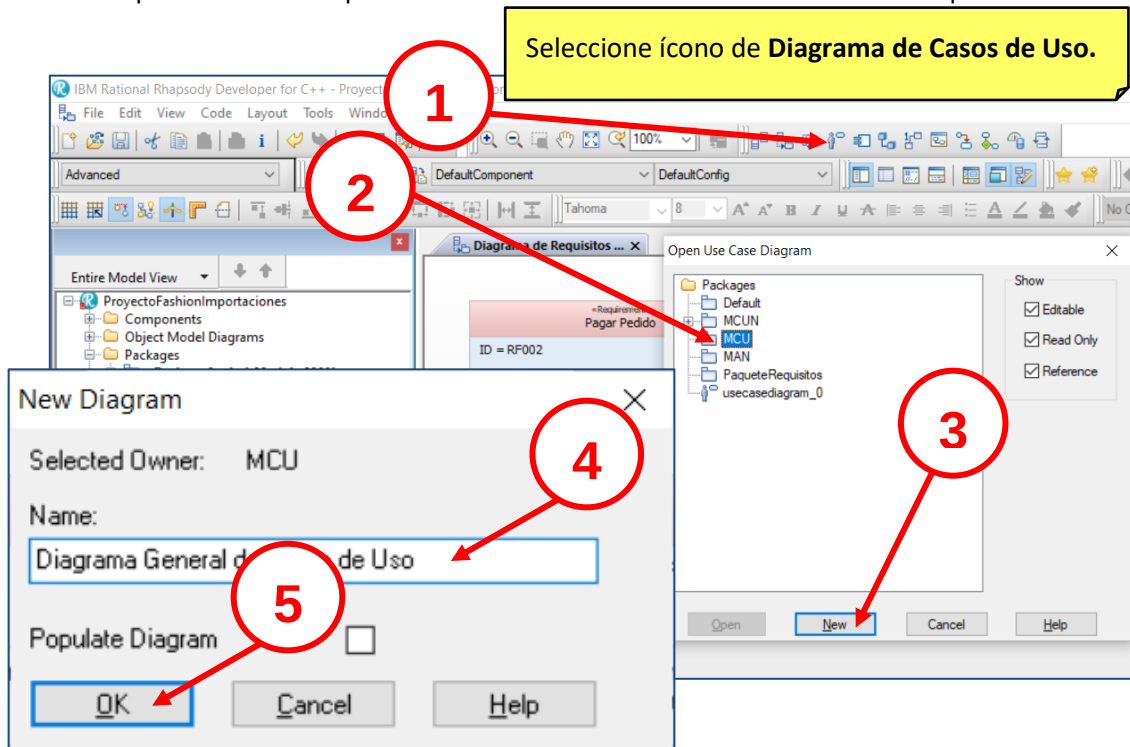


Figura 86: Creación del Diagrama General de Casos de Uso
Fuente. - Elaboración propia

Paso 7: Sobre el diagrama creado, agregue los casos de uso con sus respectivos actores que se identificó en la Matriz de Actividades Vs. requisitos. Para ello, utilice los siguientes elementos de la barra de herramientas: Use Case, Actor y Association.

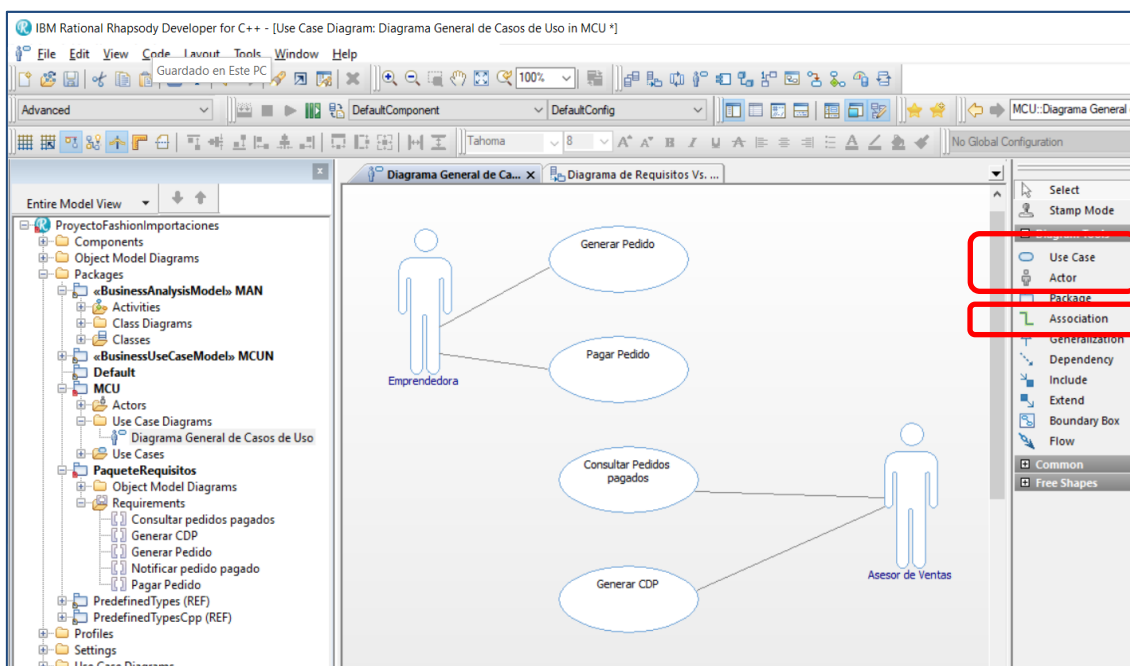


Figura 87: Creación de los casos de uso y actores en el MCU
Fuente. - Elaboración propia

Paso 8: Luego, arrastre los casos de uso sobre el Diagrama de Requisitos Vs. Casos de Uso.

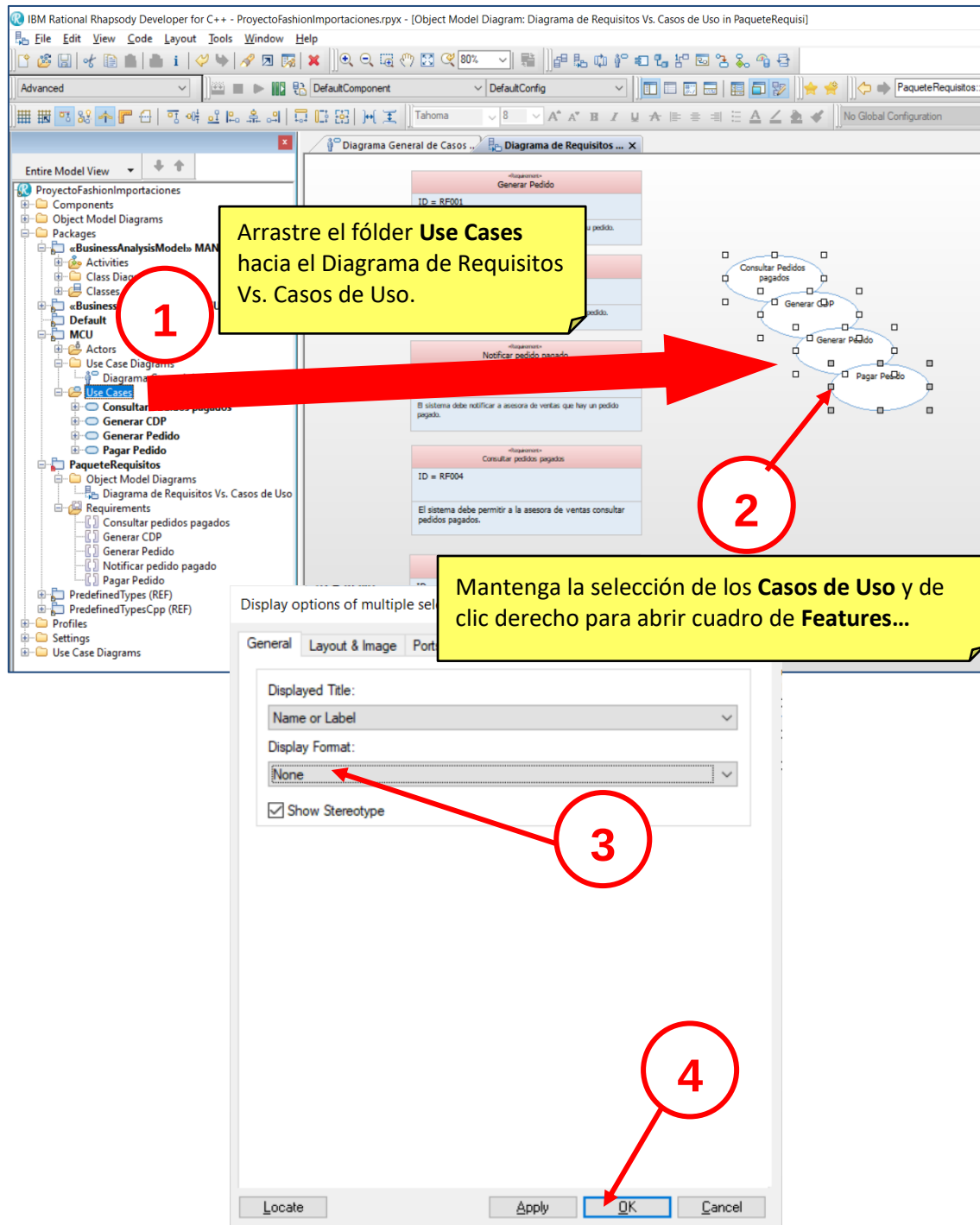


Figura 88: Casos de Uso en el Diagrama de Requisitos Vs. Casos de Uso

Fuente. - Elaboración propia

Paso 9: Agregue las relaciones de dependencia con estereotipo “Derive” desde el requisito al caso de uso, así como se muestra a continuación.

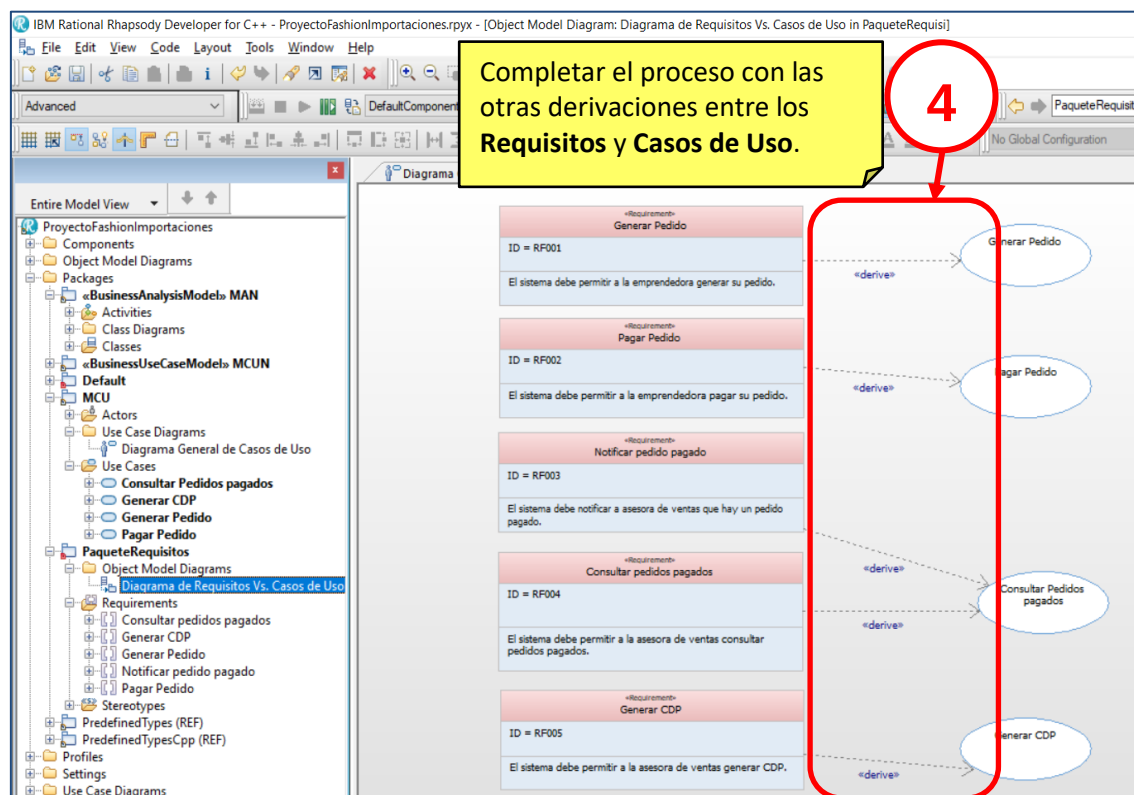
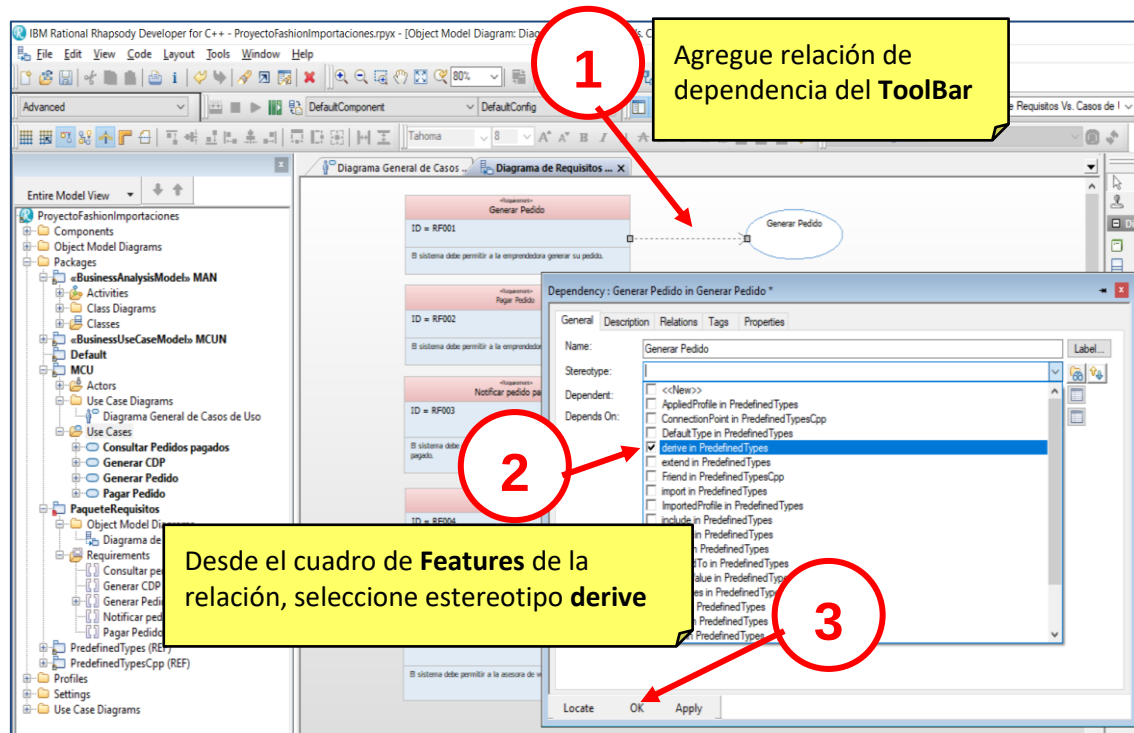


Figura 89: Diagrama de Requisitos Funcionales Vs. Casos de Uso del caso Fashion Importaciones

Fuente: - Elaboración propia

Paso 10: A continuación, puede crear una matriz para visualizar la trazabilidad entre requisitos funcionales y casos de uso. Primero, se crea un Layout de Matriz:

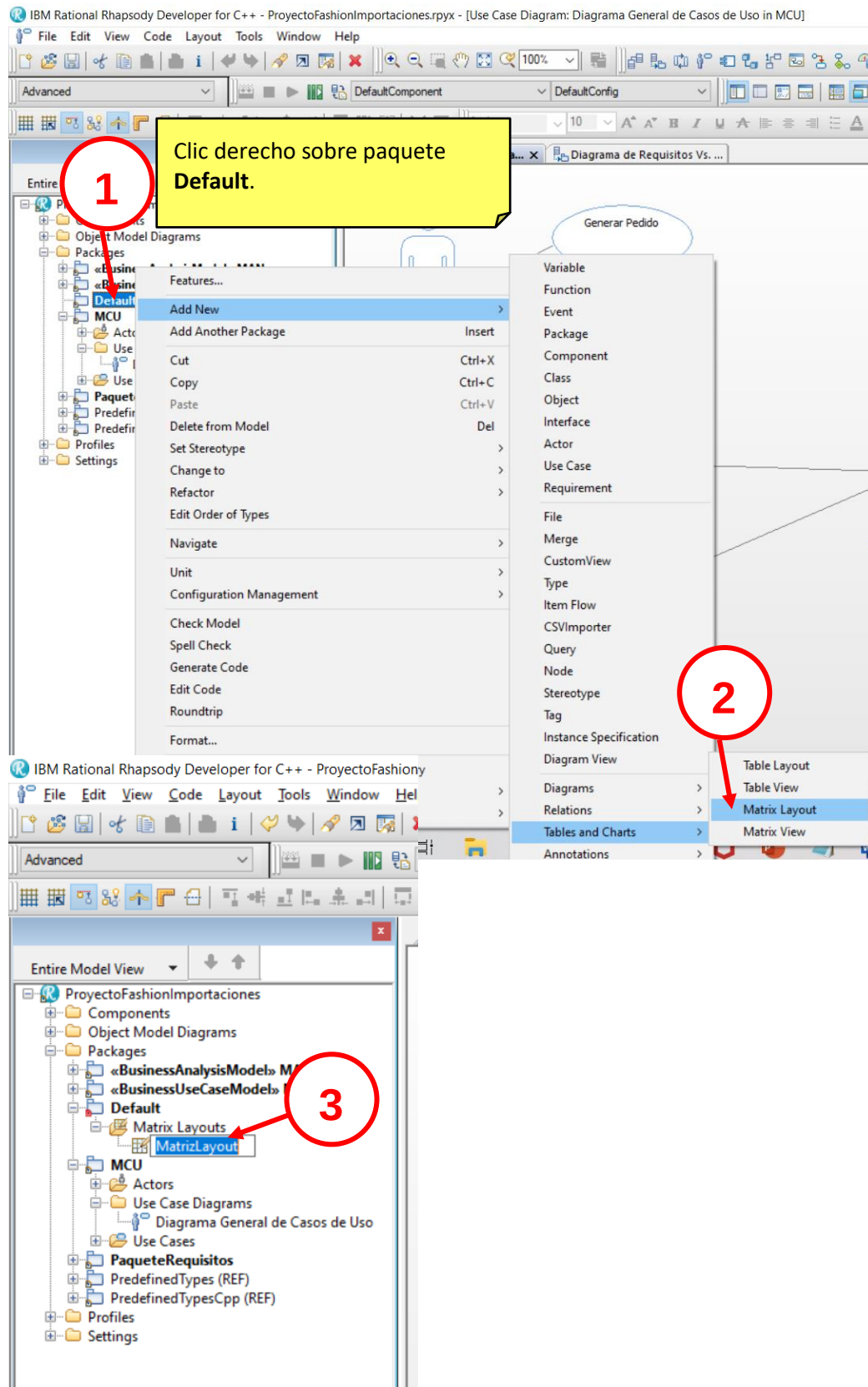


Figura 90: Creación del Layout de la Matriz Requisitos Vs. Casos de Uso

Fuente. - Elaboración propia

Paso 11: Luego, se configura el Layout de matriz creado en el paso anterior. Para ello, se definen los tipos de componentes que se mostrarán en la matriz: Requirement y Use Case.

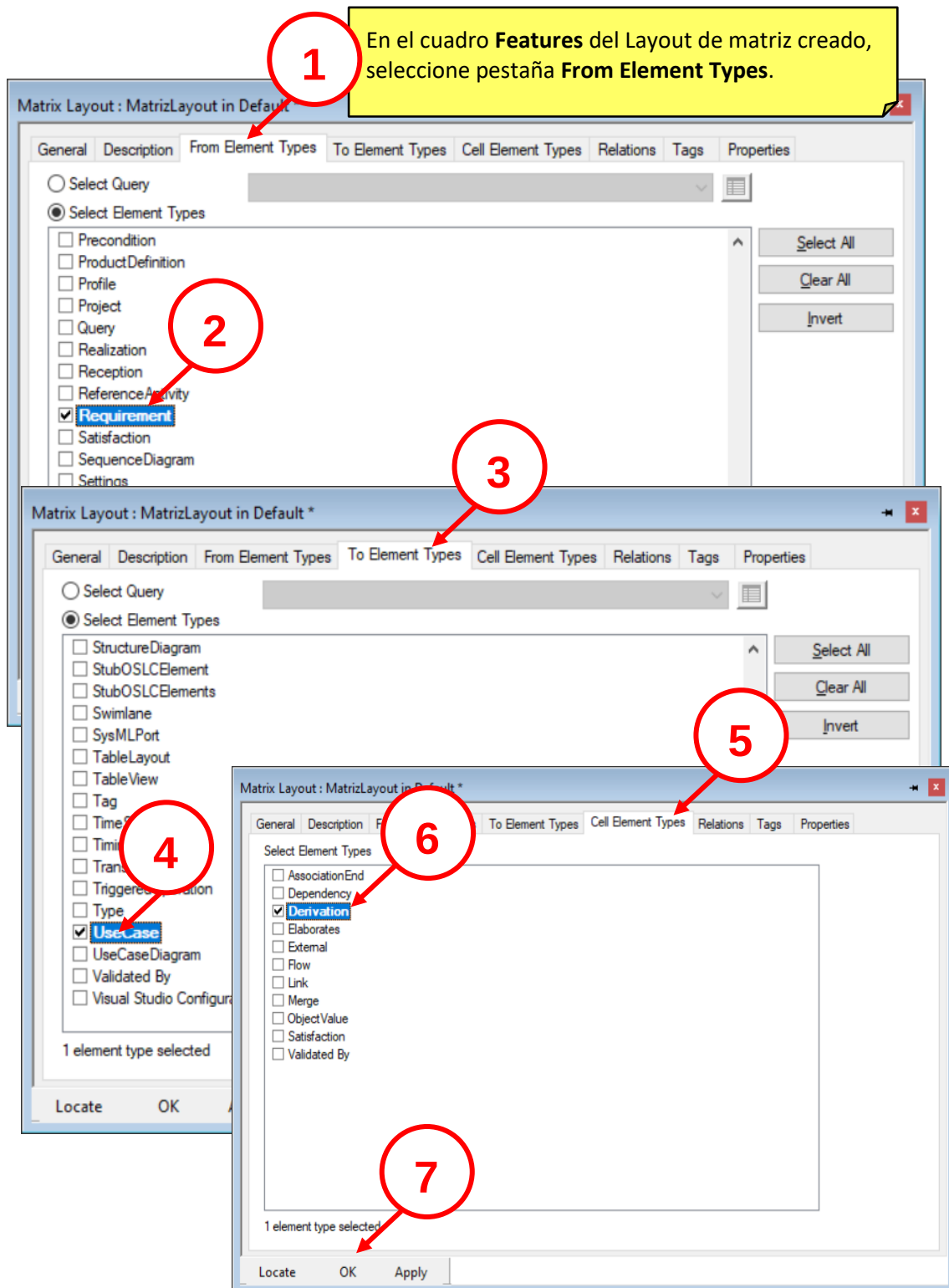


Figura 91: Configuración del Layout de la matriz Requisitos Vs. Casos de Uso
Fuente. - Elaboración propia

Paso 12: Ahora, se crea la vista de la Matriz Requisitos Vs. Casos de Uso.

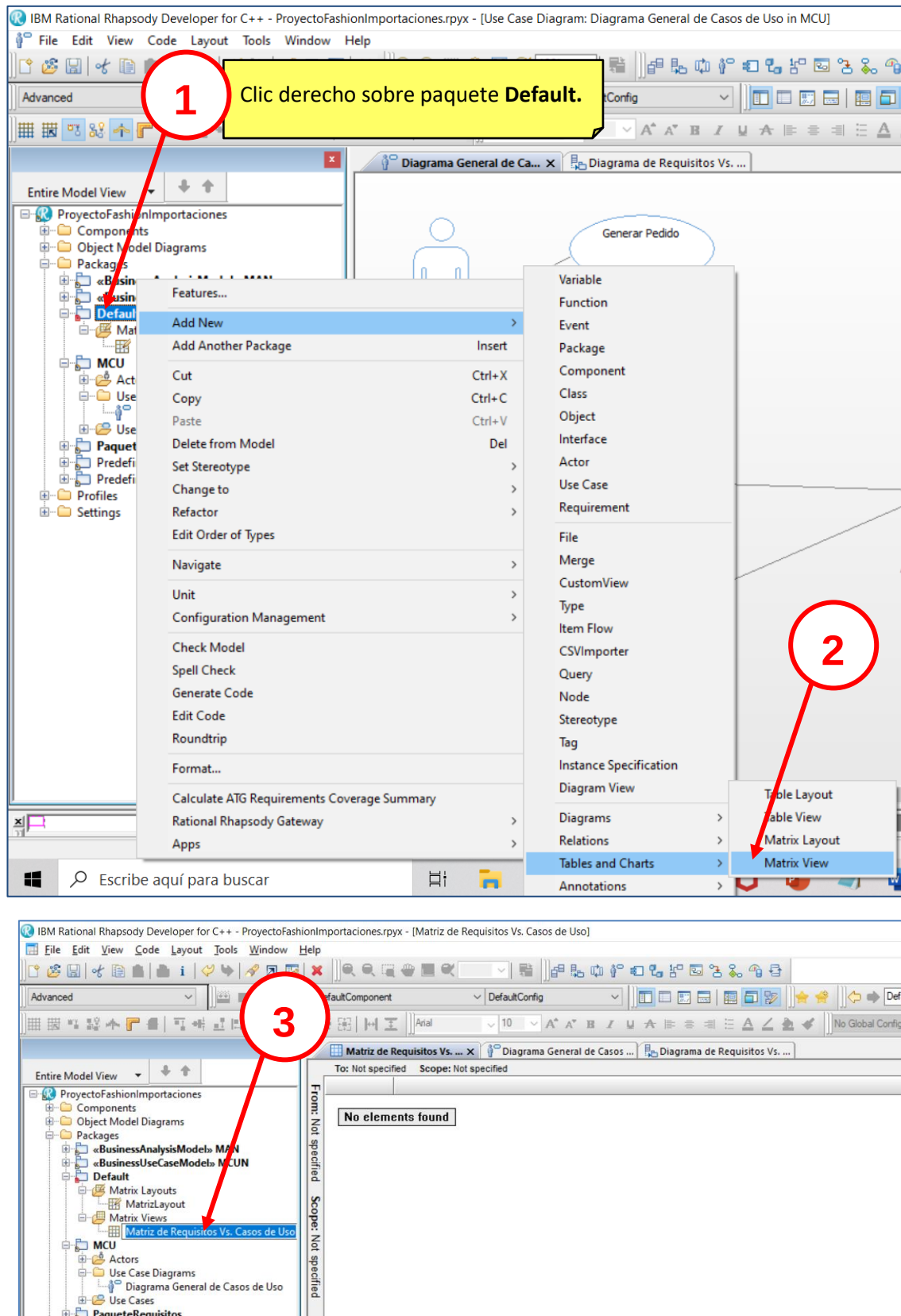


Figura 92: Creación de la vista de Matriz de Requisitos Vs. Casos de Uso

Fuente. - Elaboración propia

Paso 13: Luego, se configura la vista de la Matriz de Requisitos Vs. Casos de Uso.

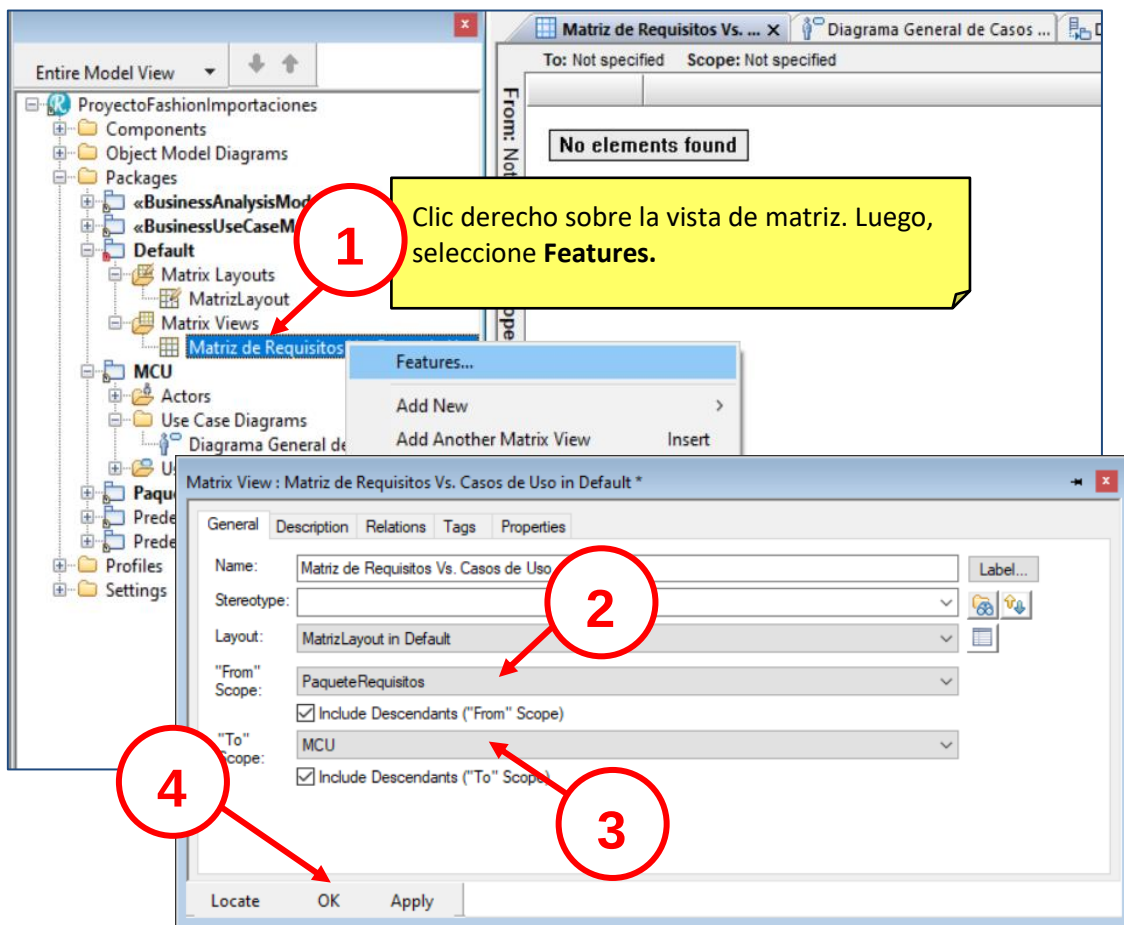


Figura 93: Configuración de la vista de Matriz de Requisitos Vs. Casos de Uso
Fuente. - Elaboración propia

Paso 14: Para visualizar la matriz, refresque (**Refresh**) los cambios realizados en **Features**.

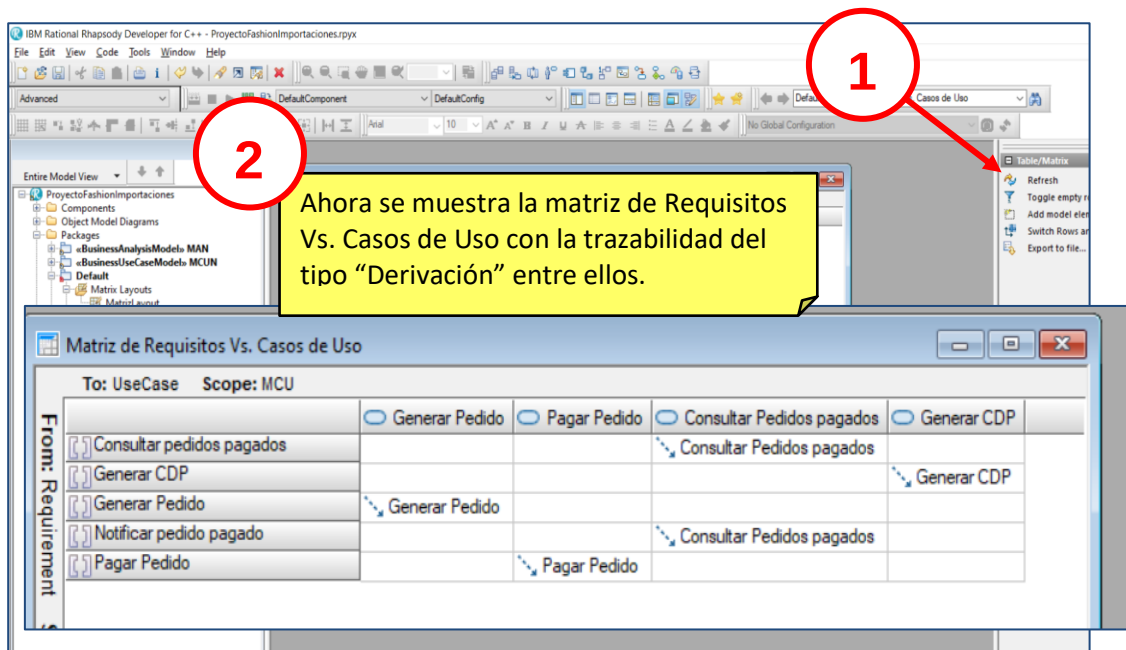


Figura 94: Vista de la Matriz de Requisitos Vs. Casos de Uso
Fuente. - Elaboración propia

Paso 15: Además, puede organizar los Casos de Uso en paquetes. Para ello, agregue el paquete “Reutilizable” (donde se ubicarán los casos de uso que se reutilizarán en más de un paquete), y por cada CUN, cree un paquete. Para el caso que estamos desarrollando crearemos el paquete “VentaProductos”. Asimismo, considere agregar casos de uso que dan soporte a la Seguridad del sistema en el paquete “Seguridad”.

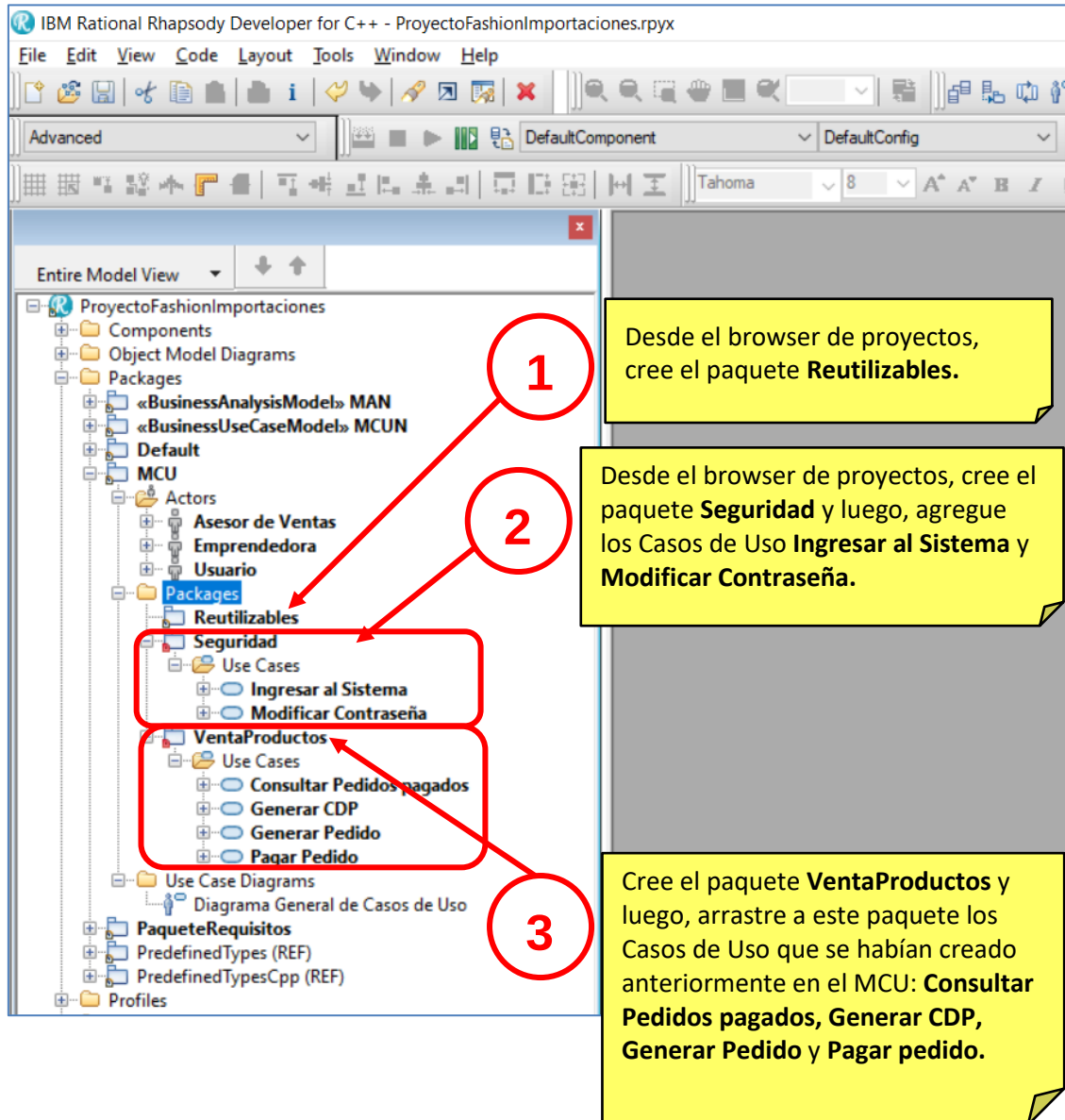


Figura 95: Paquetes Reutilizables, Seguridad y VentaProductos

Fuente. - Elaboración propia

Paso 16: A continuación, agregue el Diagrama de Casos de Uso por paquete. Empiece con Seguridad.

The screenshot illustrates the process of creating a Use Case Diagram for the Security package in IBM Rational Rhapsody Developer for C++.

- 1**: Click the 'New' button in the 'Open Use Case Diagram' dialog.
- 2**: Select the 'Seguridad' package in the 'Open Use Case Diagram' dialog's package tree.
- 3**: Click the 'New' button in the 'Open Use Case Diagram' dialog.
- 4**: Click the 'OK' button in the 'New Diagram' dialog.
- 5**: Enter the name 'Diagrama de Casos de Uso del Paquete Seguridad' in the 'Name' field of the 'New Diagram' dialog.
- 6**: Check the 'Populate Diagram' checkbox in the 'New Diagram' dialog.
- 7**: Select the 'Ingresar al Sistema' and 'Modificar Contraseña' use cases in the 'Selection' list of the 'Populate Diagram' dialog.
- 8**: The final diagram is created, showing the 'Seguridad' package containing the 'Ingresar al Sistema' and 'Modificar Contraseña' use cases.

Seleccione los Casos de Uso que se agregarán al Diagrama de Casos de Uso del Paquete Seguridad.

Se habrá creado el Diagrama de Casos de Uso del Paquete Seguridad. Repita los pasos del 1 al 7 para crear Diagrama de paquete de VentaProductos.

Figura 96: Diagrama de Casos de Uso por paquete

Fuente. - Elaboración propia

Resumen

1. El Modelo de Casos de Uso permite representar las funcionalidades del sistema a implementar.
2. Existe 2 artefactos relevantes en el modelo de casos de uso: actores y casos de uso. Con ellos, se visualiza quién interactúa en el sistema (actores) y qué deberá hacer el sistema (casos de uso).
3. El nombre de un actor debe ser un rol que participa en un proceso. También puede representar un sistema con el que se comunicará el sistema que se implementará. Finalmente, puede representar a un “Timer” cuando se ejecuta una funcionalidad definida por un tiempo específico.
4. El nombre de un caso de uso debe empezar con un verbo en infinitivo, a continuación, el dato que manipula. Ejemplo: “Mantener productos”. “Gestionar cotizaciones”, “Evaluar desempeño de colaborador”, “Registrar incidencia”, “Generar CDP”, “Buscar cliente”, y “Consultar pedidos pagados”.

Recursos

Pueden revisar los siguientes enlaces para ampliar los conceptos vistos en este tema:

- https://cgrw01.cgr.go.cr/rup/RUP.es/SmallProjects/core.base_rup/workproducts/rup_use_case_model_EF15E534.html

3.3. ESTRUCTURANDO EL MODELO DE CASOS DE USO

Esta actividad se centra en relacionar los casos de uso y los actores del sistema, e identificar sus comportamientos opcionales y excepcionales. Se establece las inclusiones, extensiones y generalizaciones entre casos de uso, y las generalizaciones entre actores.

Existen tres razones para estructurar el modelo de casos de uso:

- Hacer que los casos de uso sean fáciles de entender.
- Extraer el comportamiento común encontrado en varios casos de uso.
- Hacer que el modelo de casos de uso sea fácil de mantener.

3.3.1. Objetivos

Los objetivos de esta actividad son:

- Extraer descripciones de funcionalidad (de casos de uso) generales y compartidas que pueden ser utilizadas por casos de uso más específicos (generalización).
- Extraer descripciones de funcionalidad (de casos de uso) adicionales u opcionales que pueden extender casos de uso más específicos (relaciones de extensión).
- Extraer descripciones de funcionalidad (de casos de uso) adicionales e incondicionales incluidas en la ejecución de casos de uso específicos (relaciones de inclusión).

3.3.2. Tipos de relaciones

Existen tres tipos de relaciones entre casos de uso: incluye, extend y de generalización. Entre actores se puede utilizar solo la relación de generalización.

3.3.2.1. Relación “include”

Una relación incluye se define como la utilización de los pasos de un caso de uso como parte de la secuencia de otro caso de uso al que se llamará caso de uso base. El caso de uso incluido nunca se encuentra aislado, sino que es instanciado sólo como parte de algún caso de uso base que lo incluye. Esta relación se usa para evitar describir el mismo flujo de eventos repetidas veces, poniendo el comportamiento común en un caso de uso aparte y que será incluido por un caso de uso base. Su representación gráfica con es la siguiente:

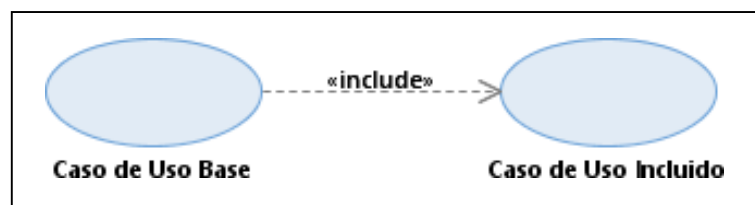


Figura 97: Relación “include” entre casos de uso
Fuente. - Elaboración propia

Para entender la ejecución de un caso de uso incluido, analice la siguiente figura. Puede observar que el comportamiento del caso de uso incluido se inserta en un punto del caso de uso base. Cuando una instancia de caso de uso, el cual sigue la descripción de un caso de uso base, llega a un punto en donde la relación incluye es definida, seguirá la descripción del caso de uso incluido.

Una vez que la inclusión se lleva a cabo, la instancia del caso de uso regresará al caso de uso base, en el punto donde se detuvo.

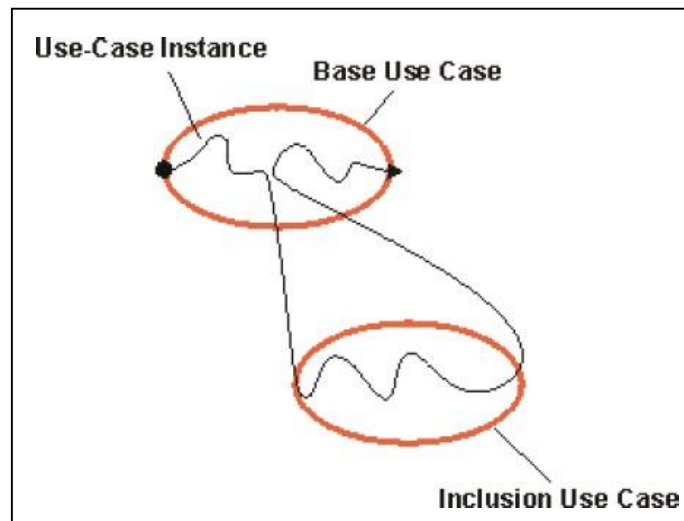


Figura 98: Ejecución de la relación de inclusión

Fuente. – Tomado de

https://cgrw01.cgr.go.cr/rup/RUP.es/LargeProjects/core.base_rup/guidances/guidelines/resources/include2.gif

3.3.2.2. Relación “extend”

Una relación extend se define como la agregación de pasos a la secuencia del caso de uso original, que pasará a conocerse como caso de uso base. Esta extensión se realiza en puntos indicados, llamados puntos de extensión, de manera específica dentro de la secuencia del caso de uso base. El caso de uso puede estar aislado, pero, en algunas condiciones, su comportamiento puede extenderse con el comportamiento de otro caso de uso.

Esta relación se utiliza para modelar la parte de un caso de uso que el usuario puede ver como comportamiento opcional del sistema. De esta forma, se separa el comportamiento opcional del obligatorio. Su representación gráfica es la siguiente:

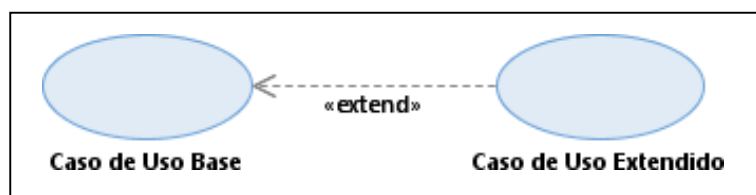


Figura 99: Relación “extend” entre casos de uso

Fuente. - Elaboración propia

La siguiente figura ilustra la ejecución de un caso extendido. Note que cuando una instancia del caso de uso base, llega a un lugar en donde un punto de extensión se ha definido, la condición en la correspondiente relación extend es evaluada. Si la condición es verdadera, la instancia del caso de uso seguirá la extensión; de lo contrario, la extensión no se ejecuta.

Una vez que la instancia de caso de uso ha realizado la extensión, la instancia de caso de uso reanuda su ejecución al caso de uso base, en el punto donde se detuvo.

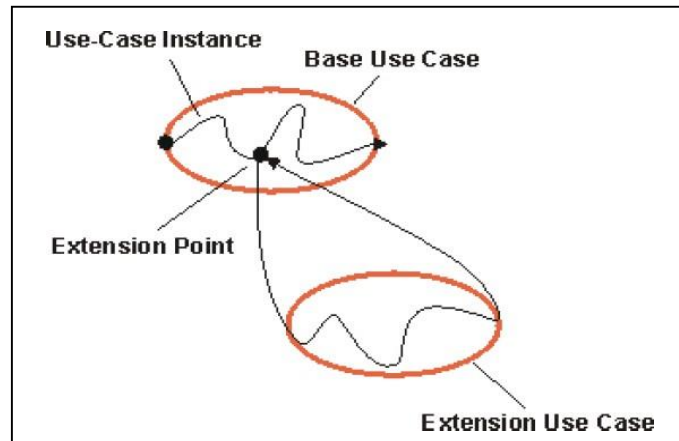


Figura 100: Ejecución de la relación de extensión

Fuente. – Tomado de

https://cgrw01.cgr.go.cr/rup/RUP.es/LargeProjects/core.base_rup/guidances/guidelines/re|sources/extend2.gif

3.3.2.3. Relación de “generalización”

La generalización entre casos de uso es como la generalización entre clases. En este caso significa que el caso de uso hijo hereda el comportamiento y el significado del caso de uso padre; el hijo puede añadir o redefinir el comportamiento del padre. La relación de generalización puede representarse también entre actores. Su representación gráfica es la siguiente:

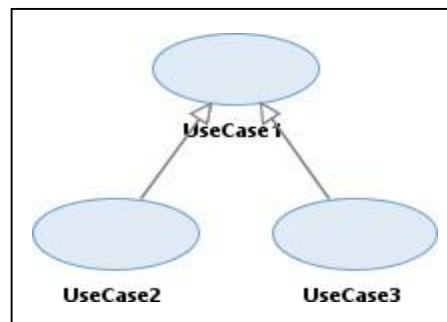


Figura 101: Relación de “generalización” entre casos de uso

Fuente. - Elaboración propia

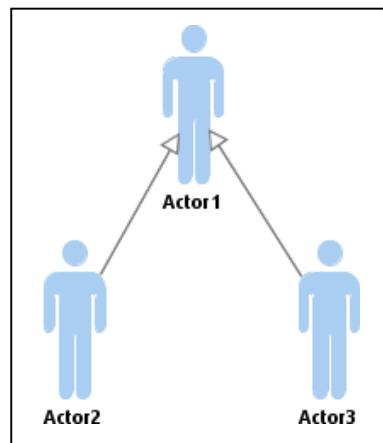


Figura 102: Relación de “generalización” entre casos de uso

Fuente. - Elaboración propia

Una instancia de caso de uso ejecutada por el caso de uso hijo seguirá el flujo de eventos descritos por el caso de uso padre, insertando comportamiento adicional y modificando el comportamiento, tal como se define en el flujo de eventos del caso de uso hijo.

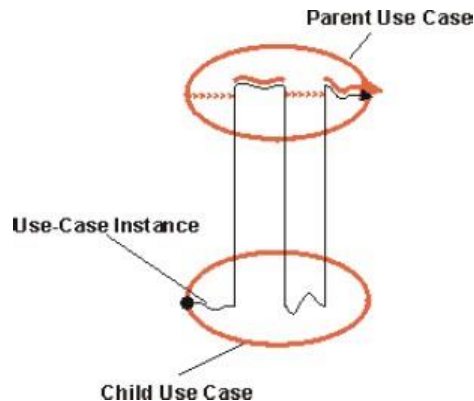


Figura 103: Ejecución de la relación de generalización

Fuente. – Tomado de

https://cgrw01.cgr.go.cr/rup/RUP.es/LargeProjects/core.base_rup/guidances/guidelines/resources/ucgen3.gif

3.3.3. Casos de uso abstractos y concretos

Se dice que un caso de uso es abstracto solo si se instancia en el contexto de otro caso de uso; es decir, dependen de otro caso de uso para instanciarse puesto que no existe un actor que lo active. Un caso de uso es concreto si es iniciado por un actor y constituye un completo flujo de eventos. "Completo" significa que una instancia del caso de uso lleva a cabo toda la operación solicitada por el actor.

Por lo tanto, se puede afirmar que:

- Los casos de uso activados por un actor son concretos.
- Los casos de uso incluidos o extendidos que únicamente pueden instanciarse cuando son llamados por el caso de uso base son casos de uso abstractos.
- En el caso de la generalización, generalmente el caso de uso padre será abstracto debido a que no están definidos completamente, pues los casos de uso hijos contienen las funciones específicas requeridas por los actores.

La siguiente figura ilustra ejemplos de casos de uso abstractos y concretos:

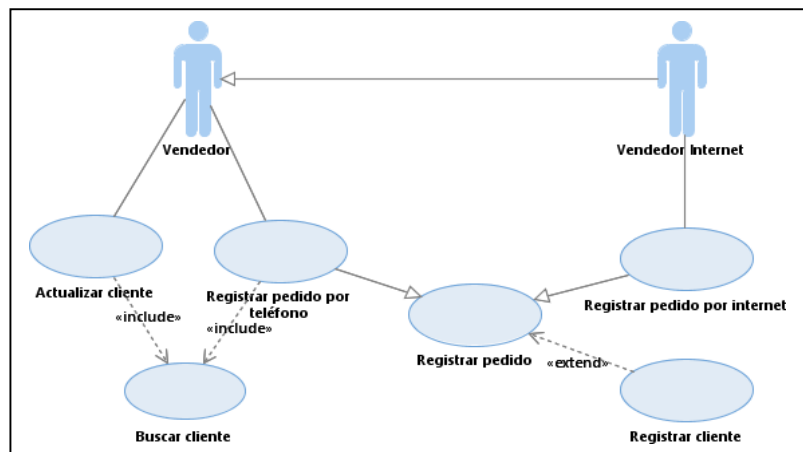


Figura 104: Diagrama de Casos de Uso estructurado

Fuente. - Elaboración propia

3.3.4. Especificación de casos de uso - ECU

No existe estándar UML para una especificación de caso de uso. Sin embargo, una plantilla para una especificación sencilla de caso de uso utilizada comúnmente contiene la siguiente información:

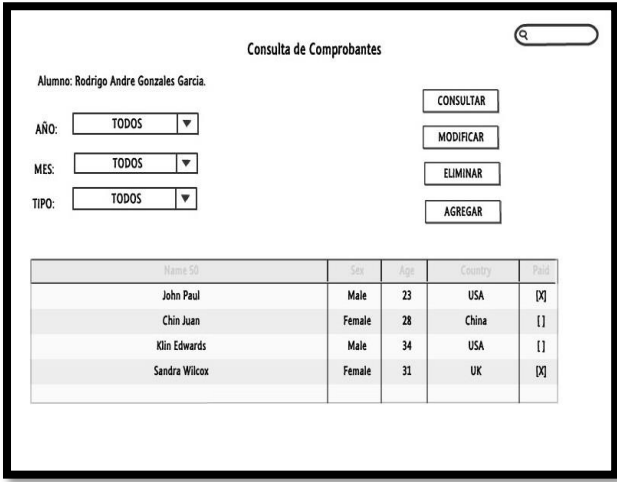
ESPECIFICACIÓN DE CASOS DE USO	
Nombre	<i>Indicar el nombre y codificación del CU</i>
Actores	<i>Indicar el actor(es) relacionados a este CU</i>
Propósito	<i>Indicar el propósito de CU</i>
Breve descripción	<i>Breve descripción</i>
Precondición	<i>Indicar las condiciones que deben cumplirse antes de iniciar el CU</i>
Poscondición	<i>Indicar las condiciones que se han cumplido luego de ejecutar el CU</i>
Evento disparador	<i>Indicar el evento que inicia el CU. Ejemplo: El caso de uso inicia cuando el Recepcionista pulsa el botón "Registrar Solicitud"</i>
Flujo Básico	<i>Indicar la secuencia de actividades que siguen al evento Disparador</i>
Sub-Flujos	<i>Indicar la secuencia de actividades de los sub-flujos si los hubiera</i>
Flujos Alternativos	<i>Indicar la secuencia de actividades de los flujos alternos si los hubiera</i>
Puntos de extensión	<i>Indicar los puntos de extensión si los hubiera. Aquí se invocan a casos de uso extendidos</i>
Requerimientos Funcionales Asociados	<i>Hay que indicar qué requerimientos funcionales son atendidos por el presente CU</i>
Requisitos especiales	<i>Indicar si existen otros requisitos a considerar.</i>
Prototipos	
<i>Agregar los prototipos elaborados en función a la ECU</i>	
	

Figura 105: Plantilla para Especificación de Caso de Uso (ECU)

Fuente: Tomado del manual de ADSI del 2019

3.3.5. Priorización de casos de uso

Es la actividad de arquitectura y planificación de proyecto el cual consiste en clasificar los casos de uso según su importancia para establecer el orden de realización de estos. En este sentido, los casos de uso con significado arquitectónico se identifican y se priorizan. Una vez que se han priorizado los casos de uso, se puede decidir el orden de desarrollo del sistema.

Se establecen períodos, ciclos o iteraciones de trabajo para desarrollar la realización de los casos de uso. Se distribuyen los casos de uso en cada ciclo de trabajo; los casos de uso primarios deben realizarse en primer orden y, luego, los secundarios. Los casos de uso opcionales se deben dejar para el final de la realización.

El propósito de otorgarles prioridad a los casos de uso es identificar los casos de uso primarios para la presente etapa de desarrollo del proyecto. Según estos criterios, se determinan los casos de uso críticos para especificarlos en esta etapa del proyecto.

Dar prioridad permitirá darle la debida atención (y con más tiempo) a los casos de uso más complejos e importante. Asimismo, al priorizar se distingue a los casos de uso críticos o primarios de los secundarios. Los criterios utilizados para determinar cuáles son primarios y cuáles son secundarios son los siguientes:

- Nivel crítico (o primario): Agrupa a los casos de uso que tienen que ver con las funciones básicas del sistema.
- Nivel de baja importancia (o secundario): Agrupa a los casos de uso que tienen que ver con las funciones de soporte del sistema y que no representan mayor riesgo para el proyecto.

En cuanto a los factores considerados en la priorización, se toman en cuenta pesos, que representan porcentajes, esto es, para cada caso de uso. Los valores que pueden tomar los factores están en la escala del 1 al 10, donde 1 indica menor importancia, y 10, mayor importancia. Se considerarán primarios a aquellos casos de uso que tengan un puntaje mayor a 6.5, ya que esto significa que superan el 65% de prioridad en el sistema (PONDERACIÓN). A continuación, se mencionan las características que se ponderan:

- Importancia en el proceso del negocio: Indica la relevancia que tiene la funcionalidad con el proceso. Su ponderación es 0.4.
- Complejidad de desarrollo: Indica la dificultad que se percibe del caso de uso, en su análisis, diseño, implementación, pruebas e integración. Su ponderación es 0.3.
- Riesgo asociado: Indica la relación que se percibe entre el caso de uso y la lista de riesgos. Un alto valor en este factor indica que el caso de uso tiene bastantes riesgos o riesgos de alto valor asociados. Los casos de uso con alto valor en este factor pueden ser considerados primarios debido a que deben ser enfrentados en las etapas iniciales. Su ponderación es 0.2.
- Impacto de los requerimientos técnicos: Indica cómo afectan los requerimientos técnicos al proceso del negocio y en qué manera el caso de uso se vería comprometido. Su ponderación es 0.1.

CASO RESUELTO: SIRI TOURS

La agencia de viajes SIRI TOURS es una agencia que se dedica a la venta de pasajes, reserva de hoteles y paquetes de viajes. La venta de pasajes inicia cuando un cliente se acerca al counter de atención. Una vez dentro, solicita un ticket de atención y espera su turno de ser atendido. Llegado su turno, se acerca a la ventanilla y pregunta los horarios que hay disponibles para un determinado destino. El asistente de ventas le informa sobre los horarios disponibles y los tipos de servicio que contiene cada uno de ellos, pudiendo ser: económico, ejecutivo, y vip. El cliente selecciona uno de los horarios ofrecidos y le indica al asistente de ventas el número de pasajes que desea comprar. El asistente de ventas realiza la reserva y la confirmación de esta. El asistente le pregunta al cliente si existe algún tipo de restricción con relación a la comida para los pasajeros. Si las hay, el asistente registra las restricciones y luego solicita la modalidad de pago. El cliente procede a indicar la modalidad y a realizar el pago. El asistente procede a emitir el comprobante de pago y luego a la impresión de los pasajes. El cliente recibe los pasajes y procede a retirarse del counter.

ECUN: Venta de Pasajes

Flujo Básico

1. El cliente se acerca a ventanilla y pregunta los horarios que hay de salida para su destino deseado.
2. La asistente de ventas informa sobre los horarios y tipos de servicios que tienen cada uno de ellos.
3. El cliente selecciona uno de los horarios y servicios ofrecidos, indicando el número de pasajes deseados.
4. La asistente de ventas procede a reservar y confirmar los pasajes requeridos.
5. La asistente de ventas pregunta sobre algún tipo de preferencia y/o restricción de los pasajeros en la alimentación a brindarse durante el servicio.
6. El cliente indica preferencias y/o restricciones en la alimentación de cada uno de los pasajeros para los que está adquiriendo sus boletos.
7. La asistente de ventas procede a registrar las preferencias y/o restricciones indicadas.
8. La asistente de ventas informa el monto a pagar y pregunta modalidad de pago que prefiere el cliente.
9. El cliente indica modalidad de pago deseada y procede a realizar el pago.
10. La asistente de ventas procede a emitir los pasajes solicitados.
11. El cliente recibe los pasajes con la conformidad de los mismos y finaliza el proceso.

Ejercicio: Elabore la ECU de la ECUN “Venta de pasajes”.

Solución

ESPECIFICACIÓN DE CASOS DE USO	
Nombre	Registrar venta de pasaje.
Actores	Vendedor.
Propósito	El objetivo del CU es permitir registrar la compra de uno o más pasajes.
Breve descripción	El CU permite a un vendedor registrar la venta de uno o más pasajes para uno o más personas.
Precondición	El vendedor debe haber sido validado como usuario del sistema.
Poscondición	El vendedor ha registrado la venta de uno o más pasajes dentro en el sistema.
Evento disparador	El caso de uso inicia cuando el vendedor pulsa el botón “Nueva Venta”.
Flujo Básico	1. El sistema muestra formulario de registro de venta. 2. El vendedor ingresa los datos necesarios para la búsqueda de pasajes (lugar de partida, fecha de partida, lugar de retorno, fecha de retorno, número de pasajeros). 3. El vendedor pulsa el botón “Buscar”. 4. El sistema muestra el listado de vuelos disponibles que cumplen con los criterios de búsqueda. 5. El vendedor selecciona el vuelo deseado y pulsa el botón “Continuar”. 6. El sistema muestra listado de vuelos de partida y el costo asociado a cada uno de ellos. 7. El vendedor selecciona el vuelo de partida y pulsa el botón “Continuar”. 8. El sistema muestra listado de vuelos de retorno y el costo asociado a cada uno de ellos. 9. El vendedor selecciona el vuelo de retorno y pulsa continuar. 10. El sistema muestra formulario para registrar los datos personales del pasajero (DNI, nombres, apellidos, dirección, teléfono, email). 11. El vendedor registra los datos solicitados por cada pasajero solicitado. Al finalizar pulsa el botón “Pagar”. 12. El sistema muestra formulario para el registro del pago. 13. El vendedor registra los datos de la tarjeta de crédito con la que se efectúa la compra y pulsa el botón “Finalizar”. 14. El sistema muestra información de resumen de la compra realizada y solicita al vendedor confirmar la compra. 15. El vendedor confirma la compra pulsando el botón “Aceptar”. 16. El sistema registra la compra y envía confirmación por correo electrónico al vendedor y a cada uno de los pasajeros. El Caso de Uso termina.
Subflujos	Ninguno.
Flujos Alternos	En el paso 16: Si se presentan problemas en el registro de la compra el sistema muestra mensaje de error: “Problemas durante el registro de pago. Favor de contactarse con su banco”.
Puntos de extensión	Si la facturación se debe realizar a otra persona, el vendedor selecciona la opción “Cambiar datos de facturación”, inmediatamente el sistema invoca al caso de uso “Registrar datos de Facturación”.
Requerimientos Funcionales asociados	RF008: El sistema debe permitir al Vendedor registrar la venta de uno o más pasajes. RF010: El sistema debe permitir al Vendedor que los datos de facturación estén a nombre de una persona diferente a la de los pasajeros.
Requisitos especiales	Ninguno.

ESPECIFICACIÓN DE CASOS DE USO

Prototipos

[Menú](#)
lastminute.com

[Contacta con nosotros](#)
[Ayuda ?](#)

[Vuelos](#)
[Hotel](#)
[Vuelo + Hotel](#)
[Fin de semana](#)
[Vacaciones](#)
[Escapadas](#)
[Cruceros](#)
[Último Minuto](#)
[Coches](#)
[Apartamentos + Casas](#)

Hoteles
Vuelos
Vuelo + Hotel

Ida y Vuelta
Sólo ida

Origen

Destino

Fecha de ida

Fecha de vuelta

Lima J Chavez Internacional - Punta Cana Página 1 de 8 | siguiente »

Ordenar por: [H. Salida](#) [H. Llegada](#) [Escalas](#) [Duración](#) [Origen](#) [Destino](#) [Precio por persona](#)

COPA Turista							
→	dom 28/02	06:48 Lima J Chavez Interna...	1	15:37 Punta Cana	07h49	9	494€ CONTINUAR
←	mié 09/03	06:15 Punta Cana	1	15:20 Lima J Chavez Interna...	10h05	9	
Detalles							

COPA Turista							
→	dom 28/02	04:49 Lima J Chavez Interna...	1	15:37 Punta Cana	09h48	4	494€ CONTINUAR
←	mié 09/03	06:15 Punta Cana	1	15:20 Lima J Chavez Interna...	10h05	9	
Detalles							

COPA Turista							
→	dom 28/02	16:41 Lima J Chavez Interna...	1	11:19 (+1) Punta Cana	17h38	9	494€ CONTINUAR
←	mié 09/03	06:15 Punta Cana	1	15:20 Lima J Chavez Interna...	10h05	9	
Detalles							

COPA Turista							
→	dom 28/02	16:41 Lima J Chavez Interna...	1	12:54 (+1) Punta Cana	19h13	9	494€ CONTINUAR
←	mié 09/03	06:15 Punta Cana	1	15:20 Lima J Chavez Interna...	10h05	9	
Detalles							

Figura 106: Prototipo de Caso de Uso
Fuente: Tomado del manual de ADSI del 2019

CASOS PROPUESTOS

RESTAURANTE EL TOYITO

Atención de Clientes

El proceso general de Atención de cliente cuenta con varios subprocesos que son efectuados por: El Maitre, los mozos, el cantor, el cocinero y el cajero. El cliente es recibido por el Maitre quien pregunta si cuenta con Reserva, verifica su Libro de Reservas y coloca la Reserva como Realizada. De no contar con una reserva, el Maitre consulta la disponibilidad de las mesas en su Mapa de Mesas y asigna una mesa disponible del comedor al comensal.

El Maitre conducirá al cliente a la mesa, asigna a un mozo para la atención y deja una Carta/Menú al Cliente. El cliente consulta la carta/menú y hace su pedido al Mozo. El mozo anota el pedido del cliente en un documento llamado Comanda y entrega una copia a la cocina.

El Cantor revisa la Comanda con el pedido y distribuye: Si el pedido incluye bebidas gaseosas, las despacha personalmente, si el pedido es de platos para preparar “canta” la orden al cocinero quien prepara el mismo, una vez que en la cocina es preparado el pedido, el cantor coloca un visto en la comanda en señal de conformidad como pedido preparado.

El mozo recibe los platos ubicados en un mostrador y sirve los mismos al cliente, el mozo visa el original de la comanda como pedido atendido y entrega una copia al cajero para que posteriormente genere la cuenta del cliente.

El cliente consumirá el plato servido y podrá repetir el proceso de pedido en varias oportunidades (se generará una comanda por cada pedido extra). Una vez terminado su consumo solicitará la cuenta al Mozo, éste le traerá un documento llamado Adelanto de Cuenta, el cual tendrá el detalle de lo consumido, el cliente lo verifica y de ser conforme procederá a realizar el proceso de Cancelación del Consumo al Mozo asignado.

Cobranza por Servicios

El cajero consulta el pedido (que normalmente contiene varias comandas) para obtener el monto a cobrar. Una vez realizado el pago por el cliente, el cajero genera el documento, registra el pago y le entrega al mozo el documento con el sello de cancelado.

El mozo se encargará de llevar a la mesa tanto el documento (Boleta o Factura) como el vuelto respectivo, cerrando así el proceso de cobranza y el proceso de Atención.

ECUN: Atención de Clientes**Flujo de Trabajo****Flujo Básico**

1. El caso de uso inicia cuando el cliente llega y solicita servicio.
2. El maitre recibe al cliente y le pregunta si cuenta con reserva.
3. Si el cliente cuenta con reserva, el maitre verifica su Libro de reservas y coloca la reserva como realizada.
4. El maitre conducirá al cliente a su mesa.
5. El maitre le ofrece algo de beber y si el cliente desea, el maitre llena la comanda y la lleva al bar y a caja.
6. El maitre asigna un mozo para la atención.
7. El mozo lleva las bebidas a la mesa del cliente y le entrega la carta/Menú y el proceso termina.

Flujo Alternativo

1. En el punto 3 del flujo básico, si el cliente no cuenta con reserva. El maitre consulta la disponibilidad de las mesas en su Mapa de Mesas y asigna una mesa disponible del comedor y continua al paso 1.1.4.
2. En el punto 5 del flujo básico, si el cliente no desea aún algo de beber, el maitre le entrega la carta/menú, asigna un mozo para la atención y el proceso termina.

Ejercicio:

1. Utilizando la plantilla proporcionada elabore la ECU del ECUN “Atención de clientes”.
2. Elabore el Modelo de Casos de Uso.
3. Elabore el Diagrama General de Casos de Uso Estructurado.
4. Elabore la ECU de una funcionalidad de nuestra Intranet Sócrates.

Resumen

1. En un Diagrama de Casos de uso Estructurado puede presentarse hasta tres tipos de relaciones: “include”, “extend” y “generalización”.
2. En una relación de generalización, el caso de uso hijo hereda la estructura, comportamiento y las relaciones del padre.
3. En una relación include, el caso de uso incluido encapsula el comportamiento necesario y es reutilizado por varios casos base
4. En una relación extend, el caso de uso extendido encapsula el comportamiento opcional de un caso base.

Recursos

Pueden revisar los siguientes enlaces para ampliar los conceptos vistos en este tema:

- https://cgrw01.cgr.go.cr/rup/RUP.es/SmallProjects/core.base_rup/workproducts/rup_use_case_model_EF15E534.html

Bibliografía

- Ambler, S. W. (13 de mayo de 2006). *The Agile Unified Process (AUP)*. Recuperado el 15 de noviembre de 2020 de www.ambysoft.com/unifiedprocess/agileUP.html
- Ambysoft (2006). *Introducción al Agile UP*. Recuperado de <http://www.cc.una.ac.cr/AUP/html/overview.html>
- Ambysoft (2021). [Visión General de la Metodología AUP]. Recuperado de www.ambysoft.com/
- Báez Pérez, C. I. (2014) *Proceso de desarrollo de software: basado en la articulación de RUP y CMMI priorizando su calidad*. Tunja : Universidad de Boyacá.
Centro de Información: Código 005.14 BAEZ.
- Bruegge, B. y Dutoit, A. (2002). *Ingeniería de software orientada a objetos*. México, D. F. : Pearson Educación.
Centro de Información: Código 005.117 BRUE
- Debrauwer, L. (2016). *UML 2.5: iniciación, ejemplos y ejercicios corregidos. (4a ed.)*. Barcelona: ENI.
Centro de Información: Código 005.117 DEBR 2016
- IBM Corporation (2020). IBM Engineering Systems Design Rhapsody. Recuperado de <https://www.ibm.com/pe-es/products/systems-design-rhapsody>
- Kendall, K. E. y Kendall, J. E. (2011). *Análisis y diseño de sistemas. (8a ed.)*. México, D. F. : Pearson Educación.
Centro de Información: Código 004.2 KEND 2011
- Lappinf, A. (2019). *Getting started with Rhapsody for systems engineering*. Recuperado de <https://www.youtube.com/watch?v=j2pglKC5f7U>
- Larman, C. (2003). *Uml y patrones. (2a ed.)*. Madrid : Pearson Educación.
Centro de Información: Código 005.117 LARM 2003
- Lasa Gómez, C. (2017). *Metodologías ágiles : Scrum, Kanban, Lean*. Madrid : Anaya Multimedia.
Centro de Información: Código 005.1 LASA
- MK Labs (2021). *Star UML*. Recuperado de <https://staruml.io/>
- Object Management Group Unified (2021). *Unified Modeling Language*. Recuperado de <https://www.uml.org/>

- Pressman, R. S. (2010). *Ingeniería del software : un enfoque práctico. (3a ed.)*. México, D. F. : McGraw-Hill.
Centro de Información: Código 005.1 PRES 2010.
- Rational Software Corp. (2001). *Rational Unified Process : overview*. Recuperado de <https://scweb.uhcl.edu/helm/RationalUnifiedProcess/>
- Sommerville, I. (2011). *Ingeniería de software. (9a ed.)*. México, D. F. : Pearson Educación.
Centro de Información: Código 005.1 SOMM/ES 2011.
- Sparx Systems (2021). *Enterprise Architect*. Recuperado de <https://sparxsystems.com/>
- Stevens, P. y Pooley, R. (2002). *Utilización de UML en ingeniería del software con objetos y componentes*. Madrid : Pearson Educación.
Centro de Información: Código 005.117 STEV
- Tutorials Point (2021). *Software - CASE Herramientas*. Recuperado de https://www.tutorialspoint.com/es/software_engineering/case_tools_overview.htm
- Watson, A. (23 de marzo de 2016). *Visual Modeling : past, present and future*. Recuperado el 15 de noviembre de 2020 de https://www.uml.org/Visual_Modeling.pdf